The Arithmetic of Intelligence

A plain-English walk through every mechanism inside a modern language model, in C++

A companion textbook for `minai.cpp` and `maxai.cpp`.

For anyone curious about how modern large language models — ChatGPT, Claude, Gemini, Llama, and every other generative AI — actually work. Not as metaphor, not as vibe, but as specific arithmetic you could read and, in principle, type out yourself. No prerequisites beyond comfort with high-school algebra and a willingness to look at code; every more advanced concept is derived on the spot when it is needed.

By the end of Section 1 you will understand the mechanism. By the end of Section 2 you will have watched a companion program grow from a 540-line digit-reverser (`minai.cpp`) into a simplified GPT (`maxai.cpp`) that accepts a prompt string, tokenizes it through a learned byte-pair subword vocabulary, runs a KV-cached autoregressive forward pass, samples with temperature / top-k / top-p, and generates text — still one C++ file, still runnable on a laptop. By the end of Section 3 you will know where the field is going, what work remains to be done, and why there has never been a better moment to be an engineer who understands how all of this actually works. If you have ever wanted to go from "LLMs are magic" to "LLMs are a specific kind of program I understand — and here is where I want to work in the field that builds them," this book is for you.

Preface

This is a book about what is really inside ChatGPT, Claude, and every other large language model. There is a lot of breathless coverage of these systems in the press, a lot of hand-waving explanations in popular science articles, and a lot of very-technical papers written for specialists. What there is much less of is plain, honest, step-by-step explanation aimed at a curious person who wants to understand the actual mechanism — not a metaphor for the mechanism, but the mechanism itself.

That is what you are holding. The book comes in three sections, each with its own companion program or lack thereof.

Section 1 is the introduction. The program at your side (`minai.cpp`) is the smallest possible version of a real language model — about 540 lines of C++ that learn to reverse a short list of digits. Every mechanism it uses is a mechanism inside every large language model on earth. Fifteen short chapters take you through what each part of that program does and why. If you want the simplest possible complete picture of how an LLM works, Section 1 alone is enough.

Section 2 is the textbook. It turns `minai.cpp` into `maxai.cpp`, a larger companion that accepts a prompt string, tokenizes it through a learned byte-pair subword vocabulary, runs a KV-cached autoregressive forward pass, samples tokens with temperature / top-k / top-p, stops on an end-of-sequence symbol, and generates

text — a simplified GPT, end to end, still in one C++ file that compiles on a laptop. Seven chapters walk the growth one idea at a time, with running measurements of loss, information-theoretic ceilings, generation quality, and inference speedups. By the end of Section 2 you will have watched the same program you started with recite the opening of the Gettysburg Address from memory, sample plausibly-varied continuations of a prompt, and run roughly 12× faster under a KV cache than without.

Section 3 is the field. It is about what to do with this understanding: why software engineering as a career is not ending, how to use AI tools competently, what open problems are being worked on in hardware/memory/efficiency and in safety/security/alignment, what the 2026 research frontier looks like, and where a curious new engineer might fit in. Six short chapters — less code-focused, more career-focused — for anyone who wonders whether the ground under them is about to give way. It is not. Section 3 has no new code; it is built on what the first two sections already showed you.

The whole-book promise: when you finish the final chapter, the only difference between what you understand and what GPT-4 does will be *quantity* — more layers, more heads, more parameters, more training data, more compute. Not one new idea. Every chapter is earnings on that promise.

This book is deliberately simple — especially in Section 1, where every optional complication has been cut. If a concept can be explained with a small number and a paragraph of plain English, that is how it is explained. If a formula is needed, it is derived in place rather than referenced. The code is presented alongside the prose; in Section 1, reading both at once is the point. In Section 2 the prose carries a bit more weight and the chapters are longer; the code stays in its file (`maxai.cpp`) and the book tells you when to glance over at it. In Section 3 there is no code at all; the chapters are short essays on what the previous two sections prepare you to do.

Who this is for

I wrote this for a particular kind of reader — someone who took calculus at some point but was never told *what calculus is for*. You may have computed some derivatives, solved some integrals, and then forgotten the whole thing because nothing you do day-to-day involves infinitesimals. That is fine. Across this book I assume:

- You are comfortable with functions: `y = f(x)`, `y = 3x + 2`, that kind of thing.
- You have seen derivatives (slope of a curve). You may not remember how to compute one by hand; I will re-derive the few we need.
- You have seen vectors — an arrow, or equivalently an ordered list of numbers like `(1, 4, -2)`.
- You have seen matrices, or you will trust me for a paragraph while I remind you.
- You can read code; you do not need to know C++ specifically.

Every concept beyond this is built from scratch.

How to use this book

`minai.cpp` is a complete, self-contained C++ transformer. It lives in the same directory as this file. Build it and run it first:

```
make
./minai
```

It prints a training log and a demo. That is the thing this book explains. Every chapter points to specific parts of `minai.cpp`. Read the code alongside — not to memorize, but to see each concept *running as code* the moment you read about it.

Roadmap

- Chapters 1–3 set up the problem: what a model is, what "learning" means, and the minimum linear-algebra vocabulary we need.
- Chapters 4–7 describe the **forward pass** of a transformer: text → numbers → attention → probabilities.
- Chapters 8–10 describe **training**: the loss function, the calculus (finally), the weight-update loop.
- Chapters 11–12 cross the bridge from math to hardware: how abstract calculus becomes matrix multiplication, and why GPUs and their memory have become the limiting resource of the entire AI industry right now.
- Chapter 13 is the view from the top: here is what GPT-4 adds on top of what you just read.
- Chapter 14 is a hands-on flag-by-flag walkthrough that lets you watch each concept fire on your own machine.
- Chapter 15 covers the one big preprocessing step MinAI skips entirely: how real LLMs turn arbitrary text into the integer token IDs a transformer can consume (BPE, SentencePiece, pretrained encoders like DeBERTa, vector databases).
- Appendix A is a side story about doing all of this in integer arithmetic on a 1970s minicomputer.
- Appendix B is a set of exercises.

There is no math in this book you could not do on paper, given a quiet afternoon. There is nothing mysterious about what the code is doing. I promise.

Chapter 1 — What a model is

Forget machine learning for a moment. You have been working with "models" since middle school. Here is one:

```
y = 3x + 2
```

Given an input `x`, it produces an output `y`. The numbers `3` and `2` are **parameters** — they are what make this model *this* model rather than some other line like `y = 5x - 1`. Change the parameters, and you have a different function.

This is a tiny model. Two parameters. It can represent any line; nothing more.

It is worth stating the through-line of the book up front. **The same recipe we are about to build for these two parameters works for two thousand, two million, or two trillion parameters, and every scale in between.** A small handwriting recognizer might have around a million. GPT-3 has 175 billion. GPT-4 is estimated at roughly a trillion. The *recipe* does not change. What grows is the amount of arithmetic required to run or train the model. You will see the same building blocks — a lookup table here, a matrix multiply there, a softmax, a residual connection — stack up identically whether there are ten of them or a hundred billion.

What "learning" means

Suppose I give you the following data:

```
 x = 1, y = 5
x = 2, y = 8
x = 3, y = 11
```

You spot immediately that `y = 3x + 2` fits. You just *learned* that model from data. To a human, the process is visual: plot the points, eyeball the line, write down the equation. To a computer, the process has to be mechanical:

1. **Guess** parameter values, maybe randomly.
2. **Measure** how badly your current guess matches the data.
3. **Nudge** the parameters in a direction that reduces the badness.
4. Repeat steps 2–3 until the badness is small.

That four-step recipe is the *entire* theory of how neural networks learn. It is called **gradient descent**, and once you see it, you will see it everywhere.

Step 3 is the hard part: "nudge them in a direction that reduces the badness." What does *direction* mean for numbers? How do you pick it? You need to know the **slope** of the badness function at your current point — because slope tells you which way is up (and therefore which way is down). Slopes are what derivatives give you. That is what calculus is for, in one sentence. We will spell it out in Chapter 2.

Why a machine needs this machinery

For `y = 3x + 2`, you could just solve by hand. Why build the four-step machinery? Because it *keeps working* when the model has a million parameters instead of two, and when there is no clean formula waiting to be guessed. The function that maps "picture of a cat" → "the label 'cat'" has no closed-form expression. The function that maps "first half of a sentence" → "next word" has no closed-form expression either. But both functions can be **approximated** by setting up a model with millions of parameters and running gradient descent until the badness is low.

`minai.cpp` has 2,240 parameters by default. It maps an 8-digit input to an 8-digit output. A closed-form rule exists ("reverse the list"), but the model doesn't know that. Training has to find, from scratch, parameter values that encode that rule. If you can accept that training a 2,240-parameter model to reverse digits works

by the *same four-step recipe* as training a 175-billion-parameter model to write sonnets, you're well on your way.

> **Read along:** Open `minai.cpp` and look at **Part 4 (Model Parameters)**. Count the float arrays declared as globals: `token_emb`, `pos_emb`, `Wq`, `Wk`, `Wv`, `W1`, `W2`, `Wout`. Each entry in each array is one of the parameters gradient descent will adjust. The total matches the "Parameters: 2240" number the program prints at startup.

Chapter 2 — Why calculus, finally

Imagine you are in a hilly landscape, in thick fog, and you want to reach the lowest point. You cannot see far enough to know where the valley is. What *can* you do? You can feel the slope of the ground right under your feet. If the ground slopes down to the north, you step north. A few feet later you check again: maybe now it slopes down to the northeast. You adjust and step again. Repeat. If the landscape is reasonably well-behaved, you will eventually reach a low point — not necessarily the global minimum of the map, but *a* local minimum good enough for most purposes.

That is gradient descent in one paragraph. Everything else is details.

The "badness function" has a shape

Back to our line-fitting problem: given data `(x_i, y_i)` for `i = 1..N`, and a model `f(x) = mx + b` with parameters `(m, b)`, define the **loss** (the technical word for badness) as the average squared error:

```
L(m, b) = (1/N) * Σ_i ( y_i − (m*x_i + b) )^2
```

This is a function of `(m, b)`. Pick any two numbers, plug them in, get a single number out (the loss). The space of possible `(m, b)` values is just a 2D plane, and `L(m, b)` is the height of the "landscape" above each point. Our goal is to find the `(m, b)` at the bottom of that landscape.

Now: what does a derivative *tell you*?

For a function of one variable `f(x)`, the derivative `f'(x)` at a point tells you how much `f` changes per small change in `x`. If `f'(5) = 3`, it means: right around x=5, moving x up by a tiny bit Δ makes f go up by about 3Δ. If `f'(5) = −7`, moving x up makes f go down (by about 7Δ). In both cases, the derivative is the *slope* at that point.

For a function of **two** variables `L(m, b)`, the generalization is a **gradient** — a little arrow with two components, one for each variable:

```
∇L = ( ∂L/∂m , ∂L/∂b )
```

That notation looks scary. It just means: the first number is “how much does L change per unit change in m, holding b fixed”, and the second is the mirror image for b. Together, the two numbers point in the direction where L goes *up* the fastest. Flip the sign and you get the direction where L goes *down* the fastest. That is the direction to step.

The update rule

Given a small positive number `α` (the **learning rate**, a knob you pick):

```
 m := m − α ∗ ∂L/∂m
b := b − α ∗ ∂L/∂b
```

That is it. That is gradient descent. Compute the gradient, take a small step opposite to it, repeat. After thousands of iterations, `(m, b)` will settle near the minimum of `L`, which is exactly the parameter values that best fit the data. The calculus course you took gives you the tool (derivatives) to compute that direction.

Let’s actually do one step by hand. Use the three data points `(1, 5), (2, 8), (3, 11)` from earlier, and start at a deliberately bad guess: `(m, b) = (0, 0)`. The model’s predictions are `(0, 0, 0)`; the targets are `(5, 8, 11)`; every error is big and positive. Working through the two partial-derivative sums (whose derivations are short and worth doing on paper if you are rusty):

```
 ∂L/∂m = −(2/3) × (1·5 + 2·8 + 3·11) = −(2/3) × 54 = −36
∂L/∂b = −(2/3) × (5 + 8 + 11)       = −(2/3) × 24 = −16
```

Both gradients are negative, which means: loss would *go down* if we increased `m` or `b`. Flip the signs (because we step *opposite* to the gradient) and apply the update rule with a modest learning rate `α = 0.01`:

```
 m := 0 − 0.01 × (−36) = 0 + 0.36 = 0.36
b := 0 − 0.01 × (−16) = 0 + 0.16 = 0.16
```

One step, and both parameters have moved in the right direction (the true answer is `m = 3, b = 2`). Do this a few hundred times and the model converges. That iterative refinement — compute gradient, take small step in the opposite direction, repeat — is the entire training algorithm, at every scale, for every model you will ever meet. A trillion-parameter model does exactly this, a few hundred million times. The only thing that differs is how many partial derivatives you compute in each step and how fast the hardware can do it.

Now plug in a real number of parameters: 2,240 for MinAI, 175 billion for GPT-3. Same procedure. The gradient has 2,240 components; you step opposite to it. Nothing conceptually harder. The only new problem is *how to compute the gradient efficiently* when the function is a huge composition of matrix multiplies — and that is what Chapter 9 is about.

> **Read along:** In `minai.cpp`, **Part 11 (Optimizer)** is three loops that implement exactly this update rule: `parameter -= learning_rate * gradient_of_parameter`. Three lines of "machine-learning theory" in the middle of each loop. That is the whole optimizer.

Chapter 3 — Vectors and matrices, the bare minimum

Models with billions of parameters need more structured representations than single numbers. You have to upgrade from scalars (single numbers) to **vectors** (ordered lists of numbers).

Vectors

A vector is an ordered list of numbers:

```
v = (0.3, -1.2, 0.8, 0.05)
```

This particular vector has **dimension** 4 because it has four components. In MinAI, `D_MODEL = 16`, so every position in the sequence is internally represented by a 16-dimensional vector.

You can add two vectors of the same dimension: add them component by component. You can multiply a vector by a scalar: multiply each component. The operation we care about most is the **dot product** of two vectors of the same dimension:

```
 a = (a_0, a_1, a_2, a_3)
b = (b_0, b_1, b_2, b_3)
a · b = a_0*b_0 + a_1*b_1 + a_2*b_2 + a_3*b_3
```

Multiply componentwise, sum the products. One single number comes out.

The dot product has a geometric meaning: it measures how *aligned* two vectors are. If a and b point in the same direction, the dot product is large and positive. If they are perpendicular, it's zero. If they point opposite, it's negative. Formally: `a · b = |a| |b| cos(θ)`, where `θ` is the angle between them.

Hold onto that intuition. In Chapter 5 you will see the machine compute one of these for every pair of tokens in the sequence, and interpret it as "how well does this position's query match that position's advertised key". Dot product = similarity is the whole mental model.

Matrices

A matrix is a rectangular grid of numbers. A matrix `W` with `m` rows and `n` columns can be thought of either as "a collection of `m` vectors of dimension `n`" or "a collection of `n` vectors of dimension `m`". Same numbers, two readings.

The operation we need is **matrix–vector multiplication**. If `W` is `m × n` and `v` is a vector of dimension `n`, then `W * v` is a vector of dimension `m`, and its `i`-th component is the dot product of row `i` of `W` with `v`:

```
(W * v)_i = Σ_j  W[i][j] * v[j]
```

A quick concrete example to make the formula less abstract. Let `W` be a 2 × 3 matrix and `v` a 3-vector:

```
W = [[1, 2, 3],       v = [10, 20, 30]
     [4, 5, 6]]
```

Then `W * v` is a 2-vector. Each entry is one dot product:

- First entry = dot(row 0 of W, v) = `1·10 + 2·20 + 3·30 = 140`
- Second entry = dot(row 1 of W, v) = `4·10 + 5·20 + 6·30 = 320`

So `W * v = [140, 320]`. Three multiplications and two additions per row, times two rows. Six multiply-adds total. At the scale of real neural networks, a matrix multiplication is still exactly this, just with a lot more rows and columns — millions of multiply-adds per call rather than six, but the arithmetic pattern is identical.

Matrix–matrix multiplication is the same thing repeated: for each column of the second matrix, multiply it by the first. The product has shape `(rows-of-first) × (columns-of-second)`.

Think of a matrix multiplication `Wv` as: "I am running `v` through a bunch of dot products (one per row of `W`), and producing a new vector where each component is one of those similarity scores." That's the mental picture. A neural-network "linear layer" is exactly one of these multiplications (plus sometimes a bias added at the end, though MinAI skips biases for simplicity).

> **Read along:** `minai.cpp` Part 8, **step 2** of `forward_block`. The nested loops compute `Q = X @ Wq`, `K = X @ Wk`, `V = X @ Wv`. Three matrix multiplies. Each innermost `for (int i = 0; i < D_MODEL; ++i)` loop is one dot product. Look at them and trust that you understand the bones of the arithmetic.

Chapter 4 — Tokens, embeddings, and position

A language model takes in symbols (words, digits, characters) and produces more symbols. But a neural network does math on vectors, not symbols. How do we bridge that gap?

The vocabulary

First, we decide on a finite alphabet of symbols the model knows about. In MinAI, the alphabet is just the ten digits `0..9`. We call this set the **vocabulary**, size `VOCAB = 10`. In GPT-3, the vocabulary is about 50,000 "subword" pieces (not whole words, and not letters — chunks like `"berg"` or `" the"`). The tokenization story is a whole other book; for us, one digit is one token.

Each symbol is assigned an integer **token id** — digit 0 gets id 0, digit 5 gets id 5, etc. GPT-3's tokenizer has a more complex assignment but does the same job.

Embeddings: turn token ids into vectors

Now for the bridge. We keep a lookup table `token_emb` of shape `[VOCAB][D_MODEL]`. That is, 10 rows (one per token) and 16 columns (the vector dimension). The `t`-th row of this table is the vector the model will use whenever it encounters token `t`. This table is **learned** — it starts out random and gradient descent adjusts it over training until each row becomes a useful encoding of the corresponding token.

Here is what the table might look like at initialization, with random small values (only the first four columns shown for brevity):

```
 token 0:  [ 0.07, -0.03,  0.04, -0.05, ... ]
token 1:  [-0.02,  0.04,  0.07, -0.06, ... ]
token 2:  [ 0.01,  0.06, -0.08,  0.02, ... ]
...
token 9:  [ 0.09,  0.02, -0.03,  0.05, ... ]
```

After thousands of training steps, the values in these rows will have shifted to whatever configuration makes the model's predictions most accurate. For our digit-reversal task the exact learned numbers are not humanly interpretable — the model just needs them to be distinct enough that it can tell the ten digits apart. But in a trained GPT-3, the 12,288-dimensional embedding for the word `dog` ends up very close (in vector-space distance) to the embedding for `cat`, and very far from the embedding for `xylophone`. Nobody coded that similarity in; it emerged from training on text where `dog` and `cat` happen to appear surrounded by similar words. That kind of emergent structure is what people mean when they say a language model "understands" a word — at base, *understanding* reduces to having a vector that sits in the right neighborhood relative to all the other words' vectors.

That is worth pausing on. *The model decides for itself how to represent the digit "3"*, by picking a 16-dimensional vector. It is not given one. It tries random ones and keeps the ones that make its predictions better. In a trained GPT-3, the embedding vectors of words with similar meaning end up close to each other in vector space — an emergent consequence of training, not a rule anyone wrote down.

Position embeddings

There is a subtlety. Look ahead to Chapter 5: attention is a sum over positions weighted by similarity. Swap the order of two input tokens, and the sum is unchanged. A pure attention model cannot tell "the cat sat on the mat" from "on the mat the cat sat." That is a problem.

Solution: keep a second lookup table `pos_emb` of shape `[SEQ_LEN][D_MODEL]`. Row `t` is a vector unique to position `t` in the input. *Add* the position vector to the token vector:

```
X[t] = token_emb[tokens[t]] + pos_emb[t]
```

Now position 0 and position 5 look different to the network even if they contain the same token. Symmetry broken.

> **Read along:** `minai.cpp` Part 8, **step 1** of `forward`. This is the literal line: `X_block[0][t][d] = token_emb[tokens[t]][d] + pos_emb[t][d];`. This is the *only place* in the whole program where raw integer token ids enter the network. Past that one line, it is all real-valued vectors.

Chapter 5 — Attention, the actual new idea

Attention is what put the "T" in GPT. Everything else in a transformer — embeddings, residuals, feed-forward networks — had existed for years before transformers. Attention is the new thing. Once you get it, the rest of the architecture falls into place.

The problem it solves

In a sentence, the meaning of a word often depends on *some* other word, not every other word. "The trophy didn't fit in the suitcase because **it** was too big." Does *it* refer to the trophy or the suitcase? To answer, you need to look at some specific other word (size-related context). A fixed recipe — "always look at the previous word" or "average everything" — won't work, because *which* other word matters depends on the meaning being computed.

Attention is a mechanism that lets each position *dynamically* decide which other positions are relevant, and pull information from them.

The soft-dictionary analogy

Imagine a dictionary lookup. You provide a key, the dictionary finds the matching entry, returns the value. That is **hard** lookup: you either match or you don't.

Attention is **soft** lookup:

1. Every position `u` in the sequence emits a **key** vector `K[u]` advertising what it has, and a **value** vector `V[u]` which is the content it would contribute if chosen.
2. When I am at position `t`, I emit a **query** vector `Q[t]` describing what I am looking for.
3. For each position `u`, I compute the **similarity** between my query and its key — a single number, the dot product `Q[t] · K[u]`.
4. I pass all those similarity scores through a softmax (Chapter 6) which turns them into a probability distribution: now they are positive and sum to 1. Call them `attn[t][u]`.
5. My new representation is the attention-weighted sum of values: `Σ_u attn[t][u] · V[u]`.

If the similarities strongly favor one position `u*`, the softmax produces something close to one-hot: 1 at `u*`, near-zero elsewhere. The weighted sum then just copies `V[u*]`. That is "hard" dictionary lookup as a limit case. With softer similarities you get a blend of multiple values. The word "soft" in "soft dictionary" means *the choice is differentiable*: you can compute how much each `Q`, `K`, and `V` should change to reduce the loss. Unlike a hard lookup, which has no derivative, this mechanism is trainable.

Where do Q, K, V come from?

From the input vector X, by three learned linear layers:

```
Q[t] = X[t] @ Wq
K[t] = X[t] @ Wk
V[t] = X[t] @ Wv
```

Each of those is a matrix multiplication: `Wq`, `Wk`, `Wv` are `D_MODEL × D_MODEL` matrices whose entries are learned parameters. So the network learns *three different views* of each input vector, one for use as a query, one as a key, one as a value. What makes a good key-view vs a good query-view? Whatever makes the final predictions better. Gradient descent sorts it out.

A tiny worked example. To make the mechanics concrete, imagine three positions with tiny 2-dimensional Q/K/V vectors (real models use 16, 64, 128 or more; the logic is identical). At position 0 we want to compute the attention output. The query is `Q[0] = [1, 0]`. The three keys are:

```
K[0] = [1, 0]
K[1] = [0, 1]
K[2] = [1, 0]
```

The three dot-product similarities are:

```
Q[0] · K[0] = 1·1 + 0·0 = 1
Q[0] · K[1] = 1·0 + 0·1 = 0
Q[0] · K[2] = 1·1 + 0·0 = 1
```

Our query is looking for things aligned with `[1, 0]`; positions 0 and 2 match; position 1 does not. Softmax over the score vector `[1, 0, 1]` gives approximately `[0.42, 0.16, 0.42]` — most of the weight lands on the two matching positions, and a small fraction on the non-match.

Now weight the values. Let `V[0] = [10, 0]`, `V[1] = [0, 10]`, and `V[2] = [-5, 5]`. The blended output is:

```
attn_out[0] = 0.42 · [10, 0] + 0.16 · [0, 10] + 0.42 · [-5, 5]
            = [4.2, 0]      + [0, 1.6]       + [-2.1, 2.1]
            = [2.1, 3.7]
```

Position 0's new representation is a mixture of what the other positions told it, weighted by how well they matched its query. If this were a real sentence, we would read `attn_out[0]` as *"position 0 pulled relevant context from positions 0 and 2 and mostly ignored position 1."* At scale, that is exactly how attention lets a pronoun at position 12 pull meaning from the noun at position 5 and ignore the filler words in between.

The scaled part

One wrinkle. When we compute `Q · K`, the numbers grow with the dimension `D_MODEL`, because we are summing `D_MODEL` products. With `D = 16` they are modest; with `D = 12,288` they are huge. Huge scores send softmax into its saturated regime (one score wins by a landslide) and gradients vanish. The fix: divide by `sqrt(D_MODEL)` before the softmax. That keeps the variance of the scores roughly 1, regardless of dimension.

This is why transformer attention is officially called "scaled dot-product attention". Without the scale, it doesn't train at large D. With it, it does.

The matrix form

We can write the whole thing as three matrix ops on the whole batch at once:

```
 scores = Q @ K^T / sqrt(D_MODEL)      # shape [SEQ_LEN, SEQ_LEN]
attn   = softmax(scores, axis=-1)     # same shape, each row sums to 1
attn_o = attn @ V                     # shape [SEQ_LEN, D_MODEL]
```

That is self-attention. `attn_o[t]` is the attention-weighted sum of values for position `t`. Do this for all `t` in parallel and you've just done one "attention layer".

> **Read along:** `minai.cpp` Part 8, **steps 3, 4, 5** of `forward_block`. The three nested-loop blocks correspond exactly to the three lines above. Look at the attention matrix the demo prints after training (in `--blocks=2` especially): rows 6 and 7 put most of their weight on column 0, which is exactly "pay attention to the right positions in order to reverse".

Chapter 6 — Softmax

After Chapter 5 we have a grid of “similarity scores” that are arbitrary real numbers — some positive, some negative. We need probabilities (positive, summing to 1) so we can interpret them as a weighting. That is what softmax is for.

The formula

```
softmax(x)_i = exp(x_i) / Σ_j exp(x_j)
```

A concrete example. If the input is `x = [2.0, 1.0, 0.1]`:

```
 exp(2.0) ≈ 7.39
exp(1.0) ≈ 2.72
exp(0.1) ≈ 1.11
sum      ≈ 11.22
```

Dividing each by the sum, `softmax(x) ≈ [0.66, 0.24, 0.10]`. All positive, all between 0 and 1, summing to exactly 1. The biggest input (`2.0`) claimed two-thirds of the probability; the middle got a quarter; the smallest got a tenth. This is the “soft” in “softmax” — the biggest input wins, but the others keep a share.

An important property worth noticing: the *differences* between inputs are what matter, not their absolute values. The softmax of `[2.0, 1.0, 0.1]` is the same as the softmax of `[100, 99, 98.1]`, because `exp(x + c) = exp(c) · exp(x)`, and the `exp(c)` factor appears in both the numerator and the denominator and cancels. This translation invariance is the basis of the “subtract the max” trick we will see below.

Three things to notice:

1. `exp(x)` is always positive. Negative inputs produce small positive outputs; positive inputs produce large positive outputs. All signs gone.
2. Dividing by the sum forces the outputs to sum to 1.
3. Exponentials *amplify* differences. If one input is meaningfully larger than the rest, it gets most of the probability. Softmax is therefore “soft argmax” — it picks the biggest input, but softly rather than making a hard choice.

Why `exp`, not some other way to make numbers positive?

You could square. You could take absolute value. Why the exponential? Two reasons.

First, `exp` has a beautiful property with calculus: `d/dx exp(x) = exp(x)`. The derivative is itself. This makes gradients of softmax clean and cheap. In Chapter 9 you will see that the combination *softmax + cross-*

entropy has a ridiculously simple derivative, and the reason is `exp` playing nicely with `log`.

Second, in the information-theoretic setup where loss is `-log(probability)`, softmax is the probability distribution that maximizes entropy for a given set of expected-value constraints. That is a deep reason but it is beyond this chapter.

The "subtract the max" trick

The formula as written has a numerical problem. `exp(100)` is about `10^43`, which does not fit in a float. If any score is that large, the whole computation overflows. But notice:

```
softmax(x + c) = softmax(x)   for any constant c
```

Why? Because `exp(x_i + c) = exp(c) exp(x_i)`, and the `exp(c)` factor appears in both numerator and denominator and cancels. So we can subtract any constant from all inputs before computing softmax, with no change to the output. In practice: subtract the max input. Now the largest value is 0, `exp(0) = 1`, and nothing overflows.

> **Read along:** `minai.cpp` Part 7, `softmax_inplace`. Three loops: find max, subtract and exponentiate, divide by sum. The subtract-max is in the second loop (`row[i] = exp(row[i] - max_val)`). This trick is not an optimization — it is correctness. Without it the program would produce `inf` on large inputs.

And see **Part 15** for the bonus: the PDP-11 had no `exp()` function at all. The original program used a 256-entry lookup table indexed by `(max - x[i]) >> 5` to produce `exp(-k/8)` in fixed-point. This is the same trick modern hardware uses to quantize LLMs for edge deployment. Appendix A has more.

Chapter 7 — Putting the block together

We now have the tools for one full forward pass of a single transformer block. Here is the whole recipe in order:

```
X                                          # input: [T, D]
│
├─→ Q = X @ Wq
├─→ K = X @ Wk                            # attention
├─→ V = X @ Wv                            # sub-layer
│   scores = Q @ K^T / sqrt(D)            #
│   attn   = softmax(scores)              #
│   attn_o = attn @ V                     #
│
```

```
H1 = X + attn_o                          # residual
│
├──→ ff_pre = H1 @ W1                    # FFN
│    ff_act = relu(ff_pre)               # sub-layer
│    ff_out = ff_act @ W2                #
│
H2 = H1 + ff_out                         # residual
```

There are three new things in this picture besides what we've already covered.

Residual connections

Notice `H1 = X + attn_o`. The attention block does **not** output a replacement for X; it outputs a *delta* that gets added to X. Why?

Consider training a 96-layer network. Early in training, the weights are random, so each layer's output is essentially noise. If each layer *replaces* its input with its output, the signal has to survive 96 rounds of random transformations, which rapidly washes it out — and no useful gradient reaches the early layers. Contrast this with the residual formulation: if each layer is initialized to near-zero, then `H1 ≈ X`, and the network starts life as (approximately) the identity function. Training then gradually teaches each layer to *modify* its input rather than replace it. The signal path from input to output always exists as a clean sum.

Residual connections are the single most important architectural invention for training deep networks. Introduced in 2015 for image classification ("ResNet"), adopted by transformers from day one.

The feed-forward sub-layer (FFN)

Attention moves information *across* positions. But some transformations happen within a single token — "if this position's vector looks like X, transform it to Y". That is what the FFN does. It is a two-layer MLP applied to each position independently:

```
ff_pre = H1 @ W1                    # expand:  D_MODEL → D_FF
ff_act = relu(ff_pre)               # nonlinearity
ff_out = ff_act @ W2                # project: D_FF → D_MODEL
```

`relu(x) = max(0, x)`. A pointwise nonlinearity. Without a nonlinearity between the two matrix multiplies, `W1` and `W2` could be combined into a single matrix (linear * linear = linear), and the FFN would have no more expressive power than a single linear layer. The ReLU breaks linearity and lets the FFN represent more complex per-token transformations.

FFN is where **most of the parameters** in a large transformer live. In GPT-3, roughly 2/3 of the parameter count is in FFN matrices. Attention gets the headlines; FFN does the bulk of the storage.

Stacking blocks

One block of (attention + FFN) is one **layer** of a transformer. Stacking `N` layers means: feed the output of block `b` into block `b+1`. The first block sees the input embedding; the last block's output goes to the final output projection. In `minai.cpp` you toggle the stack with `--blocks=N`.

Each block has its *own* copies of `Wq, Wk, Wv, W1, W2`; they are independent and learned separately. More blocks = more capacity = can represent more complicated functions, at the cost of more parameters and more training.

The output projection

After the last block, every position still has a vector of dimension `D_MODEL = 16`. We need a vector of size `VOCAB = 10` so we can ask "which digit is this?". One more matrix multiplication:

```
logits = X_final @ Wout       # [T, D_MODEL] @ [D_MODEL, VOCAB] = [T, VOCAB]
```

The output of this multiplication is called a **logit** vector (one per position in the sequence). A logit is just a raw, un-normalized score — it can be any real number, positive or negative. The word comes from "log-odds" in classical statistics, but in modern neural-network usage it has drifted to mean simply *the number you feed into softmax*. If a logit is large and positive, the corresponding digit is strongly favored; large and negative, strongly disfavored; zero, neutral. You will see the word "logit" in every paper and every library's API; it is worth memorizing.

Softmax over the `VOCAB` axis then turns each row of logits into a probability distribution over the 10 digits. Argmax of that distribution is the predicted digit.

> **Read along:** `minai.cpp` Part 8, all of `forward_block` and `forward`. You now have the vocabulary to read every line of both functions.

LayerNorm: the piece that makes deep stacks trainable

There is one more ingredient to a modern transformer block, and it is the only one that matters for training very deep networks: **layer normalization** (LayerNorm for short). It is optional in MinAI (`--layernorm=1`) but always on in real LLMs.

The problem it solves. The residual connections from earlier in this chapter help a lot, but they are not enough once you stack tens of layers. The scale of the activation vectors at each layer tends to drift — either shrinking toward zero (vanishing gradients, nothing trains) or blowing up toward infinity (`NaN`, everything dies). In MinAI you can see this happen on your own machine. Run:

```
./minai --blocks=32 --layernorm=0 --steps=200
```

Within about 150 steps the loss becomes `NaN` and accuracy sticks at 1/8. The 32-layer model's gradients exploded. Now run:

```
./minai --blocks=32 --layernorm=1 --steps=1000
```

Same architecture, same depth, same training data. LayerNorm added. Loss drops smoothly; accuracy hits 8/8 well before 1000 steps. That is the lesson in a nutshell.

What LayerNorm does. For each token's D_MODEL-wide vector, compute its mean and standard deviation *across its own components* (not across the batch, not across the sequence — just across the 16 numbers in that one vector). Subtract the mean and divide by the std so the vector has zero mean and unit variance. Then multiply and shift by two learned per-channel vectors, called **gain** and **bias**:

```
 mean  = average of x[0..D-1]
std   = std-dev of x[0..D-1]
xhat  = (x - mean) / std              # normalized: zero mean, unit variance
y     = xhat * gain + bias            # learned per-channel rescale/shift
```

The subtract-and-divide fixes the scale. The gain and bias give the model the freedom to un-do that normalization if it wants to — they start at 1 and 0 respectively, so LayerNorm is initially the identity, and training decides whether and how much to stray from it.

Where we put it. There are two conventions. In the original 2017 paper, LayerNorm came *after* each sub-layer's output (the "Post-LN" style). Modern transformers (GPT-2, GPT-3, GPT-4) place it *before* each sub-layer's input (the "Pre-LN" style):

```
 H1 = X  + Attn(LN(X))       # Pre-LN: LN before attention, residual wraps X
H2 = H1 + FFN(LN(H1))       # Pre-LN: LN before FFN, residual wraps H1
```

MinAI uses Pre-LN — it is significantly more stable for deep networks. One more LayerNorm sits just before the final output projection so the logits come out at a predictable scale.

The parameter cost is trivial. Two D_MODEL vectors per LN × 2 LNs per block + 1 final LN = 4 × D_MODEL + 2 × D_MODEL = 6 × D_MODEL per block ≈ 96 extra parameters per block in MinAI. Negligible.

Why it works (the intuition, not the proof). Normalizing the inputs to each sub-layer keeps every sub-layer operating on vectors of similar magnitude throughout training. The gradients computed backward through each sub-layer therefore also stay at similar magnitudes. Vanishing and exploding gradients, both of which are caused by scale drift compounding across many layers, are prevented by design. The formal math is a short derivation you will meet in Chapter 9 if you want it.

The takeaway. Once you add LayerNorm, the number of layers you can stack stops being limited by numerical stability and starts being limited by your patience and compute budget. GPT-3's 96 layers are only

possible because every layer is sandwiched between LayerNorms keeping the scales tame. You can reproduce that exact depth in MinAI with:

```
./minai --blocks=96 --layernorm=1 --steps=2000
```

It will run slowly because 96 blocks × 8 positions × a lot of small matrix multiplies, but it will actually train — same code path, same math, same mechanism as GPT-3. Ninety-six times. On the fixed-input task 2000 steps is plenty to hit 8/8 and plateau.

> **Read along:** `minai.cpp` Part 7b. Two functions, `layernorm_forward_row` and `layernorm_backward_row`, each about ten lines. The forward is the formula above. The backward is the derivative of the formula, derived exactly as Chapter 9 will teach you to derive things: chain rule applied one step at a time.

Chapter 8 — Loss: measuring how wrong

We can now run the model forward and get probabilities. To train the model, we need to measure how wrong those probabilities are given the correct answer, and use that measurement to drive gradient descent.

What "being wrong" feels like in probability

Suppose the correct answer is digit 7, and the model puts probability 0.8 on it. Not bad. Put 0.01 on it and you should be more embarrassed. Put 0.00001 and you should be *very* embarrassed — you have not just got the wrong answer, you were confidently wrong. A good loss function should punish confident wrongness harder than tentative wrongness.

Cross-entropy does this perfectly:

```
L = -log(p_correct)
```

where `p_correct` is the probability the model assigned to the correct answer. Run the numbers:

- `p = 1.0` → `L = 0`. Perfect.
- `p = 0.5` → `L ≈ 0.69`. Mildly off.
- `p = 0.1` → `L ≈ 2.3`. Quite wrong.
- `p = 0.001` → `L ≈ 6.9`. Very wrong.
- `p = 0` → `L = ∞`. Infinite loss for total confidence in a wrong answer.

Notice the hyperbolic curve. The `-log` penalty goes to infinity as the predicted probability approaches 0. That is the shape we want: the model is essentially *forced* to never be confidently wrong.

Why cross-entropy instead of squared error?

You could, in principle, use the squared error between the predicted distribution and a one-hot target `(0, 0, …, 1, …, 0)`. It is pedagogically tempting. But it has a specific gradient problem: when the model is *very* wrong, the squared-error gradient is *small*, because you are already far into the flat tail of a quadratic. Cross-entropy does the opposite — the wronger you are, the bigger the gradient — which makes recovery from a bad state faster.

There is also an algebraic bonus. When you stack softmax on top of logits and cross-entropy on top of softmax, a cancellation happens in the derivatives:

```
 d L(softmax(logits))
────────────────────── = probs - one_hot(target)
d logits
```

That is the entire gradient of softmax-plus-cross-entropy with respect to the raw logits. No exp, no log. Three characters of work in the code. Chapter 9 makes this explicit.

Averaging over the sequence

In MinAI, there are 8 output positions. We compute cross-entropy at each position and average:

```
L = (1/SEQ_LEN) * Σ_t (-log(probs[t][target[t]]))
```

The per-position factor of `1/SEQ_LEN` shows up in the gradient too, which is why in `backward()` you see the softmax-minus-one-hot divided by `SEQ_LEN`.

> **Read along:** `minai.cpp` Part 9, `compute_loss`. Four lines of arithmetic. The tiny `1e-12` floor on `p` is a safety net against the `log(0) = -inf` case; in practice it is never triggered during training, but it would blow up the first time a model got unlucky without it.

Chapter 9 — Calculus, the real deal: backpropagation

We have been dancing around this for eight chapters. Now we do it.

The situation

Our loss `L` is a scalar (a single number). It is computed from the parameters `(token_emb, pos_emb, Wq, Wk, Wv, W1, W2, Wout)` through a long chain of operations — matrix multiplies, softmax, residuals, ReLUs, and the cross-entropy. We want, for every parameter `θ`, the partial derivative `∂L/∂θ`. The full collection of these is the **gradient**, written `∇L`. Once we have it, gradient descent (Chapter 2) tells us the update: `θ := θ − α ∗ ∂L/∂θ`.

The challenge: there are 2,240 such partial derivatives (or 175 billion, for GPT-3). Computing each one naively from scratch would be hopeless. Enter the **chain rule**.

The chain rule, reviewed

If `L = f(g(h(x)))`, then:

```
dL/dx = f'(g(h(x))) ∗ g'(h(x)) ∗ h'(x)
```

Each factor is evaluated at the correct intermediate value. Mechanically: "derivative of the whole = product of derivatives along the path from `x` to `L`".

For composed vector functions we get the same story with matrix multiplies instead of scalar multiplies:

```
dL/dx = dL/dy ∗ dy/dx      (all matrices)
```

This is exactly what backpropagation is. It is the chain rule, applied layer by layer, from the output of the network backward to its parameters.

Two shapes of the chain rule you need

For a linear layer `Y = X @ W` (inputs `X`, weights `W`, output `Y`), given the gradient at the output `∂L/∂Y`, the gradient at the input and at the weights is:

```
 ∂L/∂X = ∂L/∂Y @ W^T
∂L/∂W = X^T @ ∂L/∂Y
```

That is a pair of matrix multiplies — the same basic arithmetic as the forward pass, just in a different shape. You can derive these by writing out one element of `Y` as a sum and applying the chain rule term by term. The result is so common in neural nets that it is worth memorizing like a phone number. You will see it four times in `backward_block` (for `Wout`, `Wq`, `Wk`, `Wv`) and twice more for the FFN (`W1`, `W2`).

The softmax + cross-entropy shortcut

Forward: `probs = softmax(logits)`, `L = -log(probs[target])`. If you grind through the chain rule using the explicit formulas for softmax and log, pages of algebra collapse to:

```
∂L/∂logits[i] = probs[i] - (1 if i == target else 0)
```

That is it. The gradient of cross-entropy through softmax is just `probs - one_hot(target)`. This is the algebraic miracle I have been teasing since Chapter 6. It is why every classifier pairs softmax with cross-entropy: the gradient computation has *no exp, no log*, just a subtraction.

A worked example: backward through one linear layer

Suppose `Y = X @ W`, with shapes:

- `X : [T, D]`
- `W : [D, V]`
- `Y : [T, V]`

Write one element of Y: `Y[t][v] = Σ_k X[t][k] * W[k][v]`. Partial derivative with respect to `W[a][b]`:

```
∂Y[t][v] / ∂W[a][b] = X[t][a]   if v == b
                      0         otherwise
```

Now apply chain rule: `∂L/∂W[a][b] = Σ_{t,v} ∂L/∂Y[t][v] * ∂Y[t][v]/∂W[a][b] = Σ_t X[t][a] * ∂L/∂Y[t][b]`.

Reassembled as a matrix: `∂L/∂W = X^T @ ∂L/∂Y`. Exactly the formula above.

The backward pass of the whole model

Run the forward pass, storing all intermediate activations. Then walk backward through the operations, applying the relevant chain-rule formula at each step. The intermediate gradients get reused — this is the efficient part of "backprop": instead of computing 2,240 derivatives from scratch, you compute each intermediate `∂L/∂activation` exactly *once* and reuse it.

There are only about six primitive operations whose local derivatives you need:

1. **Linear layer** `Y = X @ W`: the two formulas above.
2. **Addition** `Y = A + B`: gradient flows equally to both inputs (this is what makes residual connections so clean).
3. **Softmax** `y = softmax(x)`: `∂L/∂x_i = y_i * (∂L/∂y_i - Σ_j y_j * ∂L/∂y_j)`.
4. **ReLU** `y = max(0, x)`: gradient passes through where `x > 0`, zero elsewhere.
5. **Scalar multiply** `y = c*x`: gradient gets multiplied by `c` (for us, `c = 1/sqrt(D_MODEL)`).
6. **Lookup** `y = table[index]`: gradient accumulates in the looked-up slot.

That is the complete rule set. Every neural-network backward pass you will ever read is those six patterns composed.

> **Read along:** `minai.cpp` Part 10, `backward` and `backward_block`. Every step in those functions is one of the six rules above. Seriously, count — you can label each block of lines with one of the six numbers. This is the moment to sit with the code.

Chapter 10 — Stochastic gradient descent: training loop

With forward, loss, and backward in hand, the training loop is just Chapter 2's "four-step recipe" made into code:

```
for step in 1..NUM_STEPS:
    forward(tokens)                     # compute activations and probs
    loss = compute_loss(target)         # measure wrongness
    zero_param_grads()                  # wipe old gradients
    backward(tokens, target)            # compute ∂L/∂parameter for every parameter
    sgd_step()                          # parameter -= learning_rate * gradient
```

That is the entire loop.

Why it is "stochastic"

We call it SGD (Stochastic Gradient Descent) because in a real training setup, each step sees a *random* mini-batch of examples from a large dataset, rather than the full dataset. The randomness introduces noise into the gradient estimate, which — counterintuitively — helps the optimizer escape shallow local minima and find better parameter regions. In MinAI we only have one training example (the sequence `[0..7] → [7..0]`), so there is no randomness per step; our "SGD" is really just GD. But the code is identical.

Learning rate

The `α` in `θ := θ − α ∗ ∂L/∂θ` is the **learning rate**, and it is the single most important hyperparameter in all of machine learning. Too large: you overshoot the minimum and training oscillates or diverges. Too small: training takes forever. Different parameter groups often benefit from different rates, which is why `minai.cpp` defines `LR_EMB`, `LR_ATTN`, `LR_FFN`, `LR_OUT` separately.

Modern optimizers (Adam, AdamW, Lion) go further and automatically tune a per-parameter effective learning rate using running statistics of the gradients. They are strictly better for large-scale training. They also triple

the memory usage, which was unaffordable on a PDP-11 — which is one reason MinAI sticks with hand-tuned fixed rates.

How do you know when it worked?

You watch the loss fall. In MinAI, run `./minai` and see the loss drop from ~2.28 to ~0.009 over 800 steps. Once loss is near zero and accuracy is 8/8, the training worked.

> **Read along:** `minai.cpp` Parts 11 (sgd_step) and 13 (train). Part 11 is three short loops; Part 13 is *the* training loop. If you have read this far, every line is understandable.

Chapter 11 — How calculus actually runs on silicon

We have spent a whole chapter on the calculus of backpropagation — gradients, chain rule, six primitive-operation rules. But a CPU has no *derivative* instruction. A GPU has no *gradient* instruction. They can add numbers, multiply numbers, load from memory, store to memory. That is all. So how does the abstract calculus from Chapter 9 actually become electricity moving through transistors?

Answer: we reduce calculus to arithmetic. Specifically, we express every derivative we need as a **matrix multiplication**, and then we run matrix multiplications on hardware built for exactly that.

Why it is always a matrix multiplication

Look back at the six-rule list from Chapter 9:

1. Linear layer, forward and backward: two matrix multiplies each.
2. Addition / residual connection: pointwise add.
3. Softmax: row-wise sum and divide.
4. ReLU: pointwise “zero if negative”.
5. Scalar multiply: pointwise multiply by a constant.
6. Lookup: indexed read from a table.

Five of the six are trivial pointwise or row-wise operations. One — the linear layer — is a matrix multiplication. In a trained transformer, matrix multiplications account for the overwhelming majority of compute — typically 90% or more of both forward and backward pass time. Everything else is rounding error.

So the question “how do we run calculus on hardware?” reduces to “how do we run matrix multiplications fast?” And matrix multiplication is a question with a very concrete answer.

What a matrix multiply actually costs

Multiply two matrices `Y = X @ W`. Let X have shape `[M, K]` and W have shape `[K, N]`. The output Y has shape `[M, N]`, and each of its elements is a dot product of length K — that is, K multiply-adds. There are `M × N` output elements. Total arithmetic work:

```
M * N * K  multiply-adds
```

A multiply-add is the simplest useful operation a computer can do. Grab two numbers, multiply them, add to a running total. On modern hardware this is a single instruction called an **FMA** (fused multiply-add). Every computer — CPU or GPU — is essentially an FMA machine with extra life support.

To make this concrete: one attention layer in GPT-3 computes a Q @ K^T that is roughly `2048 × 12288 × 12288 ≈ 300 billion FMAs`. Per forward pass, per layer. The backward pass doubles it. Across all 96 layers and trillions of training tokens, the total compute to train GPT-4 has been estimated at roughly `10^25` FMAs. Ten septillion. The hardware exists to make those FMAs finish in a reasonable number of months.

Tensors: just "arrays with more than two dimensions"

You will hear the word *tensor* everywhere in modern ML (PyTorch's core type, "TensorFlow", Google's "Tensor Processing Units"). In this context, a tensor is nothing more than a multi-dimensional array of numbers. A vector is a 1-tensor. A matrix is a 2-tensor. The 3-tensor `X_block[SEQ_LEN][D_MODEL]` (stacked across `MAX_BLOCKS+1`, making it 3D total) is exactly what MinAI stores, and exactly what PyTorch would call a tensor.

Operations on tensors are generalizations of operations on matrices. A "batched matrix multiply" is a pile of matrix multiplies in parallel — once you can do one quickly, you can do a batch quickly. The word *tensor* in software doesn't carry the physics meaning; it's just "a multi-dimensional array with nice operations."

Reducing calculus to arithmetic, in practice

Here is the practical flow every framework (and MinAI) follows:

1. **The user writes a forward pass** — a specific composition of the six-or-so primitive operations (linear layer, softmax, ReLU, add, etc.).
2. **Each primitive has a hand-derived backward formula**. Somebody did the calculus once. For the linear layer, it is the `X^T @ dY` and `dY @ W^T` pair. For ReLU, it is "pass through where input was positive, zero otherwise". These formulas live in a library and are reused forever.
3. **The backward pass is the forward pass run in reverse**, applying the pre-derived formulas at each step, with matrix multiplies where the forward had matrix multiplies.

The result: *no calculus is ever done by the computer at runtime.* The calculus was done by a human, once, symbolically, to produce formulas. What runs on the silicon is just repeated matrix multiplications. This is the critical insight the user rarely sees stated plainly.

Modern frameworks add an automation layer: **automatic differentiation** (autograd). The user writes only the forward pass; the library records every primitive op called, then walks the list in reverse applying the library's stored backward formulas. The computer is still not "doing calculus" — it is still just dispatching matrix multiplies. The autograd layer just figures out *which* matrix multiplies to dispatch, so the human doesn't have to.

MinAI does not have autograd. The backward pass in `minai.cpp` Part 10 is hand-coded, step by step, each block labeled with which rule applies. Read it once and you have seen precisely what a framework like PyTorch is doing behind a cleaner API.

What CPUs and GPUs can do per second

A modern CPU core, using SIMD instructions (NEON on Apple Silicon, AVX on x86), can do roughly 16–32 FMAs per cycle. With a 3 GHz clock and 8–16 cores, a good CPU tops out at a few hundred billion FMAs per second. Sounds huge. Now divide 10^25 training FMAs for GPT-4 by a few hundred billion per second. That is a few thousand years on a single CPU.

You need something much better. Enter the GPU.

> **Read along:** `minai.cpp` Part 8 — every forward-pass matrix multiplication has the identical triple-nested loop structure:
>
> ```
> for (int t = 0; t < ROWS; ++t)
>  for (int j = 0; j < COLS; ++j) {
>  float s = 0;
>  for (int i = 0; i < INNER; ++i) s += A[t][i] * B[i][j];
>  Y[t][j] = s;
>  }
> ```
>
> That innermost line — `s += A[t][i] * B[i][j]` — is one FMA. Every matrix multiply, anywhere in any AI system in the world, is a way to dispatch billions or trillions of those FMAs to as many cores as possible at once.

Chapter 12 — GPUs and memory: why hardware is the limit right now

This is the chapter that explains why NVIDIA is suddenly one of the most valuable companies in the world.

CPU vs GPU, in one paragraph

A CPU has a handful of very clever cores. Each one can run arbitrary branchy code — web servers, compilers, string parsing, game logic. Deep caches, out-of-order execution, branch prediction, speculative loads. A CPU is a generalist.

A GPU has thousands of much simpler cores. Each core is bad at branching and arbitrary code. They are built to do *the same arithmetic operation on a huge pile of numbers at once*. That shape — "same op, lots of data" — is exactly matrix multiplication. An NVIDIA H100 has about 14,000 general shader cores *plus* a few hundred specialized **Tensor Cores**, each of which performs a small matrix multiplication (e.g., 4×4 × 4×4) in a single instruction. Peak throughput at low precision: around 2,000 trillion (two quadrillion) FMAs per second — roughly 6,000× a strong CPU. That ratio is the entire reason AI training happens on GPUs.

GPUs were not invented for AI. They were invented for 3D graphics, which happens to also be "apply the same matrix op to a pile of vertices/pixels". It is an accident of history that graphics cards from the 1990s became the computation backbone of the 2020s AI boom. (Except it isn't entirely an accident: dense matrix math is fundamental to a lot of physics and engineering.)

The memory wall — why compute is no longer the bottleneck

Here is the part almost nobody outside the field knows. The bottleneck of modern AI is not "how fast can we multiply". It is "how fast can we **move** numbers from memory to the arithmetic units". This is called the **memory wall**, and it has dominated hardware design for over a decade.

The arithmetic is fast. You load W and X into registers, execute an FMA, done in a nanosecond. But *loading* W and X from DRAM into registers takes *hundreds* of nanoseconds. If you have to load fresh data for every FMA, the multiplier sits idle 99% of the time. The whole game of modern GPU design is to keep the multipliers fed.

Three numbers worth keeping in your head:

1. **How big are the models?** GPT-3 has 175 billion parameters. At 2 bytes each (16-bit floats), that is 350 gigabytes *just for the weights*. A single NVIDIA H100 GPU has 80 GB of on-chip memory (a special kind called **HBM**, for High Bandwidth Memory). You cannot fit GPT-3 on one H100. You need many GPUs whose memory is pooled via high-speed interconnects (NVLink, InfiniBand). A GPT-4-scale training run might use thousands of GPUs simultaneously, coordinated.

2. **Activation memory during training.** Backprop needs every intermediate activation from the forward pass. For large models this rivals parameter memory. Techniques like **gradient checkpointing** (throw away activations on the forward pass and recompute them during backward) explicitly trade compute for memory — because memory is scarcer.

3. **Bandwidth matters more than capacity.** Your laptop's DDR5 system RAM moves about 50 GB/s. An H100's HBM moves about 3,000 GB/s — 60× faster. That is why model weights live on the GPU, not in system RAM: the CPU-to-GPU link (PCIe) is too slow to feed the multipliers. *Keeping weights close to the compute is the whole point.*

Inference has a memory problem too: the KV cache

Even after a model is trained, running it costs memory. When you ask ChatGPT a question, the model generates one token at a time. For each new token, attention needs access to the K and V vectors of every previous token (see Chapter 5). If it re-derived them from scratch every time, it would re-process the entire conversation every single token. Unacceptably slow.

The fix: cache K and V for every seen token in GPU memory. This is called the **KV cache**. Its size grows linearly with the length of the conversation. For GPT-4 generating a 100,000-token response, the KV cache alone is tens of gigabytes.

This is why long context windows are expensive and why context length is a hardware story, not an algorithmic one. Longer context = more RAM needed on the GPU. Full stop.

Why “right now”

Three things collided around 2020–2023 to create the current moment:

1. **Scaling laws held.** For years, researchers dreaded diminishing returns: double the parameters, get nothing. Instead, doubling kept working, and kept working, and kept working. The “bitter lesson” of AI is that the thing you’d most want *not* to be true turned out to be true: brute force wins.

2. **Transformers generalized.** The same architecture used for language turned out to work for images (Vision Transformers), audio, protein structures, robot control. One hardware stack (GPUs with HBM) now serves almost every large AI workload.

3. **Hardware barely kept up.** Training GPT-4-scale models requires enough HBM, enough interconnect bandwidth, and enough power delivery that only a handful of companies in the world can assemble the necessary cluster. HBM is the choke point: HBM manufacturers (SK Hynix, Samsung, Micron) sell everything they can produce for the next several years. The latest NVIDIA generation, Blackwell, did not dramatically speed up the multipliers compared to the previous generation; what it *did* was double or triple the amount of memory per GPU.

The headline version: **compute grew faster than memory bandwidth for twenty years. Now AI has reached the scale where memory bandwidth is the limit.** Every engineering effort in AI hardware right now is about moving bytes into the multipliers faster. That is why companies spend billions on GPU clusters, why the stock market cares so deeply about a company whose main product is a *memory-saturating math engine*, and why headlines about “datacenter power consumption” and “more GPUs” and “more HBM” feel omnipresent. That is the curtain.

> **Read along:** MinAI has none of this. Our model is 2,240 floats, 9 KB of memory, trains in under a second on any CPU. Every concept is present, but the hardware reality is absent — because a model this small doesn’t need a GPU, doesn’t need HBM, doesn’t have a KV cache worth caching. Chapter 11’s forward-pass triple-nested loop *would run exactly the same pattern* on a GPU, just ten million times faster because the GPU dispatches ten thousand FMAs in parallel instead of sixteen.

Chapter 13 — From MinAI to GPT-4

You now understand every mechanism in every transformer. The scale-up to a real LLM is purely *quantitative*. Here is the full list of differences:

MinAI (with defaults)	GPT-4 (estimated)
1 transformer block	~96 blocks stacked
1 attention head per block	~96 attention heads per block, each a mini version of our single head, concatenated
`D_MODEL = 16`	`D_MODEL ≈ 12,288` or more
`D_FF = 32` (2× `D_MODEL`)	`D_FF = 4 × D_MODEL`
`VOCAB = 10` (digits)	`VOCAB ≈ 100,000` (subword pieces)
`SEQ_LEN = 8`	`SEQ_LEN` in the hundreds of thousands
optional LayerNorm (`--layernorm=1`)	always-on LayerNorm before each sub-block for training stability
optional causal mask (`--causal=1`)	always-on causal mask (for generative models)
one fixed training example	trillions of tokens of text
~2,240 parameters	~1,000,000,000,000+ parameters

Multi-head attention is worth a paragraph. Instead of one set of `(Wq, Wk, Wv)` of full size `[D_MODEL × D_MODEL]`, you have `H` sets of `[D_MODEL × (D_MODEL/H)]`. Each head performs the Chapter-5 attention independently on its share of the dimensions. Then the `H` outputs are concatenated back into a `D_MODEL` vector. Why? Different heads learn to attend for different reasons — one head might focus on syntactic agreement, another on topical relevance. You get `H` distinct attention patterns instead of one.

LayerNorm is a normalization applied to each token's vector before each sub-block: `LayerNorm(x) = (x - mean(x)) / std(x) * gain + bias`. It keeps activations from growing or shrinking as depth grows, which makes 100-layer stacks trainable. MinAI has this now (as of the `--layernorm=1` flag) — see the dedicated section at the end of Chapter 7 for the math and why it works, or just run `./minai --blocks=32 --layernorm=0` to watch gradients explode into `NaN` and then `./minai --blocks=32 --layernorm=1` to watch the same depth train happily.

None of this changes the ideas in Chapters 1–10. Multi-head attention is attention, four times over, in parallel. The depth is the same loop. The training is the same SGD. The loss is the same cross-entropy.

If this is all one big program, then the difference between MinAI and GPT-4 is that GPT-4 is a *much bigger* one big program. The bitter lesson of the last decade is that, at least for language, “more of the same” scales shockingly well. Nobody predicted how far it would go.

Chapter 14 — A guided tour: grow your understanding through the flags

You’ve read twelve chapters of theory and one chapter scaling it up. This chapter is hands-on. Run these commands in order, and after each one look at two things: the loss curve printed at the end, and the attention matrix from the demo. Each command changes *one idea* at a time and lets you see the effect.

`minai.cpp` prints a loss sparkline like this after every run:

```
Loss curve (800 steps, 60 columns, higher bars = higher loss):
  2.280 |▅▅▅
```

Stack two transformer layers. Parameters double (4,032 from 2,240). Final loss should go an order of magnitude lower (around 0.002). Look at block 1's attention matrix — you'll start to see peaks on specific input positions (especially rows 6 and 7), suggesting the second layer is beginning to learn actual positional reversal.

The lesson. More capacity = lower loss, but also different *internal strategies*. A 2-layer model often finds a cleaner solution than a 1-layer model squeezed to do everything.

Step 3 — Force generalization with random inputs

```
./minai --random=1 --steps=5000
```

Now the model sees a fresh random 8-digit sequence every training step. It cannot memorize; it must learn the *rule*. The training log gains a held-out accuracy column — performance on 64 sequences the model has never seen. Watch the train-accuracy column jitter wildly (0% / 12.5% / 25% / 0% / ...) — that is the single-example noise in action. Held-out accuracy crawls upward, maybe to 15-25% by step 5000. The model is struggling, and with batch size 1 it may not grok at all within patience. Step 5 will show why bigger batches change this.

The lesson. This is the moment the word "generalization" becomes concrete. Memorizing is easy; learning a rule is hard. Real LLMs are trained almost entirely in this regime — every example seen once, with held-out data measuring whether the model is actually *learning* vs *storing*. The gap between train accuracy (which jitters wildly from step to step) and held-out accuracy (which rises smoothly) is the heartbeat of machine learning.

Step 4 — Causal masking actually bites, and here is exactly why

```
./minai --random=1 --causal=1 --steps=2000
```

Same setup as before, but attention can only look backward. For reversal, position 0 needs to see position 7 — and now can't. Over 2000 steps you'll see held-out accuracy stuck at about 10–15%, barely above the random baseline. You would be forgiven for concluding the model just "can't learn" this task. But if you run it much longer:

```
./minai --random=1 --causal=1 --steps=200000
```

Something fascinating happens. For roughly the first 5,500 steps the loss stays flat around 2.30 and held-out accuracy is stuck at ~10%. Then, suddenly, between step 5,500 and step 9,000, the loss plunges from 2.30 to 1.30 and held-out accuracy leaps from 17% to 55%. This sudden breakthrough after a long apparent stall is a real named phenomenon, **grokking** (Power et al., 2022). The model spends thousands of steps quietly reorganizing its internal representation without visible progress, then crosses a threshold and the loss

collapses almost instantly. From that point on, held-out plateaus at about **55%**, and *will never go higher no matter how long you train*.

Why exactly 55%? A counting argument.

With causal masking on an 8-digit reversal, each output position `t` needs to predict `input[7-t]` while only being allowed to see `input[0..t]`:

output `t`	needs `input[7-t]`	allowed to see	best possible
0	`input[7]`	`input[0]`	10% (random guess)
1	`input[6]`	`input[0..1]`	10%
2	`input[5]`	`input[0..2]`	10%
3	`input[4]`	`input[0..3]`	10%
4	`input[3]`	`input[0..4]` ← includes `input[3]`	100%
5	`input[2]`	`input[0..5]` ← includes `input[2]`	100%
6	`input[1]`	`input[0..6]` ← includes `input[1]`	100%
7	`input[0]`	`input[0..7]` ← includes `input[0]`	100%

The last four positions *can* see the digit they need to retrieve and learn to reproduce it perfectly. The first four positions are being asked to predict an input they are not allowed to see. Averaging across all eight positions:

```
(4 × 10% + 4 × 100%) / 8 = 55%
```

That is the architectural ceiling. Your model converges to 55.5%. That is not a coincidence — it is the ceiling being hit.

Why the ceiling is not just "hard" — it is mathematically impossible to beat

In `--random=1` mode, each input digit is drawn **independently and uniformly** from 0-9. So for output position 0, which may only observe `input[0]` but needs to predict `input[7]`:

```
P(input[7] | input[0]) = P(input[7])
```

The conditional probability equals the unconditional probability. Knowing `input[0]` gives you **zero bits of information** about `input[7]` — they are as unrelated as two separate dice rolls. This is a property of the *data generator*, not the *model*. No function of `input[0]` can predict `input[7]` better than the marginal distribution, because no such function exists. Not a bigger model, not more training, not a cleverer architecture. If the information the model would need is not in its input, no model can conjure it.

The four unreachable positions therefore converge to cross-entropy loss `log(10) ≈ 2.30` (uniform guessing). The four reachable positions converge to essentially zero loss. Averaged over all eight positions:

```
loss ≈ (4 × 2.30 + 4 × 0) / 8 ≈ 1.15
```

Which is exactly the plateau your program settles at after grokking. The ceiling is not a flaw in the model or a failure of training — it is the model being *correct*: doing precisely what the available information allows, no more, no less.

But real language models use causal masks and work fine — why?

Because natural language is loaded with statistical structure. "The cat sat on the ___" strongly predicts "mat" or "couch" or "floor". The conditional distribution `P(next_word | previous_words)` is *vastly* narrower than the unconditional `P(any_word)`. Past tokens carry enormous information about future tokens, so a causal language model can extract meaningful signal from what it is allowed to see.

Random digit reversal has the *opposite* property: past positions carry no information about future positions, because the data was generated independently. Causal masking is *fine* for tasks where the future depends on the past; it is *fatal* for tasks where the future is statistically independent of the past.

The takeaway — a core ML engineering skill

When a model's accuracy plateaus, there are two very different possibilities:

- **(a)** It has not found the solution yet. More compute, more data, better architecture, different hyperparameters might unstick it.
- **(b)** The information the model would need to improve is not in its input. No amount of anything will help.

Distinguishing these two cases is one of the core skills of a machine-learning engineer. Here — because we know exactly how the data is generated — we can *prove* this is case (b). The model learned every bit there was to learn; the remainder is not there to be learned. That proof comes out of the table above plus the independence property of the data. It is the same style of argument Claude Shannon formalized in 1948 for information theory, and it still runs every decision in the field.

Step 5 — Batch training smooths the curve AND unlocks grokking

```
./minai --random=1 --batch=16 --steps=15000
```

Each training step now averages gradients over 16 random sequences instead of one. Compare the loss sparkline to Step 3's: this one is visibly smoother, and critically the model now has enough clean signal to actually learn the rule. Expect held-out accuracy to stay ~10% for several thousand steps, then **grok** — a sudden breakthrough where held-out accuracy climbs from ~20% to ~100% over a window of 2–3 thousand steps. The exact step count of the breakthrough varies with the RNG seed, but it almost always happens well before step 15000.

The lesson. Every real LLM trains with large batches — GPT-3 used batches of millions of tokens per step. The effect is exactly what you see here: the gradient direction is less noisy, so the optimizer can take more confident steps. This is also why GPUs are useful: a batched matrix multiply of 16 sequences fills a GPU's parallel arithmetic units much better than one sequence does (see Chapter 12).

Step 6 — Sequence length = quadratic attention cost

```
./minai --seq_len=16 --steps=2000
```

Double the sequence length from 8 to 16. Watch the per-step log; it will run visibly slower. Attention's score matrix is now 16×16 = 256 entries per block instead of 8×8 = 64. That `O(N²)` scaling is the reason long-context LLMs are expensive.

The lesson. Every scaling number you read in the press — "GPT-4 supports 128k tokens" — translates to "the inner attention operation costs 128,000² = 16 billion entries per layer". That is the memory and compute price of long context, directly visible in our program slowing down.

Step 7 — A harder task needs more capacity

```
./minai --task=sort --blocks=2 --random=1 --batch=16 --layernorm=1 --steps=10000
```

Switch the task to "sort the input ascending". This is harder than reversal: each output position needs to know the *entire* input to decide its rank. Run it with two blocks, batch 16, and LayerNorm on for stability, and give it 10k steps so grokking has time to land. Watch held-out accuracy climb substantially above 10% even though sort is much harder than reverse.

The lesson. Architecture has to match task difficulty. Try `--task=sort --blocks=1 --ffn=0` — you'll likely see it fail to learn at all, because "half a layer" doesn't have the computational depth to do sorting. Bigger models aren't always better on easy tasks (too much capacity overfits), but they become *necessary* on hard ones. This is the scaling law in miniature.

Step 8 — The local task

```
./minai --task=mod_sum --random=1 --batch=16 --layernorm=1 --steps=5000
```

Target[t] = (input[t] + input[t+1]) mod 10. Each output depends on only two adjacent inputs. Attention barely has to do anything — the FFN does most of the work. Accuracy should climb toward 100% within the 3000-step budget. Try it again with `--ffn=0` and watch it struggle — FFN is doing the arithmetic here.

The lesson. Different tasks stress different parts of the transformer. Attention = cross-position communication; FFN = per-position computation. A truly general model has both because different sub-

problems need each.

Step 9 — Deep stacks explode without LayerNorm

```
./minai --blocks=32 --layernorm=0 --steps=200
```

Thirty-two layers, no normalization. Watch what happens — within about 150 steps the loss column prints `nan` and accuracy locks at 1/8. The 32-layer model's gradients have exploded to infinity and then past it. The model is dead. This is not a bug in MinAI; it is exactly what happens to any deep neural network without normalization. It is the wall that stopped transformers from going deep before LayerNorm (and its cousin BatchNorm) solved the problem.

The lesson. Architecture has numerical constraints that compound with depth. "Just stack more layers" doesn't work by itself; you need machinery to keep the scales of activations and gradients under control across all those layers.

Step 10 — LayerNorm makes the same depth trainable

```
./minai --blocks=32 --layernorm=1 --steps=1000
```

Exact same architecture, same depth, same training signal. Only change: LayerNorm added. Loss drops smoothly, accuracy hits 8/8 long before step 1000 and then just keeps driving the loss down. Same problem, entirely different outcome, because of one normalization step per sub-layer.

The lesson. LayerNorm is the single most important piece of machinery separating 2017-era shallow transformers from the 100-layer models that followed. Once you have it, depth stops being limited by numerics and starts being limited by compute.

Step 11 — GPT-3 depth, in a single-file C++ toy

```
./minai --blocks=96 --layernorm=1 --steps=2000
```

Ninety-six layers. The exact layer count of GPT-3. In ~820 lines of C++ on a laptop. It will run for a few minutes because 96 layers × 2000 steps × many small matrix multiplies, and it converges slowly because we still only have `D_MODEL=16` width to spread 96 layers across. But it *trains*. No crashes, no NaN, loss falling steadily, accuracy heading to 8/8. That is the skeleton of GPT-3, running in full, on your own machine.

The lesson. This is the moment MinAI stops being a toy demo and starts being a true miniature of a real LLM. Every idea you read about in frontier-model papers — depth, residuals, pre-LN, batch training, attention, softmax, cross-entropy — is present here. The only difference is magnitude. Same dials, same physics.

Step 12 — Hierarchical weight quantization (the “organist” trick)

```
./minai --random=1 --seq_len=32 --batch=16 --blocks=4 --layernorm=1 --steps=10000 --extra_demos=1
```

After training finishes, the program runs three bonus demos. The first quantizes the trained `Wout` matrix three ways — Q8 flat, Q4 flat, Q4 with hierarchical scaling — and reports the precision cost of each. The hierarchical Q4 uses a “feet” (global) scale, a “left hand” (per-column) scale, and a “right hand” (per-weight 4-bit) index. This is the compression scheme behind llama.cpp’s Q4_K_M format that lets Llama-2 70B run on a 64 GB laptop.

The lesson. You don’t have to store every weight at 32 bits. With a hierarchy of scales you can get down to ~5 effective bits per weight while keeping most of the precision. Bandwidth is the bottleneck (Chapter 12); fitting 4-6× more weights per byte loaded is 4-6× more tokens produced. This is how 2025-era open-weight LLMs reach consumer hardware.

Step 13 — Speculative decoding (fast cortex + slow cortex)

Same command as Step 12 (which uses `--seq_len=32` for enough windows to populate a good histogram). The second demo trains a tiny 1-block, no-FFN, no-LayerNorm “draft” — the classic 1,216-param MinAI — then prints a side-by-side rollout on one held-out sequence, an accepted-tokens histogram across all held-out windows (groups of K=4 consecutive positions), and an honest speedup formula that accounts for the draft’s own compute cost.

How the verify-K-in-one-pass trick works. This is the core cleverness of speculative decoding, worth spelling out because it is not obvious. In normal (autoregressive) inference, producing one new token requires one forward pass of the big model. Spec decoding replaces that with: (1) the draft predicts K candidate tokens one at a time (K cheap forward passes), then (2) the big model runs ONE forward pass that receives the draft-proposed prefix + K tokens and outputs its own argmax at *every* position in one go (transformers are embarrassingly parallel across sequence positions). The big model’s output at position `t` tells you what the big model *would have produced* at `t`. Compare against draft[0..K-1]: accept tokens from left to right up until the first disagreement, then take the big model’s argmax at the disagreeing position, discard the rest of the draft’s guesses. Net output: one big-model forward delivers between 1 and K+1 tokens depending on agreement. Cost: `K * C_draft + 1 * C_big`. That is the source of the speedup — amortizing the expensive big-forward over multiple accepted draft tokens.

The lesson. Our tiny 1-block draft struggles on a 32-digit random reversal; agreement lands around 15-20%, the histogram shows most windows accepting zero tokens, and the “honest speedup” comes out to about 0.8× — meaning spec decoding would *slow this configuration down*. That is itself the lesson: a draft must be both much cheaper AND frequently right to help. The reference table in the demo output shows what speedup you’d get at various agreement rates and cost ratios — at 80% agreement with a 10× cheaper draft, the speedup jumps to 2.4×, which is roughly what production systems achieve pairing a 1-7B draft with a 70B verifier.

Step 14 — KV cache tiering (Miller's 7 in silicon)

Same command as Step 12. The third demo sweeps a "hot working memory" size `W` from 0 to `seq_len`, re-running the forward pass with `K` / `V` quantized to Q8 for every position older than `W`. It prints accuracy at each `W`, with Miller's 7 marked.

The lesson. Human working memory holds about 7±2 items; older memories get compressed into longer-term storage that is slower but much larger. Production LLMs do *exactly* this with their KV cache: keep the most recent handful of tokens at full precision, progressively quantize older ones, evict the oldest. This is how 1M-token-context models become tractable — no GPU has a terabyte of HBM, so the cache has to be tiered. On MinAI specifically the Q8 round-trip is too fine-grained to move the accuracy needle, as the note at the end of the demo explains. It is also the only extension in this program that came *directly* from a human-brain observation: you already know the engineering answer because your skull is solving the same problem.

Putting it together

By now you should have:

- Seen gradient descent work in real time (Step 1).
- Felt the difference between memorization and generalization (Step 3).
- Seen an architectural choice break a model (Step 4).
- Watched batching smooth training noise (Step 5).
- Felt `O(N²)` attention slow down (Step 6).
- Matched architecture capacity to task difficulty (Step 7).
- Seen that FFN matters on some tasks and not others (Step 8).
- Watched a deep network explode into NaN without normalization (Step 9).
- Watched the same depth train cleanly with it (Step 10).
- Actually trained a 96-layer transformer on your own machine (Step 11).
- Compressed a weight matrix with a three-level scale hierarchy (Step 12).
- Measured the draft-model agreement rate that drives speculative decoding (Step 13).
- Quantized old tokens and seen the biological-working-memory / LLM-KV-cache alignment (Step 14).

That is not a metaphor for what it's like to train an LLM. That *is* what training an LLM is. The only thing GPT-4's researchers are doing differently is doing it with a million times more of everything — more parameters, more data, more compute — while fighting the same fundamental tradeoffs between capacity and overfitting, speed and batch size, context length and memory. Same dials, same curves, same lessons.

The curtain is down.

Chapter 15 — From digits to real text: tokenization and pretrained vectors

MinAI has a trivial vocabulary: ten tokens, one per digit. The mapping from symbol to integer ID is obvious — the digit `3` is token `3`. That is the last free lunch we got. Real language offers no such convenience. This chapter covers the one big piece MinAI does not model, which is how a real language model takes the arbitrary text you type and turns it into a sequence of integer IDs that a transformer can accept. It is worth understanding because, unlike attention or backprop, tokenization is a *separate* engineering system from the model itself — a preprocessor sitting in front of the neural network — and it has real consequences for what the model can and cannot do.

The problem

A transformer needs a finite alphabet of possible input symbols — the vocabulary, size `V` — because its token-embedding table has exactly `V` rows (one per symbol). MinAI has `V = 10` and the mapping is trivial. English text does not come with an obvious finite alphabet. What do you do?

Three approaches exist, and each has a defining weakness.

Approach 1: one token per character. `V` is small — about 128 if you stay ASCII, 50,000+ if you want full Unicode, a few million for every emoji ever minted. Every word becomes a sequence of character tokens: "cat" is `[99, 97, 116]`. It works but is wildly wasteful. A 10-word sentence is now a 60-character sequence, and the transformer's `O(N²)` attention is computing 3,600 score entries per layer for what ought to be a 10-token thought. Attention cost blows up faster than the vocabulary reduction saves.

Approach 2: one token per word. The vocabulary is the dictionary — call it 50,000 words. Compact, fast, short sequences. But `run`, `runs`, `running`, `runner` are four unrelated token IDs to the model; their shared meaning has to be re-learned per token. Proper nouns, typos, and new words (`MinAI`) have no token at all, so you either add a catch-all `[UNK]` token — collapsing every unknown word into one bucket — or you give up on those words entirely. This is how pre-2017 NLP systems worked, and the ceiling was low.

Approach 3: subword tokens. Tokens smaller than words but larger than characters. "tokenization" might be split into `["token", "ization"]`. "running" into `["run", "ning"]`. The model sees the shared prefix `run` and can generalize across related words. Rare or invented words degrade gracefully — every string decomposes into *some* sequence of known subwords, even if that sequence has to fall all the way down to single characters. This is the sweet spot, and it is what every LLM worth its memory uses.

Byte-Pair Encoding (BPE): the dominant subword algorithm

BPE was invented in 1994 for file compression and repurposed for NLP in 2016 (Sennrich et al.). The training procedure is simple enough to describe in full:

```
 Start with every character as its own token.
Repeat K times:
  Count the most frequent adjacent pair of tokens in the corpus.
  Add that pair as a new single token to the vocabulary.
  Replace every occurrence of the pair in the corpus with the new token.
```

Train this with, say, `K = 50,000` merges on a large English corpus and you get:

- Very common words (`the`, `and`, `of`) become single tokens within the first few iterations.
- Common suffixes (`ing`, `ed`, `ation`) become single tokens soon after.
- Rare or technical words stay split into multiple tokens that usually decompose along morpheme boundaries.
- Any unseen input is always expressible, because in the worst case BPE falls all the way back to characters.

Concrete examples from GPT-style tokenizers: - `"running"` → `["run", "ning"]` - `"MinAI"` → `["Min", "AI"]` (the tokenizer never saw it during training but both pieces exist) - `"antidisestablishmentarianism"` → maybe `["anti", "dis", "establish", "ment", "arian", "ism"]`

GPT-2/3/4 use BPE with about 50k–100k tokens. Llama uses a similar algorithm called SentencePiece, with about 32k tokens. Claude uses a proprietary variant. The parameters differ; the mechanism is the same. The chosen vocabulary size is a trade-off: bigger vocabulary means shorter sequences per sentence (faster inference) but a bigger embedding matrix (more memory, more parameters to train).

Once you have tokens, everything else is MinAI

Here is the point worth pausing on: after the tokenizer runs, the transformer does not know or care where the integers came from. It just sees a sequence of IDs:

```
"The cat sat"  →  tokenizer  →  [464, 3797, 3332]
```

And those IDs go into a `token_emb` lookup table, exactly like MinAI's. In MinAI that table has 10 rows. In GPT-3 it has 50,257 rows × 12,288 columns = roughly 618 million parameters just for token embeddings. Same exact line of code. The table is bigger — and that is the entire difference.

Position embeddings still add. Attention still runs. FFN still runs. Softmax at the output still runs over the vocabulary (now 50,000 possible next tokens instead of 10 possible next digits). Cross-entropy loss still compares probs to a one-hot target. Every mechanism you've learned applies, unchanged, at full scale.

Pretrained embeddings: don't train them from scratch if you don't have to

Training good token embeddings requires seeing each token in many contexts, so the model can figure out what it "means". That is expensive. Worse, a fresh model starts with random embeddings, so its early training budget gets burned just bringing the embeddings to something sensible before the higher layers can do anything useful.

The fix is **transfer learning**: take somebody else's already-trained embeddings and use them as your starting point. Two flavors exist.

Static embeddings. Word2vec (2013), GloVe (2014), FastText (2016) — relatively shallow algorithms that produce a fixed ~300-dimensional vector per word. The same vector every time, regardless of context. Fast

to look up, decent for word-similarity tasks, but blind to context: `bank` next to `river` and `bank` next to `deposit` get the identical vector.

Contextual embeddings. BERT (2018), RoBERTa (2019), DeBERTa (2020) — transformer *encoders* that read a whole input and produce per-token vectors whose content depends on the surrounding tokens. Now `bank`-river and `bank`-deposit get genuinely different vectors. These are vastly more informative, and "pretrained vectors" in 2025 almost always means this kind.

DeBERTa as a concrete example

DeBERTa ("Decoding-enhanced BERT with disentangled attention", He et al. 2020) is a transformer encoder: you feed it a BPE-tokenized string, it returns a 768-dimensional vector (base variant) or 1024-dimensional vector (large variant) for each input token. Internally, DeBERTa is built from *exactly* the mechanisms in this book: multi-head attention, residuals, LayerNorm, feed-forward blocks, cross-entropy loss. It was pretrained on billions of tokens of English with a self-supervised "guess the masked word" objective. The outcome is embeddings that carry remarkable semantic structure — words with similar meanings end up close in the vector space, synonyms cluster, and the surrounding context gets baked into every output.

Typical downstream uses:

- **Semantic search.** Encode a query and a billion documents with DeBERTa. Find documents whose embedding is closest to the query's. This is how "meaning-based" retrieval works, as distinct from keyword matching.
- **Classification.** Feed DeBERTa's output through a small classifier head. Train only the head. You get state-of-the-art sentiment analysis or topic detection for almost free.
- **Retrieval-augmented generation (RAG).** Instead of relying only on a language model's internal (parametric) memory, retrieve relevant documents with a DeBERTa-style encoder, paste them into the prompt, then let a generative model answer. Every "chat with your PDF" product does this.

The vector database connection

Once you have a DeBERTa-style encoder, every piece of text in your corpus becomes a 768-dim vector. A billion pieces of text is a billion-row matrix. Searching that matrix for the nearest neighbor of a query vector is a whole subfield — **vector databases** (FAISS, Milvus, Pinecone, Weaviate). They use *exactly* the hierarchical quantization tricks from Part 20 of `minai.cpp`: product quantization, residual vector quantization, inverted-file indexes. The organist metaphor you already know from this project — feet get you to a region, left hand to the chord, right hand to the exact note — comes directly from here. Matching a query vector against stored embeddings is the application that most directly maps onto that metaphor.

What tokenization does *not* add conceptually

It is easy to feel that tokenization is a huge new idea layered on top of the transformer. It is not. It is a *preprocessing step*. It turns arbitrary text into integer IDs. That is its only job. Every lesson in this book still applies to what the model does *after* that.

What real LLMs add beyond MinAI because of tokenization:

- **A bigger `token_emb` table.** 100,000 × 12,288 floats instead of 10 × 16. Together with the output projection `Wout`, the embedding and un-embedding weights account for a very large fraction of total LLM parameters (roughly a quarter of GPT-3, less in bigger models where the middle dominates).
- **A bigger `Wout` table** with the same shape flipped, since it predicts which token comes next.
- **Sometimes tied weights.** Because `token_emb` and `Wout` have reciprocally matching shapes (`V × D` and `D × V`), many real LLMs use the same matrix for both, saving hundreds of millions of parameters. This is a parameter-sharing trick, not a new idea.
- **A tokenizer dependency.** A separate file (typically `tokenizer.json`) defining the BPE merges and a runtime that applies them before any of your code sees the text. Swap tokenizers and your model breaks — the integer IDs no longer mean what the embedding table was trained to expect.

Everything else is identical to MinAI. Same attention, same residuals, same backprop, same cross-entropy loss. The entire book you just read describes every operation a GPT-4 inference does, once the token IDs are in hand.

Summary

Tokenization is the reason a 175-billion-parameter language model can accept any question at all in any language. It turns the open-ended problem "accept English text as input" into the bounded problem "accept a sequence of integers from a fixed 50–100k alphabet." Once you have the integers, the transformer described in Chapters 1–13 of this book is what processes them — the very same mechanism, enlarged.

Pretrained contextual encoders like DeBERTa are *themselves* transformers applied to tokenized text. Their output vectors are then reusable as features for downstream tasks, including being stored in vector databases for semantic search. In that sense, tokenization + pretrained encoders form a software stack on top of which most practical NLP is built. MinAI skips that stack — its vocabulary is ten digits and its embedding table is learned from scratch — but every real-world LLM sits on top of it.

If you understood MinAI, you understand the model inside DeBERTa, inside GPT-4, inside the open-weight Llama that runs on your laptop. The tokenizer is the one piece of machinery you have to pick up separately.

Section 2 — From MinAI to MaxAI

Section 1 is the introduction. You now understand every mechanism in every transformer and have a running, complete 540-line program to show for it. If you wanted to stop here you could, with no guilt and no gap.

Section 2 is the textbook version. The companion program is `maxai.cpp`, which starts as a byte-for-byte copy of `minai.cpp` and gains exactly one new idea per chapter. By the end of Chapter 22 the program will

accept a prompt string, tokenize it through a learned byte-pair subword vocabulary, run a cached autoregressive forward pass, sample tokens with temperature / top-k / top-p, stop on EOS, and generate text — a simplified GPT, end to end. All of it in one C++ file.

The expected reading pattern shifts slightly in Section 2. In Section 1 we read the prose and the code in lockstep; in Section 2 we read the prose carefully and glance at `maxai.cpp` when we want to see a mechanism in running form. The code still compiles cleanly at every step, and `diff minai.cpp maxai.cpp` at the end of any chapter shows exactly what that chapter added. But the density of the prose is higher, the chapters are longer, and the payoff moments — the first time the model generates something that looks like English — are worth sitting with.

Nothing in Section 2 is conceptually new. Every idea is either a direct application of something Section 1 established or a very small extension thereof. What Section 2 offers is depth: worked numerical results, ceiling calculations tied back to information theory, side-by-side timing measurements, and the parts of a real LLM stack (sampling, vocabulary, corpus training, tokenization, caching) that Section 1 only gestured at. Read it slowly and you will have a working mental model of the entire stack — from the character a user types to the character that comes back.

Chapter 16 — From reverse to next-token

What this chapter does

Everything in the first fifteen chapters was in service of a single arbitrary task: reverse an eight-digit list. Useful for teaching, but no human has ever actually wanted that done. This chapter pivots `maxai.cpp` to the task every generative LLM is actually trained on. The code delta is tiny — one new case in `make_target()`, five plumbing edits, and a startup validation — but the conceptual distance is considerable: after this chapter the program meets the technical definition of a language model.

We will stay in the “predict digits” sandbox for three more chapters while we add the generation loop (17), sampling (18), and a widened alphabet (19). Only in Chapter 20 do we move to real text. The hop from digits to English is smaller than it looks: the arithmetic is identical; the size of the symbol table and the amount of training data change, and little else does.

The task, in one equation

Every generative LLM — ChatGPT, Claude, the autocomplete on your phone — is trained with one sentence-long objective:

> Given a prefix of tokens, assign a probability distribution over the next token. Push probability mass onto

> the token that actually comes next.

In code, given everything MinAI already provides, this is one line:

```
target[t] = tokens[t + 1];        // next-token (language modeling)
```

Compare against reversal's target:

```
target[t] = tokens[T - 1 - t];    // reverse
```

The forward pass, the cross-entropy loss, the six-rule backward pass of Chapter 9, the SGD optimizer of Chapter 10 — none of them care which rule produced `target[]`. Cross-entropy just wants to know the index of the correct class at each position; backprop just needs `probs[t] - onehot(target[t])` as the output-layer gradient (the algebraic miracle from Chapter 6 is untouched); the weight-update loop is byte-identical. The entire conceptual pivot from "sequence-puzzle solver" to "language model" is *one change to one lookup table at training time*.

Readers who noticed that MinAI's existing `--task=shift` does `target[t] = tokens[(t + 1) % T]` are right: shift is *almost* this. The modular wraparound at the last position is the only real difference, and it happens to be wrong for language modeling — the token after the final input is not the first input; it is unknown. So we add a new task rather than overload shift, and we handle that final position honestly (see further down).

The four axes of a language-modeling task

Before we start changing things it helps to name the degrees of freedom explicitly. Every supervised sequence-learning setup has four independent knobs, and naming them sharpens the rest of this chapter — and of the rest of Section 2, because the remaining chapters each adjust one or two of the four without touching the others:

1. **Target rule.** Given the input sequence, what is the right answer at each position? For MinAI this was a hand-designed puzzle (reverse, sort, mod_sum). For next-token prediction it is `target[t] = tokens[t + 1]`. For a captioning model it would be "the next word of the caption given the image and the prefix generated so far." All of these are target rules, and the loss function itself is blind to how the target was generated — it just consumes the index of the correct class.
2. **Attention mask.** Which input positions may each output position look at? MinAI exposed this via `--causal=0|1`. For next-token prediction the answer is forced: causal. We will enforce it at startup.
3. **Loss masking.** Which positions actually contribute to the loss — and therefore to gradients, and therefore to the model's learned behavior at those positions? This is the subtle knob. We will mask the final position out of the loss entirely, because there is no `t + 1` to predict when `t = T - 1` and scoring it would be either dishonest (if we invented a placeholder target) or arbitrary (if we scored it against random noise).

4. **Vocabulary.** What is the finite symbol set the model predicts over? MinAI's was ten digits; GPT-3's is ~50,000 BPE subwords. Chapter 19 grows ours from ten digits to the printable ASCII range; Chapter 21 introduces a tiny byte-pair encoder of our own. Chapters 17 and 18 leave the vocabulary alone.

These four knobs are orthogonal: changing one does not require changing the others. That orthogonality is a big part of why the transformer recipe has generalized so well across domains. Image models adjust (1) and (4); audio models adjust (4); code models use a different tokenizer but the same (1) (2) (3). Section 2 will walk each knob in turn and show the code delta each adjustment requires.

Causal masking stops being optional

In MinAI the `--causal=1` flag was a knob: on, and each output position could only look backward; off, and every output position saw every input. For the reverse task, off was the helpful default — position 0 needs to see position 7 to do its job.

For next-token prediction the choice is no longer yours. If position `t` is asked to predict `tokens[t + 1]`, and the attention block is allowed to look at *all* positions including `t + 1`, then the "task" collapses to a trivial copy-from-the-future. The attention pattern at position `t` learns to place all its weight on column `t + 1`; the value path copies the target's vector forward; the output projection reads it off. Loss drops to near-zero in about thirty steps. The model has learned precisely nothing about the data distribution.

MaxAI enforces the pairing at startup. If you invoke `--task=next_token` without `--causal=1`, `parse_args()` prints a helpful error and exits:

```
error: --task=next_token requires --causal=1
       (a non-causal next-token model can trivially peek at
        the very token it is asked to predict, so training is
        meaningless; see Chapter 16 of ARITHMETICOFINTELLIGENCE.md for why)
```

You can delete the check (it is five lines near the end of `parse_args`) if you want to run the "cheating" experiment as a pedagogical demonstration — watching the loss drop impossibly fast and the model learn exactly nothing is itself instructive — but the default stance is "safe by construction."

Autoregressive ≡ causal. The word *autoregressive* shows up constantly in ML literature; it is used in two slightly different senses but in the transformer context it means exactly the same thing as "causally masked." The output at position `t` is a function only of positions up to and including `t`. The statistical sense (from classical time-series theory — AR(p) models, ARMA, ARIMA) is conceptually identical: each step predicts itself from a window of its own past. The statistics and machine-learning communities converged on the same word for structurally the same setup.

Decoder-only ≡ autoregressive transformer. The other term you will hear is *decoder-only*, and it is just a name with a historical backstory. The 2017 "Attention is All You Need" paper had two halves — an *encoder* (non-causal, reads the whole source sentence; built for machine translation) and a *decoder* (causal, generates the target sentence one token at a time). GPT-1 kept only the decoder half; BERT kept only the encoder half; T5 went back to both; modern frontier LLMs (GPT-4, Claude, Llama, Gemini, DeepSeek) are all

decoder-only. The names stuck from the 2017 layout even though almost nobody still builds full encoder-decoders for autoregressive text. So when you read “decoder-only” in a paper, translate in your head to: *uses a causal attention mask and is trained with next-token prediction*. Nothing more.

Teacher forcing: why training still runs in parallel

Here is the piece that confuses most readers the first time. If the task is “generate one token at a time,” why does our training loop still look like MinAI’s — one forward pass, every position scored at once?

Because during training we already know the entire input sequence. Position 0 tries to predict position 1; position 1 tries to predict position 2; position 6 tries to predict position 7. All seven predictions are computable in *one forward pass* because the causal mask guarantees each only depends on the inputs it was entitled to see. All seven gradients are computable in one backward pass for the same reason. Every scored position contributes its own cross-entropy term, the terms are averaged, and the optimizer takes a single step against that average. One training step, `T_eff` supervised examples, zero wasted compute.

This parallel-training shortcut is called **teacher forcing**, and it is one of the few genuinely clever tricks this book will name explicitly. The name comes from the fact that each position is fed the *true* previous tokens — as if from an ever-correct teacher — rather than whatever the model might have produced on its own. Without teacher forcing, if every training step had to generate one token at a time and feed its own output forward before supervising the next position, training would be roughly `T` times slower. For a 2048-position context that is three orders of magnitude. Teacher forcing is the reason real LLMs can train on trillions of tokens in reasonable wall-clock time.

Exposure bias: the catch. At training time the model only ever sees perfect prefixes. At inference time (Chapter 17 onward) it has to consume its *own*, possibly imperfect, outputs. For short generations this mismatch is harmless. For very long generations it can compound — a small error at step 4 corrupts the input to step 5, which compounds into a bigger error at step 6, and so on until the generation drifts off-distribution. Real LLMs mitigate this in two ways: (a) **scheduled sampling** during training — occasionally feeding the model’s own guess instead of the truth, so it practices recovering from its own errors; and (b) **RLHF and related post-training** — reinforcement-learning fine-tuning that explicitly optimizes for long-horizon output quality rather than per-token log-likelihood. MaxAI will use plain teacher forcing with no scheduled sampling; our sequences are short enough that exposure bias does not bite hard, but you may still see its shadow when you run Chapter 17’s `generate()` loop and watch the model slip into a repetitive rut.

This whole training pattern — learn from (input, true-output) pairs and imitate the teacher — is the oldest idea in machine learning. It shows up under names like “behavior cloning” and “imitation learning” in robotics, and simply as “supervised learning” in statistics. Teacher forcing is specifically the sequential version. It is not new to transformers, it will not go away, and you will see it everywhere from recurrent networks in the 1980s to the language models of 2026. The difference is always scale and whatever post-training is layered on top.

The last position, handled honestly

At position `t = T - 1` (the final one) there is no `t + 1` in the input to predict. There are three ways to handle this. Each has a different honesty profile and a different plumbing cost. The first draft of Chapter 16 took the shortcut; this chapter moves to the honest version before we go further, because the ceiling lessons that follow depend on clean numbers.

Option 1 — predict self. Set `target[T - 1] = tokens[T - 1]` and let the model learn "at the last position, predict what you were given." With causal attention the model can see itself, so this is trivially learnable in a handful of steps. Cost: the headline accuracy number is systematically inflated by `1/T`, because one position out of `T` is a guaranteed free point once training kicks in. For `T = 8` that is a full 12.5 percentage points of lie. The loss ceiling is also shifted because one position contributes near-zero loss instead of `log(VOCAB)`. Clean to code; dishonest to report.

Option 2 — end-of-sequence token. Add one extra symbol `EOS` to the vocabulary and set `target[T - 1] = EOS`. Now the final position is really learning something ("sequences end here"), the accuracy denominator is still `T`, and the loss math is unbiased. This is what every real LLM does. It requires growing `VOCAB` from 10 to 11, which in turn requires widening every embedding and output-projection shape, which in turn requires re-tuning by at least a small amount. We will take this option in Chapter 19 when the vocabulary is already being overhauled (digits → characters) and adding one extra symbol is essentially free. Doing it now would be premature.

Option 3 — mask the final position out of the loss. Do not score position `T - 1`. No loss contribution, no gradient flowing back through its prediction, no accuracy credit or penalty. The prediction is meaningless by construction, so we never look at it. The reported numbers average over the `T_eff = T - 1` positions that *are* scored. This is the industrial-standard trick for handling padding tokens in variable-length batches, used in every modern training pipeline. For our fixed-length setup the "mask" is a single trailing position, but the machinery is general.

We take option 3. The code change is a single helper in Part 3 of `maxai.cpp`:

```
static int effective_seq_len() {
    return cfg.task == Task::NextToken ? (cfg.seq_len - 1) : cfg.seq_len;
}
```

Four call sites switch from `T` to `T_eff`:

- **`compute_loss()`** in Part 9 — loops `t = 0..T_eff-1` and divides the sum by `T_eff`. Average loss over scored positions.
- **`backward()`** in Part 10 — initialises `g_logits[t][v] = (probs[t][v] - onehot(target[t])[v]) / T_eff / batch` for `t = 0..T_eff-1`, and then **explicitly zeroes** `g_logits[T-1][·]` for `t = T_eff..T-1`. Without that explicit zero, the last row of `g_logits` would still hold whatever noise happened to be in memory, and a nonsense gradient would flow back through `Wout`, the final LayerNorm, and the entire block stack from that position. Zeroing is the source-level way to say "this position does not supervise anything."
- **`count_correct()`** in Part 14 — loops `t = 0..T_eff-1`. No free point from the masked position.

- **`eval_heldout_accuracy()`** in Part 15 — same loop bound, and the denominator is `NUM_EVAL * T_eff` rather than `NUM_EVAL * T`. This last piece matters: if you divided by `NUM_EVAL * T` while only counting hits in `T_eff` positions, you would systematically *understate* held-out accuracy by a factor of `T_eff / T`. The denominator has to match the population you counted over.

For completeness: the value we store into `target[T - 1]` is now arbitrary — every read site skips it. We leave it as `tokens[T - 1]` because that is guaranteed to be a valid `VOCAB` index and therefore cannot crash any future code path that forgets to check. A small cosmetic: the demo output in Part 17 still prints the full target array, so you will see `target[T-1]` shown as equal to `tokens[T-1]` at the end of the target row. Ignore it; it is not scored.

This is the real industrial pattern. Anywhere a paper mentions "we mask padding tokens out of the loss," "we only supervise on response tokens, not prompt tokens," or "the loss is averaged over non-masked positions," this is the pattern. For next-token prediction in a production LLM: - Prompt tokens are typically *not* supervised (you don't train the model to "predict its own prompt" — you just want it to condition on it). - Response tokens *are* supervised. - Padding tokens (inserted to make variable-length sequences rectangular) are *never* supervised. - The denominator is always the number of supervised tokens, not the number of tokens in the tensor.

Our single-trailing-position mask is the simplest possible instance of a pattern that, at scale, governs everything from RLHF response-only training to preference-tuning with DPO. Seeing it in this minimal form is worth the retrofit.

Cross-entropy revisited — computing the loss ceiling

Chapter 8 introduced cross-entropy as the "badness function" for classification: `L = -log(p_correct)`, averaged across positions. Now that the positions no longer carry structure (i.i.d. uniform digits), we have the rare luxury of being able to compute the loss floor *exactly*.

At each scored position `0..T_eff-1`, the target is `tokens[t + 1]`, an integer drawn uniformly and independently from `0..9`. The model produces a distribution `probs[t][·]` over the 10 symbols. The per-position loss is:

```
L_t = -log(probs[t][tokens[t+1]])
```

What is the best distribution the model could possibly place there? The distribution that minimises *expected* cross-entropy against the true data distribution is the true data distribution itself. For uniform digits that is `(1/10, 1/10, ..., 1/10)`. The resulting expected loss per position is:

```
E[L_t] = -Σ_v (1/10) * log(1/10) = log(10) ≈ 2.3026
```

That is the floor. No training procedure, no architecture improvement, no amount of compute can push the loss below `log(10)` on uniform 10-symbol data — because no conditional distribution `P(input[t+1] | input[0..t])` has any more information than the marginal `P(input[t+1]) = 1/10`. Independence of the

data generator means the past literally carries zero bits about the future. We derived this argument differently in Chapter 14 Step 4 for the causal-reversal ceiling; in that case the ceiling was 55% accuracy because 4 positions out of 8 could cheat off information that actually was in their reachable past. Here no such reachable past exists for any position (memoryless source), and the ceiling is pure uniform.

The accuracy ceiling follows. Argmax of the best-possible distribution is whichever class happens to be tied at `1/10` — effectively a random tie-break among 10 classes. Expected accuracy = `1/10 = 10%`. With `T_eff` scored positions each at the same ceiling, the sequence-level held-out accuracy ceiling is also `10%`.

You will watch the loss curve settle at `~2.30` and never go below it. Held-out accuracy will plateau at `~10%`. Not because anything is broken — because the model has already learned everything there is to learn, and there is nothing more to find.

This is the exact argument Claude Shannon formalized in 1948 when he invented information theory. The minimum cross-entropy of a source is its **entropy** `H(X) = −Σ p(x) log p(x)`. For a uniform distribution over `N` symbols, `H = log(N)`. A trained language model's cross-entropy on held-out data is an empirical estimate of `H(source)` — and because real English has rich statistical structure, it is much lower than `log(50000)`. GPT-4 attains something on the order of **3 bits per token** on held-out English, down from the ~8 bits you would get by assigning every ASCII byte equal probability. That compression ratio *is* the model's knowledge about English. The lower the cross-entropy, the more the model has internalized about how the source produces its sequences.

Training a language model is, mechanistically, an iterative procedure to lower cross-entropy against a data stream. Everything else — attention, residuals, normalization, scale — is in service of making that procedure work at billions-of-parameters scale. When you hear someone say "the model learned English," they mean: the cross-entropy of the trained model on held-out English is close to the inherent entropy of the English source. When you hear "the model is overfitting," they mean: the cross-entropy on the training data is falling below the source entropy, which can only be achieved by memorizing rather than modeling. Once you see the training objective through the entropy lens you see the same picture in every ML paper in the field.

> *Echo back to Chapter 14 Step 4.* Chapter 14 established the template for this kind of reasoning: when a model's accuracy plateaus, the first question is not "why is the model bad?" — it is "is the information the model would need even in its input?" For causal reversal on random digits we derived a 55% ceiling from a per-position counting argument; for next-token on random digits we derive a 10% ceiling from the independence of the data generator. Both arguments are information-theoretic, and both are absolute: no architecture improvements, no hyperparameter sweeps, no larger batches can beat them. Recognizing which plateaus are real (information-theoretic) and which are training artifacts (local minima, a bad learning rate, a dead ReLU) is one of the most valuable day-to-day skills a practitioner develops, and this book will drill it every chapter.

What you get after Chapter 16

Build and run:

```
clang++ -std=c++17 -O2 -o maxai maxai.cpp
./maxai --task=next_token --causal=1 --random=1 --batch=16 --steps=5000
```

Expected behaviour:

- Held-out accuracy climbs toward the `~10%` ceiling within the first few thousand steps and sits there. You will likely see it pass through numbers like 8%, 9%, 10%, 11% — the tiny jitter around 10% is sampling noise on the 48-point held-out set, not real improvement.
- The printed loss sparkline flattens around `2.30` (= `log(10)`, the entropy of the uniform 10-symbol source).
- The attention matrix of block 0 converges to a uniform lower-triangular pattern: `row out=t` places equal weight on every column `in <= t`. With no usable signal from any past position, "attend equally to everything you can see" is the provably optimal heuristic — every attention head in the stack arrives there independently, without any outside coordination. Watching this happen is itself a small lesson in "gradient descent finds the best available behaviour, even when the best available behaviour is 'give up trying to differentiate.'"

If instead of uniform random digits you run the fixed-input default (`--random=0`), held-out accuracy is not a meaningful quantity (there is only one training example, no test set), but the training loss will drop to near-zero: the model memorizes the single sequence. Memorization is easy; learning structure from i.i.d. noise is impossible. Real ML lives on the spectrum between those two extremes.

Preview: Chapter 17

We have a trained next-token predictor. What we do not yet have is a way to *use* it the way a human uses ChatGPT — type a prompt, get text back, token by token, each output fed in as the input to the next. That is a short loop around `forward()`, and it is what turns a parallel-trained model into a generative one. Chapter 17 adds that loop, and along the way makes two important observations about what "generation" really means and what the computational cost structure of naive (uncached) generation looks like. Chapter 22 will eventually fix the naivety with a KV cache. But the next three chapters work with the naive version, because it is easier to see.

> **Read along:** the Chapter-16 code delta is spread across six parts of `maxai.cpp`:
>
> - **Part 3** — new `Task::NextToken` enum entry (with an inline comment locating it in the larger story), `parse_task` recognises it, the usage string mentions it, `effective_seq_len()` defines the loss-masking rule, and `parse_args()` now refuses to start on `--task=next_token --causal=0` with an explanatory `die()`.
> - **Part 9** (`compute_loss`) — loop now runs over `[0, T_eff)` and divides by `T_eff`. One comment-line change; two lines of arithmetic change.
> - **Part 10** (`backward`) — `g_logits` is set to `(probs - onehot) / T_eff / batch` for `t < T_eff` and explicitly zeroed for the remaining rows. Several paragraphs of commentary around a handful of lines of arithmetic.

- **Part 12** (`make_target`) — the one new case in the switch, surrounded by a long comment walking through the three options and why we took option 3.
- **Part 14** (`count_correct`) and **Part 15** (`eval_heldout_accuracy`) — both loop over `[0, T_eff)` ; the latter also adjusts its denominator to `NUM_EVAL * T_eff` .

`diff minai.cpp maxai.cpp` shows every change. The code delta is on the order of fifteen active lines; the comments around them are considerably more. That ratio is deliberate — the whole point of Section 2's code is that the *ideas* fit in fifteen lines, and the commentary is what earns the ideas.

Chapter 17 — Inference as a feedback loop

What this chapter does

After Chapter 16 we have a trained next-token predictor — a function that takes a sequence of tokens and returns, for every position, a probability distribution over what the next token might be. What we do *not* have is any way to use it the way a human uses ChatGPT: type a prompt, get text back, one token at a time. That gap is what this chapter fills. The fix is a four-line loop around `forward()` . No new math. No new weights. The architecture does not change one bit. What changes is that we start *using* the model sequentially.

This is worth sitting with for a moment, because it is the first time in the book a model has two modes. During **training** the model consumes an entire sequence at once (teacher forcing, Chapter 16) and every position contributes simultaneously to the gradient. During **inference** — the mode this chapter adds — the same model runs over and over, each run producing exactly one new token, which then gets appended to its own input for the next run. Same weights. Same forward pass. Completely different usage pattern. The word *autoregressive* refers to this usage pattern: the model regresses on its own prior outputs. Chapter 16 made the model autoregressive-by-training; Chapter 17 makes it autoregressive-in-use.

The loop, in its simplest form

Here is the entire algorithm:

```
function generate(prompt, max_len):
   tokens = prompt                              # length P
   while len(tokens) < max_len:
       probs   = forward(tokens)                # [len(tokens), VOCAB]
       next_tok = argmax(probs[len(tokens) - 1])
       tokens.append(next_tok)
   return tokens
```

Six lines. One call to `forward()` per generated token. One argmax per call. Every real LLM on earth — ChatGPT, Claude, the open-weight Llama running on someone's gaming laptop, the AI autocomplete on a programmer's keyboard — is built around a loop exactly this shape. The specific differences from our toy version are, in order:

1. **Sampling instead of argmax.** Chapter 18 replaces `argmax` with a draw from `probs[·]`. Argmax is called *greedy decoding*. It maximizes per-token log-likelihood but almost always produces worse text than sampling does. More on why in a moment.
2. **A much larger vocabulary.** Chapter 19 grows `VOCAB` from 10 digits to a character-level alphabet; Chapter 21 introduces a tiny byte-pair encoder. Once the vocabulary is a few tens of thousands of subwords, `probs[t]` is a 50000-wide vector and argmax/sample over it is still a constant-time operation — no new machinery required.
3. **Variable-length termination.** Chapter 19 also introduces an end-of-sequence symbol so generation can stop on a learned signal rather than on a fixed `max_len`.
4. **A KV cache.** Chapter 22 replaces the repeated from-scratch `forward()` calls with an incremental one that only does the new position's worth of work. This is the single largest performance win in LLM inference engines; without it, serving a 128k-context model would be impossible.

But the skeleton — *run forward, pick a token, append, repeat* — is everything. Learn the skeleton, and every variant is a decoration.

Why the loop works at all

A reader new to autoregressive inference tends to ask, correctly, "what are you feeding the model at positions after the prompt? We haven't generated those yet." The MaxAI answer is: *zeros, which are ignored, because of the causal mask*.

Recall from Chapter 5 that attention is a weighted sum over input positions, and that the causal mask (Chapter 7, Chapter 14 Step 4) zeros out every weight for positions strictly greater than the current one. So at position `t` the output depends *only* on tokens `[0..t]`. Whatever we have stored at positions `t+1..T−1` is never read. The loop exploits this property exactly:

- At iteration `k` the filled portion of the buffer is positions `0..current_len−1`.
- Positions `current_len..T−1` hold stale zeros (from initialization) or earlier generated tokens.
- We run `forward()` over the whole buffer, because `forward()` is a fixed-shape function and we do not want to rewrite it.
- We read only `probs[current_len - 1]` — the distribution for the next token, derived from a causal context of exactly the filled prefix.

`forward()` does waste some compute on the outputs of positions `current_len..T−1`, and those outputs are genuinely meaningless — they depend on the garbage in the unfilled region. But we never read them, so the meaninglessness is invisible. Causal attention transforms "I have a partial sequence" into "I have a full-length buffer I only partially read from," and the distinction stops mattering.

This little property of causal attention — that the right half of the sequence can be anything you like and the left half's outputs are unaffected — is the reason inference can be implemented as a fixed-size buffer with a moving write pointer, and it is a surprisingly large engineering win. Pytorch's `generate()` method does exactly this. JAX's `scan` does exactly this. llama.cpp does exactly this. MaxAI does exactly this.

The O(N²) problem, and why Chapter 22 exists

Look again at the pseudocode. Every iteration calls `forward()` on the whole current sequence. The model does not know, and has no way of knowing, that positions `0..current_len-2` produced the identical keys and values the last time we called it. It redoes all that work from scratch. Iteration 1 touches 1 position of attention work; iteration 2 touches 2; … iteration `T - P` touches `T`. Total attention work to produce `T - P` tokens: roughly `O(T²)`.

For `T = 8` this is comically cheap. A full rollout is maybe 50 microseconds on any modern laptop. For `T = 128000` — the context length real LLMs now support — naive autoregressive inference is catastrophic. The quadratic cost dominates the actual per-token compute by orders of magnitude, and serving a model at useful speed becomes impossible.

The fix is the **KV cache**. Cache the per-position `K` and `V` vectors the first time they are computed; on subsequent iterations, only compute `K` and `V` for the one new position, and attend over the concatenation of the cached tensors and the new one. This reduces the per-token attention cost from `O(T²)` total to `O(T)` total — which is the minimum possible, since every new token must at least read the T-long history. The cache itself grows linearly with the sequence; at long contexts it becomes the dominant memory cost of serving a model (Chapter 12 of Section 1 covered this from the memory-wall angle).

Chapter 22 adds the cache to MaxAI. For the next five chapters we eat the quadratic cost because at `T = 8` it is invisible and because running without the cache makes the learning story cleaner.

What "stops" the loop

In MaxAI the loop stops when `current_len == cfg.seq_len`. Once the buffer is full the causal context is full and there is nowhere to put the next token. This is the simplest possible termination condition and it is fine for a toy model.

Real LLMs terminate three different ways, in various combinations:

1. **End-of-sequence token.** A special symbol `EOS` is included in the vocabulary. The model is trained so that, at the correct end of a response, it assigns high probability to `EOS`. The inference loop watches for `EOS` in the sampled token and stops if seen. Chapter 19 adds this to MaxAI (for free, since we are already resizing the vocabulary).
2. **Hard cap.** A maximum number of generated tokens, chosen by the caller (`max_new_tokens` in most APIs). This is the backstop: even if the model would never emit `EOS`, the serving system stops at the cap and returns a truncated response.
3. **Sliding window / ring buffer.** When the caller wants arbitrarily long outputs, old tokens are dropped from the *left* of the context as new ones are generated. This is how modern long-context inference

engines handle “conversations that exceed the context window.” It is lossy — the model forgets the earliest prefix — but it is the only thing that works at truly long horizons without specialized architectures.

We stick with option 2 (= the fixed `seq_len`) for this chapter and add option 1 in Chapter 19.

Running the rollout: three instructive cases

`maxai.cpp` now calls `demo_generate()` automatically at the end of a run whenever the task is `next_token`. The three invocations below are each set up to teach a different piece of the story.

Case 1 — memorization as recitation

```
./maxai --task=next_token --causal=1 --steps=800
```

Fixed input mode: the single training example is `tokens[t] = t`, so training converges to near-zero loss and the model has memorized the sequence `0,1,2,3,4,5,6,7`. The generation demo then prompts with `[0]` and watches the model recite:

```
 prompt  (len=1): 0
step  0: context=[0]              pick=1  (top3: 1@1.00  2@0.00  7@0.00)
step  1: context=[0 1]            pick=2  (top3: 2@0.99  3@0.00  1@0.00)
step  2: context=[0 1 2]          pick=3  (top3: 3@0.99  2@0.00  5@0.00)
step  3: context=[0 1 2 3]        pick=4  (top3: 4@1.00  2@0.00  3@0.00)
step  4: context=[0 1 2 3 4]      pick=5  (top3: 5@1.00  3@0.00  6@0.00)
step  5: context=[0 1 2 3 4 5]    pick=6  (top3: 6@0.99  5@0.00  8@0.00)
step  6: context=[0 1 2 3 4 5 6]  pick=7  (top3: 7@0.99  1@0.00  9@0.00)
final : [0 1 2 3 4 5 6 7]
```

Every distribution is sharp — the chosen token sits at probability `≈ 1.00` and the rest share the remaining microscopic mass. This is not a surprise: the model has a single training example and zero regularization, so the softmax over logits is free to peak arbitrarily hard on the memorized target. In production this is the “overfitting to the training set” regime. In pedagogy it is perfect: you get to watch memorization *as it executes*.

Case 2 — no information, no prediction

```
./maxai --task=next_token --causal=1 --random=1 --batch=16 --steps=3000
```

Uniform random training. From Chapter 16 we know the loss ceiling is `log(10) ≈ 2.30` and the accuracy ceiling is 10%. The generation demo uses a deterministic three-token prompt and watches the model guess:

```
 prompt  (len=3): 1 5 3
step  0: context=[1 5 3]            pick=1  (top3: 1@0.10  3@0.10  4@0.10)
step  1: context=[1 5 3 1]          pick=9  (top3: 9@0.10  7@0.10  4@0.10)
step  2: context=[1 5 3 1 9]        pick=4  (top3: 4@0.10  9@0.10  0@0.10)
step  3: context=[1 5 3 1 9 4]      pick=7  (top3: 7@0.11  5@0.10  9@0.10)
step  4: context=[1 5 3 1 9 4 7]    pick=5  (top3: 5@0.10  2@0.10  4@0.10)
final : [1 5 3 1 9 4 7 5]
```

Notice the probabilities. The top three are all `0.10` — the distributions are, to within training noise, uniform. The model has learned the *only* useful thing it could learn on i.i.d. random digits: the marginal distribution over the next token, which is exactly uniform. Argmax of a uniform distribution is whichever tie-break wins (here, the smallest index among tied entries), so the "pick" column is basically a deterministic tie-break rather than a real prediction. This is what zero-information inference looks like in practice — the machinery runs, the loop spits out tokens, and the output carries no more signal than rolling a ten-sided die at each step. Chapter 14 Step 4's argument, now visible at inference time: *if the information the model would need to do better is not in its training data, no amount of architecture or compute can manufacture it*.

Case 3 — off-distribution prompt, and hallucination in miniature

```
./maxai --task=next_token --causal=1 --steps=800 --gen_prompt="2 4 6"
```

Here things get interesting. The model is the Case-1 model — it has memorized the single training example `[0,1,2,3,4,5,6,7]`. But we are now handing it a prompt it has *never* seen: `[2, 4, 6]`. It has no memorized continuation for that prefix. Watch what happens:

```
 prompt  (len=3): 2 4 6
step  0: context=[2 4 6]              pick=3  (top3: 3@0.37  7@0.26  5@0.13)
step  1: context=[2 4 6 3]            pick=4  (top3: 4@0.98  5@0.00  3@0.00)
step  2: context=[2 4 6 3 4]          pick=5  (top3: 5@0.99  6@0.00  3@0.00)
step  3: context=[2 4 6 3 4 5]        pick=6  (top3: 6@1.00  5@0.00  8@0.00)
step  4: context=[2 4 6 3 4 5 6]      pick=7  (top3: 7@0.99  1@0.00  9@0.00)
final : [2 4 6 3 4 5 6 7]
```

Three things to notice, in order of importance.

At step 0 the distribution is genuinely uncertain — `3@0.37`, `7@0.26`, `5@0.13`. This is the model looking at an unfamiliar prefix and being unable to commit. Nothing in training told it what comes after `[2, 4, 6]`, so the softmax over the output logits comes out spread across several plausible-looking continuations. That spread is a form of honesty the model has no choice about: without a sharp signal from the data, the logits are small and softmax over small logits is flat.

From step 1 onward the distribution snaps back to near-certainty — `4@0.98`, `5@0.99`, `6@1.00`, `7@0.99`. What happened? The *position embeddings* (Chapter 4) happened. Position 3 in the memorized training sequence was the digit `3`; position 4 was `4`; position 5 was `5`; and so on. Those facts are baked

into the weights. When the model sees position 3 — regardless of what the token *at* position 3 actually is — its priors say "position 3's next token is the digit 4." It commits to that and plows ahead.

The final generation is `[2 4 6 3 4 5 6 7]` — the first three tokens from the prompt, then the memorized completion starting at position 3. This is, in miniature, how LLM **hallucination** works. The model is presented with a prefix its training has not prepared it for; the part of it that knows "the world" (its stored priors) overrides the part that is supposed to respond to context; output comes out confident but wrong. Everyone who uses a production LLM has seen this behaviour — the model asserts, with full conviction, something that is not true. The mechanistic reason is visible in this eight-token toy: when context signal is weak and priors are strong, priors win. A confidence score on the prediction (step 0's `0.37` peak) would have warned the user that the model was guessing; the downstream steps' `≈ 1.00` peaks would have hidden it completely.

This is the single most important inference-time observation in the book. *The model is not "aware" it is guessing.* Its confidence on a step is a function of the logits at that step, not a function of whether its answer is actually right. Real products that matter (medical assistants, code assistants) put enormous effort into detecting this gap between "model confident" and "model correct" — techniques with names like calibration, conformal prediction, retrieval-augmented generation. Chapter 22's KV cache and Chapter 19's EOS token are inference-time engineering. This — detecting when the model is out of distribution — is inference-time *epistemology*, and it is a much harder problem.

Why argmax is not what real LLMs do

Greedy argmax decoding has a well-known pathology: it produces boring, repetitive text. The highest-probability next token after "the cat sat on the" is probably " mat"; after "the cat sat on the mat, the cat sat on the" it is probably " mat" again; the greedy loop walks directly into a rut and stays there. You can see a miniature version of this in Case 2's rollout: argmax of a roughly-uniform distribution is whichever class happens to tie-break highest, so the output sequence is not random — it is a deterministic walk through a near-uniform distribution.

The fix is to *sample* from `probs[t]` rather than take the argmax. A single sample from a uniform distribution is actually uniform; a sample from a peaked distribution is usually the peak, occasionally something else. Sampling turns "the model is confident → same token" into "the model is confident → usually the peak token, rarely something else" — which breaks the ruts, introduces variety, and (because the distribution itself is mostly correct) does not usually sacrifice quality.

Chapter 18 makes this change. It also introduces two decorations every production LLM supports — **temperature** (a scalar that sharpens or flattens the distribution before sampling) and **top-k / top-p** (truncations that throw away the long tail of very-improbable tokens before sampling, so the one-in-a-thousand flukes never happen). The combination is roughly: `distribution = top_p(top_k(softmax(logits / temperature)))`, and every serving API exposes these as knobs because they are the only knobs a user has to shape output style without retraining.

For Chapter 17 we leave that for later and run the greedy loop. Greedy is the simplest thing, it makes the mechanics unambiguous, and the Case-3 demo above is arguably more striking with argmax than it would be

with sampling — the hallucinated continuation is crisp and deterministic rather than jittery.

What is built and what is not

After Chapter 17, MaxAI is a tiny but real generative language model. You can build it, train it, and ask it to produce token sequences from a prompt. The program even has a command-line flag (`--gen_prompt="..."`) for user-supplied prompts. If you squint this is the same user-facing interface ChatGPT exposes, minus the web server, minus the token streaming, minus the preference tuning, and (crucially) minus a vocabulary that can represent words.

What is *not* built, in order of how soon it arrives:

- Chapter 18: sampling with temperature / top-k / top-p (real decoding).
- Chapter 19: a character-level vocabulary and an EOS token (real-alphabet prompts and natural stops).
- Chapter 20: training on real English text (data with actual structure).
- Chapter 21: a BPE tokenizer (the last hop from character-level toys to production vocabularies).
- Chapter 22: a KV cache (serving-scale performance).

Each of these is additive; `maxai.cpp` keeps compiling and running after each addition. You can stop at any chapter and have a complete, shippable thing. That is part of why we are growing the code chapter by chapter rather than presenting it finished: every stopping point is itself a demonstration of a real system.

> **Read along:** Part 18b of `maxai.cpp` is the entire Chapter-17 code delta. Three new functions: `parse_prompt_digits()` (string → token list), `generate()` (the loop), and `demo_generate()` (the wrapper that picks a default prompt when the user does not supply `--gen_prompt=`). `main()` gains one line: if the task is `Task::NextToken` , call `demo_generate()` after the existing `demo()` . Part 3 gains the `cfg.gen_prompt` field and its parser. The loop itself — the thing this entire chapter is about — is about twelve lines of C++ in the middle of `generate()` . The rest of the Part is commentary and printing.
>
> `diff minai.cpp maxai.cpp` now shows the Chapter 16 delta plus the Chapter 17 delta. The overall file has grown by roughly 200 lines, most of which are comments, and `maxai.cpp` still compiles with no warnings under `clang++ -std=c++17 -O2 -Wall -Wextra` . Nothing in `minai.cpp` changed; anyone starting the book today can still read just Section 1 and stop cleanly.

Chapter 18 — Sampling, not argmax

Why argmax is not what real LLMs do

Chapter 17's `generate()` loop picks the highest-probability token at every step. This is called **greedy decoding** (or, equivalently, **argmax decoding**, or "temperature-zero decoding") and it has three pathologies that every production LLM sidesteps:

1. **It is deterministic.** The same prompt always produces exactly the same output. Humans asking a chatbot the same question twice expect variation; greedy decoding gives identical responses. More importantly, deterministic sampling cannot explore — the model cannot offer multiple candidate answers, cannot "try again with a different angle", cannot be coaxed into creativity. Every interesting product property on top of a raw LLM (re-prompting, re-ranking, tree-of-thought, Monte Carlo rollouts for reasoning) requires a stochastic decoder underneath.

2. **It produces repetitive, low-quality text.** Holtzman et al. (2019) showed experimentally that argmax-decoded neural text is wildly different from human-written text in statistical properties — argmax produces phrases like "the the the" or "I am I am I am" far more than humans do, because once a high-probability word has appeared the same word is often the highest-probability continuation. The model collapses into self-reinforcing loops. Real text is *supposed* to sometimes include lower-probability words; argmax never does.

3. **It does not match the generator it was trained to be.** The training objective is "assign high probability to what actually follows." A model that minimises cross-entropy is, in the limit of infinite data, the true conditional distribution. Running the argmax of that distribution is not the same as sampling from it — the argmax is only the most-likely single continuation, and the most-likely single continuation is not what the generator was trained to produce. Sampling is. Under Shannon's source coding theorem, a generator matching the source entropy produces output that looks hard to compress; the signature of natural output, in other words, is exactly the variety greedy decoding refuses to give.

The fix is simple to state and needs a small amount of machinery to implement cleanly: *draw a random sample from the probability distribution at each step, rather than taking its argmax*.

Sampling from a categorical distribution

At each generation step the model produces a vector of probabilities `p[0..VOCAB-1]` that sums to 1. We want a random index `i` drawn with probability `p[i]`. The textbook algorithm — and the one MaxAI uses — is **inverse CDF sampling**:

```
 draw u uniform in [0, 1)
walk i = 0, 1, 2, ...
accumulating sum s = p[0] + p[1] + ... + p[i]
return the first i for which u < s
```

For `VOCAB = 10` this is ten multiplies and a comparison. For `VOCAB = 50000` it is fifty thousand of each — still trivial, still far less work than one forward pass. The cost of sampling is negligible at every scale; the cost of not sampling is the quality drop above.

A subtle point worth pausing on: this is literally what dice are. “Roll a die” is inverse-CDF sampling from a uniform discrete distribution. Real languages are dice with 50,000 faces, each weighted by how likely the next word is given the sentence so far. A language model’s job is to learn the weighting; the sampler’s job is to roll.

Temperature: a single knob that sharpens or flattens the distribution

A fully-trained model produces a particular shape of distribution at each step — sometimes sharp (one token dominant at probability 0.9, the rest sharing 0.1), sometimes flat (ten tokens each at 0.1). You cannot retrain the model to change that shape at inference time. But you *can* scale the logits before softmax. One scalar, called **temperature**, does exactly that.

```
probs = softmax(logits / T)
```

For `T = 1` the distribution is whatever the model was trained to produce. For `T < 1` the division *increases* the spread between logits; softmax amplifies the peak; the distribution gets sharper. As `T → 0` the distribution approaches a one-hot at the argmax, which is why `temperature=0` is synonymous with greedy decoding (`sample_token()` actually short-circuits to argmax as a fast path). For `T > 1` the division *shrinks* the spread; softmax flattens; the distribution moves toward uniform as `T → ∞`.

In one-sentence pop summaries: “temperature is a creativity knob.” That is not wrong. More precisely: temperature trades off *sampling concentration* for *sampling diversity*. At `T = 0` every choice is the model’s top pick, which is conservative and boring. At `T = 1` you draw from the model’s honest distribution, which is what the objective trained it to be. At `T = 2` you oversample the long tail — you get more rare tokens than the data actually justifies, which can be useful for brainstorming and harmful for factual answers. Production LLM APIs almost always default to `T = 1` or `T = 0.7`.

Top-K: truncate the long tail by count

Temperature alone has a known failure mode. If you set `T = 1.5` to get more variety, you *also* oversample genuinely implausible tokens — the bottom of the distribution gets boosted alongside the middle. In a 50,000-word vocabulary, even the lowest-probability 1% represents 500 words, any of which would make the sentence ungrammatical. Occasionally one of them gets drawn, and the whole generation derails.

The simplest fix is to *just not sample from the long tail*. **Top-K sampling** keeps the K highest-probability tokens, zeros the rest, renormalises, and samples from the truncated distribution:

```
 take the top K tokens by probability
set every other token's probability to 0
renormalise so the kept tokens sum to 1
sample from the truncated distribution
```

Typical K is 40. The idea is that the model’s top 40 candidates are “reasonable,” and the other 49,960 might be one-in-a-million flukes we do not want. Combined with moderate temperature this gives you diversity

within the plausible set without tail accidents. MaxAI exposes `--top_k=N`; the default is 0 (disabled).

Top-P (nucleus) sampling: truncate by cumulative probability

Top-K has one awkwardness. If the distribution is sharp, `K = 40` is massively too many — most of the 40 have near-zero probability and will never be picked anyway. If the distribution is flat, `K = 40` is too few — we have cut away tokens that genuinely deserved a chance. The fix is to adapt the truncation to the distribution.

Top-P sampling — also called **nucleus sampling** — sorts tokens by probability in descending order, then keeps the smallest prefix whose cumulative probability reaches a threshold `p`:

```
 sort tokens by probability, descending
walk through them accumulating cumulative probability
stop at the first token where cumulative prob >= top_p
keep all tokens up to and including that point, zero the rest, renormalise
sample from the truncated distribution
```

For `top_p = 0.9`, a sharp distribution might keep just the top 2 tokens (they already sum to 0.92); a flat distribution might keep 30 tokens (before the cumulative reaches 0.90). This gives the "keep the plausible set" intuition without having to guess a magic K. Nucleus sampling was introduced in Holtzman et al. 2019 and is now the default decoding strategy in essentially every modern LLM serving stack. MaxAI exposes `--top_p=F`; the default is 1.0 (disabled).

The production pipeline

Every mainstream LLM inference API exposes the same four knobs in the same order:

```
sample( renormalise( top_p( top_k( softmax( logits / temperature ) ) ) ) )
```

Reading right to left: divide by temperature, softmax, apply the top-K cut, apply the top-P cut, renormalise to sum to 1, sample. The order matters: applying top-P to a temperature-1 distribution is a different cutoff point than applying it to a temperature-2 distribution, because the shape has changed. `sample_token()` in `maxai.cpp` follows the same order, to the letter.

Any of the four can be disabled by setting it to the identity value: `temperature=0` short-circuits to argmax; `top_k=0` skips the K-truncation; `top_p=1.0` skips the P-truncation. A typical production setting is `T=0.7, top_k=0, top_p=0.95`.

Four demonstrations

`maxai.cpp` now exposes `--temperature`, `--top_k`, `--top_p`, and `--sampling_seed`. The four runs below each teach a different piece of the sampling picture.

Demo 1 — sampling on a sharp distribution behaves like greedy

```
./maxai --task=next_token --causal=1 --steps=800 --temperature=1.0
```

This is the memorized fixed model (same training as Chapter 17 Case 1). The distribution at every step has one token near probability 1.00 and the rest near 0.00. Sampling from a near-one-hot distribution returns the peak with overwhelming probability; the rollout is identical to greedy: `[0 1 2 3 4 5 6 7]`. Even at `T = 2.0` the distribution does not flatten enough to matter — the logit separations are *so* large (gradients ran freely on one example for 800 steps) that dividing the logits by 2 still leaves a near-one-hot after softmax. You cannot coax diversity out of an overconfident model with modest temperature; the only escape is a very high T. Try `--temperature=100`:

```
 step  0: context=[0]              pick=7  (top3: 1@1.00  2@0.00  7@0.00)
step  1: context=[0 7]            pick=1  (top3: 2@0.92  1@0.04  3@0.03)
step  2: context=[0 7 1]          pick=6  (top3: 2@0.49  3@0.48  1@0.02)
step  3: context=[0 7 1 6]        pick=2  (top3: 4@0.36  2@0.34  7@0.16)
...
final : [0 7 1 6 2 9 3 3]
```

This is fascinating. At `T = 100` the sampler at step 0 genuinely goes off-script (picks `7`, not the argmax `1`). From then on the prompt `[0, 7]` is off-distribution — the model has never seen this prefix — and the model's *raw* distributions (the top-3 column, still printed at temperature 1 for clarity) get progressively flatter as the context drifts further from what training produced. Steps 2, 3, 4 show distributions that are genuinely uncertain (`0.49 / 0.48 / 0.02`, `0.36 / 0.34 / 0.16`). The model is in the same positional-prior fallback we saw in Chapter 17 Case 3 — "I do not know the answer, I will guess based on position." Temperature bit; hallucination cascaded; the confident top-3 of steps 5–7 does *not* mean the prediction is correct, only that the model has regained its prior. Every property you could want to illustrate about sampling and hallucination is visible in those seven lines.

Demo 2 — sampling on a flat distribution gives real variety

```
./maxai --task=next_token --causal=1 --random=1 --batch=16 --steps=3000 --temperature=1.0
```

Uniform-random training, model at the 10% ceiling. Each step's distribution is approximately uniform. Sampling from uniform *is* uniform — which is what we finally see: a genuinely varied rollout like `[1 5 3 7 1 6 2 9]`, unlike Chapter 17's deterministic tie-break walk `[1 5 3 1 9 4 7 5]`. Same model, same prompt, same forward passes — but with sampling the output is actually random, not pseudo-random-via-tie-break. This is the regime where sampling does its most visible work.

Demo 3 — top-P as an adaptive filter

```
./maxai --task=next_token --causal=1 --random=1 --batch=16 --steps=3000 \
       --temperature=1.0 --top_p=0.5
```

Same model as Demo 2. Top-P at 0.5 keeps the smallest prefix whose cumulative probability reaches 0.5. Because the distribution is approximately uniform at 0.10 per class, the nucleus is about 5 tokens — half the vocabulary. Sampling draws from those 5, not from all 10. You get constrained diversity rather than unconstrained diversity; roll counts are still random, but the space being rolled over is cut in half. In a real LLM with much sharper distributions the nucleus would usually be two or three tokens wide, and this filter would prevent long-tail accidents.

Demo 4 — reproducibility via seed

```
./maxai --task=next_token --causal=1 --random=1 --batch=16 --steps=3000 \
       --temperature=1.0 --sampling_seed=0xDEADBEEF
```

Exactly the same model as Demo 2, different `--sampling_seed`, different rollout. This is worth demonstrating because reproducibility is a real engineering concern. Real LLM serving stacks expose a `seed` parameter precisely so that users can reproduce or differentiate runs without retraining the model. Under the hood it does what `maxai.cpp` does: a separate RNG stream, distinct from the training/eval ones, seeded by a parameter the user controls.

When does sampling *really* matter?

Not on memorized toy sequences; not on pure i.i.d. noise. Sampling's big value shows up on real text, where the model's distributions are *moderately* sharp — not one-hot, not uniform, but concentrated on a handful of plausible continuations. That is the regime where "pick the argmax" and "sample from the distribution" produce visibly different quality, and where temperature/top-K/top-P are the main user-facing levers between dull-and-safe and creative-and-risky output. Chapter 20 moves us to that regime by switching training from random digits to actual English. Chapters 18 and 19 are the machinery you will want in place by the time you get there.

What is built and what is not

After Chapter 18, MaxAI can sample. It exposes the exact four knobs that every hosted LLM API exposes — temperature, top-K, top-P, seed — and implements them in the order real serving stacks do. What it still cannot do is emit text that a human reads as text, because the vocabulary is still ten digits. Fixing that is Chapter 19.

> **Read along:** All of Chapter 18's code lives in `maxai.cpp` Part 18b (the chapter now shares the header with Chapter 17 since both concern the inference path). New pieces: - `sampling_rng_state` — dedicated xorshift32 stream, initialized from `cfg.sampling_seed` in `main()`. - `rand_uniform_unit()`

> — inverse-CDF's uniform draw, 24 bits of mantissa precision. - `sample_token()` — the whole pipeline: temperature scaling, softmax-with-max-trick, optional top-K truncation, optional top-P truncation, inverse-CDF sample. About 80 lines including comments. - `generate()` (Chapter 17 function) — one line changed: the argmax call became a `sample_token()` call with the configured knobs. - `main()` — one line added to initialise `sampling_rng_state` from `cfg.sampling_seed` after `parse_args()`.
>
> Config gained four fields (`temperature`, `top_k`, `top_p`, `sampling_seed`) with defaults that preserve the Chapter-17 greedy behaviour. `parse_args()` gained four `--` parsers; `print_usage()` gained five lines of help. And the code delta still compiles cleanly under `-Wall -Wextra`.

Chapter 19 — Widen the vocabulary

What this chapter does

Chapter 19 is the first chapter where `maxai.cpp` starts looking less like MinAI and more like a small real LLM. The change is simple to state: we expand the vocabulary from 10 digits to 28 character-level tokens (26 lowercase letters, space, and a reserved end-of-sequence symbol). The model architecture, training loop, loss, sampler, and generation loop — none of them change. The only thing that changes is `VOCAB` (which is now a runtime variable between 10 and 28), the way we encode text into token ids before feeding the model, and the way we decode token ids back to visible characters for display.

This is, mechanically, the entire "getting to text" step. After Chapter 19 you can type

```
./maxai --task=next_token --causal=1 --vocab=chars --gen_prompt="hello "
```

and the program will tokenize the prompt into integers, feed them to the same transformer we have been building for three chapters, sample tokens autoregressively, and print the continuation as a string. The text will not be meaningful yet — training is still on toy sequences — but the pipeline is end-to-end text-in, text-out.

Chapter 20 will make the text meaningful by swapping the training data from toy memorization / uniform-random noise to actual English. Chapter 21 will replace our character-level tokenizer with a byte-pair encoder so the vocabulary looks like a real LLM's. Chapter 22 will add the KV cache so the generation loop runs at the speed a real LLM needs. Chapter 19 is the joint that lets those three later chapters plug in without rewriting the core.

The runtime refactor

Until Chapter 18 the file declared `constexpr int VOCAB = 10;` at the top, and every static array (`token_emb[VOCAB][D_MODEL]`, `Wout[D_MODEL][VOCAB]`, `logits[SEQ_LEN][VOCAB]`, `probs[SEQ_LEN][VOCAB]`, and all of their gradient twins) was sized with that constant. You cannot change a `constexpr` at runtime, and C++ does not allow variable-length arrays at file scope, so "make the vocab a runtime choice" is surgical: it requires a compile-time *upper bound* for the declarations and a separate *current value* that the rest of the code reads.

```
constexpr int MAX_VOCAB = 64;     // compile-time upper bound; arrays sized to this
int VOCAB               = 10;     // runtime value, assigned by parse_args()
int SAMPLEABLE_VOCAB    = 10;     // range of tokens that random data generators emit
int EOS_TOKEN           = -1;     // -1 = this vocabulary has no EOS
```

Every array declaration in the file now uses `MAX_VOCAB`. Every loop bound, modulo, and function argument that cares about the live distribution size uses `VOCAB`. Every random data generator uses `SAMPLEABLE_VOCAB` — a deliberately smaller range that excludes reserved tokens (like EOS) so they never appear as accidental noise inside a training sequence. And `EOS_TOKEN` is a sentinel: negative means "no EOS in this vocabulary" and anything else is the specific token id to treat as end-of-sequence.

Writing this down in plain terms: the architecture of the model has a *capacity* for up to `MAX_VOCAB` different symbols (64, in our case), but on any given run it uses exactly `VOCAB` of them. The remaining rows of the token-embedding table and of the output-projection matrix simply sit at whatever their initial values were — the forward pass never reads them, because the output loop only runs from 0 to `VOCAB - 1`. The unused capacity costs a trivial amount of memory and lets us switch vocabularies without recompiling. Real serving systems do something similar, typically with heap allocation rather than static arrays, but the idea — "size for the maximum you will ever need, use the current amount each call" — is the same.

The two vocabularies

`maxai.cpp` now understands two preset vocabularies, selected by `--vocab=<name>`:

`--vocab=digits` (default). Ten tokens, one per decimal digit. `VOCAB = 10`, `SAMPLEABLE_VOCAB = 10`, `EOS_TOKEN = -1`. This preserves Chapters 16–18 exactly — every demo transcript from earlier in this book still matches the new `maxai.cpp` byte-for-byte. If you want to remember how the earlier chapters felt, this is still the mode to run.

`--vocab=chars`. Twenty-eight tokens laid out as:

token id	symbol
0..25	lowercase 'a'..'z'
26	' ' (space)
27	`<EOS>`

`VOCAB = 28`, `SAMPLEABLE_VOCAB = 27` (so EOS never shows up as accidental noise in random data), `EOS_TOKEN = 27`. The encoder folds uppercase letters to lowercase for the reader's convenience; anything else in a prompt (digits, punctuation, emoji) is rejected at parse time with an explanatory error. Chapter 20 will widen the letter set slightly to include common punctuation when we move to real text; Chapter 21 will replace this whole layout with a byte-pair encoder.

The arithmetic of the model is completely blind to what any of these tokens *mean*. The token embedding is a lookup table — Chapter 4 of Section 1 — and nothing in training reads the "meaning" of the embedding rows. The model learns vectors for token-id-0, token-id-1, etc., and whatever those vectors end up encoding is a pure function of what role those tokens play in the training data. In digit mode the vectors come to encode "digit 3 tends to follow digit 2"; in char mode they come to encode "letter 'b' tends to follow letter 'a'." Same mechanism, same training loop, different statistical meaning — and only because the training data changed.

The encoder and decoder

Chapter 15 of Section 1 spent its whole length on tokenization because tokenization is a big topic at production scale. Ours is trivial:

```
static int encode_char(char c) {
    if (c >= 'a' && c <= 'z') return c - 'a';
    if (c >= 'A' && c <= 'Z') return c - 'A';   // case-fold for kindness
    if (c == ' ')             return 26;
    return -1;                                   // unknown character
}

static char decode_token_char(int tok) {
    if (tok >= 0 && tok < 26) return (char)('a' + tok);
    if (tok == 26)            return ' ';
    return '?';                                  // EOS and anything else
}
```

Eight lines. No edge cases, no merge rules, no ambiguities. The character-level tokenizer is the simplest possible lossless text-to-token mapping, and for 28 symbols it is exactly what we want. The production-scale tokenizer (BPE in Chapter 21) is a sophisticated generalization; everything structurally important about tokenization is already here.

Two small helpers wrap these for display. `print_token(tok)` writes one token: a character in char mode or `"<EOS>"` for the special EOS id, or just `"%d "` in digit mode to preserve Chapter 16–18 transcripts. `print_tokens(arr, n)` wraps the whole sequence in quotes in char mode (so leading spaces stay visible) and leaves digit output unchanged.

EOS: closing the loose end from Chapter 16

Chapter 16 had a small honesty problem. At position `t = T - 1` in a next-token task, there is no `t + 1` to predict, so the loss would have to either score something meaningless or drop the position entirely. Section 1

of this book laid out three solutions and picked the expedient one (option 3, mask the position out of the loss) with a promise to revisit.

Chapter 19 delivers the promised revisit by taking option 2: *add an explicit end-of-sequence symbol and train the model to emit it at the final position*. The mechanics are straightforward:

```
case Task::NextToken:
    for (int t = 0; t < T - 1; ++t) target[t] = tokens[t + 1];
    target[T - 1] = (EOS_TOKEN >= 0) ? EOS_TOKEN : tokens[T - 1];
    break;
```

In char mode `EOS_TOKEN = 27`, so the final target becomes the EOS id; `effective_seq_len()` now returns `T` (every position is scored, including the last), and the model receives honest gradient signal telling it "position T-1 should be EOS." In digit mode `EOS_TOKEN = -1`, so we fall back to the Chapter-16 placeholder and mask the position out exactly as before. Both regimes coexist in the same file, selected at runtime by the vocabulary flag.

There is a corresponding change to generation. `generate()` now checks each sampled token against `EOS_TOKEN`, and if it matches, the loop breaks *before* appending the token to the output (EOS is a control signal, not content). Chapter 20 will actually exercise this — with variable-length training data, the model will learn to emit EOS at different positions in different sequences, and the generation loop will terminate cleanly when the model says it is done. For Chapter 19 our training data always ends at exactly `T - 1`, so the model always emits EOS there, which is the same position the context window fills anyway. The early-stop code path is correct and fires in principle; the regime just does not exercise it yet.

This is the right order to do things, by the way. Build the EOS plumbing first, in a setting where it is easy to verify (fixed-length training, EOS always at T-1). Then turn on the variable-length data in Chapter 20 and watch the plumbing already work.

Four demonstrations

Demo 1 — the digit-mode pipeline is unchanged

```
./maxai --task=next_token --causal=1 --steps=800
```

This is exactly the Chapter-17 fixed-example demo. The loss curve, attention matrix, generation rollout, and final output are byte-for-byte identical to the earlier chapters' transcripts. The runtime refactor was designed to leave digit mode untouched; this demo proves it did.

Demo 2 — memorize the alphabet, recite it from `"a"`

```
./maxai --task=next_token --causal=1 --vocab=chars --steps=800
```

Fixed-input mode, character vocabulary. The single training example is `tokens[t] = t`, which under the char encoder maps to `"abcdefgh"`. The training target for next-token is `"bcdefgh<EOS>"`. Eight hundred steps is more than enough to memorize the mapping. The generation demo then prompts with `"a"` and watches the model recite:

```
 prompt  : "a"  (len=1)
step  0: context="a"       pick=b  (top3: b@1.00  c@0.00  e@0.00)
step  1: context="ab"      pick=c  (top3: c@1.00  e@0.00  b@0.00)
step  2: context="abc"     pick=d  (top3: d@1.00  e@0.00  f@0.00)
step  3: context="abcd"    pick=e  (top3: e@1.00  d@0.00  c@0.00)
step  4: context="abcde"   pick=f  (top3: f@1.00  <EOS>@0.00  d@0.00)
step  5: context="abcdef"  pick=g  (top3: g@1.00  b@0.00  c@0.00)
step  6: context="abcdefg" pick=h  (top3: h@1.00  b@0.00  <EOS>@0.00)
final : "abcdefgh"
```

This is structurally identical to Chapter 17's memorized digit demo — same training regime, same model, same confidence near 1.00 on every step. The only substantive difference is that the output now looks like letters. Notice also the top-3 at steps 4 and 6: `<EOS>` is showing up as a near-zero alternative, because the model has learned that EOS is a legal token at some position (specifically T-1). At position 4 the model assigns a tiny probability to "sequence might end here" — an honest reflection of the training distribution.

Demo 3 — the same hallucination pattern, now in English letters

```
./maxai --task=next_token --causal=1 --vocab=chars --steps=800 --gen_prompt="hel"
```

Chapter 17 Case 3 demonstrated that when a memorized model is prompted with something off-distribution, it falls back on positional priors and confidently hallucinates the memorized completion starting at whatever position the prompt filled. The same pattern repeats here in letters:

```
 prompt  : "hel"  (len=3)
step  0: context="hel"      pick=d  (top3: d@1.00  f@0.00  <EOS>@0.00)
step  1: context="held"     pick=e  (top3: e@1.00  d@0.00  c@0.00)
step  2: context="helde"    pick=f  (top3: f@1.00  <EOS>@0.00  d@0.00)
step  3: context="heldef"   pick=g  (top3: g@1.00  b@0.00  c@0.00)
step  4: context="heldefg"  pick=h  (top3: h@1.00  <EOS>@0.00  b@0.00)
final : "heldefgh"
```

The model has never seen `"hel"` in training. Its priors, learned from `"abcdefgh"`, say "position 3's next token is `d`, position 4's is `e`, position 5's is `f`, position 6's is `g`, position 7's is `h`." Those priors override the prompt completely after step 0 and the generation is the memorized suffix starting from position 3. The distribution at step 0 is still near-1.00 on `d` because the model's position embeddings are *very* confident about this single training example — with more training diversity the top-3 at step 0 would spread out. But the structural pattern is unmistakable: *when in-context information is weak and positional priors are strong, priors win*. The Chapter-17 hallucination lesson restated in a setting where the output reads like a human typing

word fragments rather than digit puzzles, which is cosmetically closer to how LLM hallucination actually looks in production.

Demo 4 — the char-mode ceiling with EOS supervision

```
./maxai --task=next_token --causal=1 --vocab=chars --random=1 --batch=16 --steps=5000
```

Uniform-random character sequences, with EOS supervision at the final position. We can compute the ceiling exactly and compare.

- Positions 0 through T-2 (seven of the eight): the target at each is `tokens[t+1]`, drawn uniformly from `SAMPLEABLE_VOCAB = 27` (letters + space). Best possible per-position loss: `log(27) ≈ 3.296`. Best possible per-position accuracy: `1/27 ≈ 3.7%`.
- Position T-1 (one of eight): the target is EOS, deterministically. Best possible per-position loss: `0` (put all mass on the one correct class). Best possible per-position accuracy: `100%`.
- Average loss: `(7 × 3.296 + 1 × 0) / 8 ≈ 2.884`.
- Average accuracy: `(7 × 3.7% + 1 × 100%) / 8 ≈ 15.7%`.

The program reports loss `2.888` and held-out accuracy `15.4%`. The ceilings match the derivation to within training noise — the tiny gaps reflect the finite eval set (52 points × 8 positions) rather than real model imperfection.

Two things worth noticing in the rollout for this demo:

```
 prompt  : "taa"  (len=3)
step  0: context="taa"     pick=i  (top3: i@0.04  a@0.04  r@0.04)
step  1: context="taai"    pick=o  (top3: o@0.04  d@0.04  v@0.04)
...
```

First, the top-3 probabilities hover around `0.04` (= `1/27`), not `0.10`, because the model has learned the marginal distribution over letters+space but puts negligible probability on EOS (which never occurred in mid-sequence positions during training). Second, the generated string `"taaiomzf"` is gibberish — which is the correct thing to emit from a model that has learned "I know absolutely nothing about what comes next, but I know it is a letter." This is the Chapter 16 ceiling argument made visible in text: *a perfectly trained model on zero-information data generates perfect gibberish.* Chapter 20 will replace the random training data with actual English, at which point this same model will start producing output that looks like English, because the source distribution will have teeth.

What is built and what is not

After Chapter 19, MaxAI:

- has a runtime-selectable vocabulary of 10 or 28 symbols,
- accepts character-string prompts like `"hello "` in char mode,

- tokenizes via a trivial character-level encoder,
- supervises a real EOS token at the final position,
- provides an EOS-emission early-stop in the generation loop,
- and preserves every Chapter 16–18 demo exactly in digit mode.

What it still does not do:

- It is not trained on meaningful text (Chapter 20 fixes this by sliding a window across a canonical English paragraph).
- Its tokenizer is single-character (Chapter 21 replaces it with BPE).
- Generation is still $O(N^2)$ per rollout (Chapter 22 adds a KV cache).

Each of the three remaining chapters slots into the infrastructure built here without rewriting any of the core. That is the payoff for doing the refactor cleanly.

> **Read along:** Chapter 19's code delta touches more parts of `maxai.cpp` than earlier chapters because it is a runtime refactor plus a feature addition:
>
> - **Part 1** — `VOCAB` ceased to be a constexpr; three new runtime globals (`MAX_VOCAB` , `VOCAB` , `SAMPLEABLE_VOCAB` , `EOS_TOKEN`) took its place. Every static array is now sized with `MAX_VOCAB` ; every loop bound uses `VOCAB` .
> - **Part 3** — `enum class Vocab { Digits, Chars }` and the `cfg.vocab` field; `parse_vocab()` recogniser; usage help gains a five-line `--vocab=NAME` block; `parse_args()` now sets `VOCAB / SAMPLEABLE_VOCAB / EOS_TOKEN` from the chosen `cfg.vocab` and bounds-checks against `MAX_VOCAB` .
> - **Part 3b** (new) — `encode_char` , `decode_token_char` , `print_token` , `print_tokens` . Eight small functions that do the entire tokenization and display work.
> - **Part 12** (`make_target`) — the `Task::NextToken` case now emits `EOS_TOKEN` at position `T-1` when the vocabulary has one. `effective_seq_len()` respects the distinction.
> - **Part 13** (data) — `build_heldout()` and `sample_example()` draw tokens from `SAMPLEABLE_VOCAB` rather than `VOCAB` , keeping EOS out of random training noise.
> - **Part 18** (demo) — vocab line in the header; input/target/output rows print through `print_tokens()` .
> - **Part 18b** (generate) — verbose header shows the vocabulary; the inner loop uses `print_tokens` / `print_token` for every output; the post-sample stop condition checks `EOS_TOKEN` and breaks before appending.
> - **Part 18b** (`parse_prompt`) — renamed from `parse_prompt_digits` ; now handles both vocabularies.
> - **main()** — boot banner now includes `vocab=<name>(<size>)` .
>
> Despite the number of sites touched, no piece of logic was rewritten — everything is still the Ch 16 loss, the Ch 9 backprop, the Ch 11 SGD. Only the shapes, sizes, and displays changed. `diff minai.cpp maxai.cpp` now hovers around 900 lines, of which the vast majority is Chapter 19's commentary and display plumbing; the actual behavioural delta is 30–40 lines of logic.

Chapter 20 — Train on real text

What this chapter does

This is the chapter the whole book has been building toward. Every mechanism in Chapters 1 through 19 is now in place: embeddings, multi-head-free attention, residuals, LayerNorm, cross-entropy, sampled autoregressive generation, a character-level vocabulary with EOS. Chapter 20 does exactly one thing: it replaces the training data from "uniform random noise" to "a paragraph of actual English," and discovers that everything already built suddenly starts doing something useful. The loss floor — the `log(VOCAB)` information-theoretic ceiling derived in Chapter 16 — collapses, because English is not random, and the model's job changes from "learn a ceiling you cannot exceed" to "learn as much structure as a corpus offers and fit under what the data permits."

Up to now the book has spent a great deal of time on why random training data tells you essentially nothing about whether a model "works" — the ceiling arguments in Chapters 14 and 16 established that an i.i.d. uniform source has zero exploitable conditional information, no matter how large you make the model. Chapter 20 is the point where that lesson pays off: you are about to watch the exact same model, trained on exactly the same number of steps with the same batch size and learning rate, suddenly drop its loss by a factor of twenty. Nothing changed except the data distribution.

The data pipeline change

Chapters 16 through 19 used two data regimes: the single fixed example (memorization) and i.i.d. uniform random sequences (the zero-information regime). Chapter 20 introduces a third regime — the **sliding-window sampler over a fixed text corpus**:

```
 pick a random start position s in [0, len(corpus) - T - 1]
input  = corpus[s..s+T-1]           # T characters
target = corpus[s+1..s+T]           # the same window shifted by one char
```

Each training step picks a fresh random window. With a 165-character corpus and `T=16`, there are about 150 unique windows; at `batch=16` steps each sees 16 of them. Over thousands of steps the model sees every window many times. This is exactly the sliding-window setup real LLMs use on infinitely larger corpora — for a 10-trillion-token dataset, there are roughly `10^13` possible unique windows, and training simply samples among them.

The only structural difference between this and GPT-4's actual training data pipeline is quantity. Our corpus fits in a string literal; theirs needs a petabyte of sharded storage and distributed readers. Our sampler is five lines of `xorshift32`; theirs is a production data loader with shuffling, caching, and fault tolerance. The *rule* — pick a random window, supervise next-token prediction over it — is the same.

Fixing up the target at position T-1

Chapter 16 picked “mask the final position” for digit mode, and Chapter 19 picked “train EOS at T-1” for char mode. Corpus training breaks both choices, because for a sliding window embedded in a longer stream, position `T−1` *has* a real next character — namely `corpus[s+T]` — and scoring against either a masked placeholder or a fake EOS wastes or actively corrupts the gradient signal there.

The first time I wrote Chapter 20 I left the EOS-at-T-1 rule in place and the loss immediately *exploded* during training: the model was being told “after this random 24-character window from the middle of an English paragraph, emit EOS” at every window, when EOS was in fact never present in the corpus. The learning task at T-1 was impossible, the gradients there were inconsistent, and they contaminated the useful learning at positions 0..T-2. Loss climbed above `log(VOCAB)` instead of falling below it.

The fix is that `sample_from_corpus()` explicitly overrides the `NextToken` target at position `T−1` with the real next character from the corpus, bypassing `make_target()`’s EOS substitution:

```
if (cfg.task == Task::NextToken) {
    for (int t = 0; t < T; ++t) {
        out_target[t] = encode_char(corpus[start + t + 1]);
    }
}
```

Now every position has an honest target. `effective_seq_len()` returns the full `T` (nothing to mask), gradients flow cleanly through every step, and loss falls fast. This is the right general principle: *the target at position T-1 should be whatever actually comes next when T-1 is inside a stream of more text*. EOS is only correct when `T−1` is really the end — which is the Chapter 19 fixed-length regime, not the Chapter 20 sliding-window regime.

Real LLMs handle this explicitly. When training data contains `<|endoftext|>` markers at document boundaries, those markers are part of the normal next-token target — a position just before `<|endoftext|>` is supervised to predict `<|endoftext|>`, and a position right after is supervised to predict the first token of the next document. The same three lines of `sample_from_corpus()` would implement this for us; we do not need to, because our corpora are small and we are treating them as single documents.

The two corpora shipped

MaxAI bundles two small English corpora, selected by `--corpus=<name>`:

`--corpus=seashells` is the classic “she sells sea shells” tongue-twister, ~165 characters, extremely repetitive. It is designed to be the easiest possible real-text training target: the bigram `"se"` → `"a"` (from “sea”) and `"sh"` → `"e"` (from “shells”) each appear many times, and the model can drive its per-position loss close to zero after only a few thousand steps. Great for watching the ceiling fall dramatically.

`--corpus=gettysburg` is the opening of Lincoln’s Gettysburg Address, lowercased, punctuation removed, ~210 characters. Formal English, more diverse vocabulary, more irregular structure. Harder to fit, more

interesting to read the rollout.

Both are hard-coded string constants in `maxai.cpp` Part 13b. Adding a new corpus is three lines of code (a new enum value, a new string constant, a new case in `get_corpus_text()`); nothing else in the program needs to change.

Demo 1 — watching the ceiling break

```
./maxai --task=next_token --causal=1 --vocab=chars \
        --corpus=seashells --seq_len=16 --batch=16 --steps=5000 \
        --blocks=2 --layernorm=1
```

Two-block model, LayerNorm enabled (for stability at depth), 5000 steps over random 16-character windows of the seashell corpus.

Loss curve summary the program prints:

```
Loss curve (5000 steps):  3.545  ...  0.224
```

The initial loss of `3.545` matches `log(VOCAB) = log(28) ≈ 3.332` to within random-init noise — the model's first predictions are uniform over the 28-symbol output. The final loss of `0.224` is *an order of magnitude below* that ceiling. In bits-per-character (dividing nats by `log(2) ≈ 0.693`):

- **Initial**: `3.545 / log(2) ≈ 5.12 bits/char` — "no information about what comes next."
- **Final**: `0.224 / log(2) ≈ 0.32 bits/char` — "on average, 0.32 bits of uncertainty per character."

The model has compressed this corpus to 0.32 bits per character. For comparison: Shannon's 1951 experimental estimate of the per-character entropy of English prose is about 1 to 1.5 bits; GPT-4-scale models achieve roughly 0.8–1.2 bits on held-out English. Our 0.32 bits/char is *lower* than the true entropy of English — which is a sign that we have essentially memorized this particular 165-character corpus rather than learned English. Of course we have: 165 characters is not enough to learn English. The point is the *shape* of what happened: loss fell where in Chapters 16 and 19 no amount of training could have moved it, because the data now had structure to fit.

Demo 2 — the model recites the corpus

```
./maxai --task=next_token --causal=1 --vocab=chars \
        --corpus=gettysburg --seq_len=24 --batch=16 --steps=8000 \
        --blocks=2 --layernorm=1
```

Same architecture, Gettysburg corpus, 8000 steps. Final loss `~0.14`. The generation demo uses the first 8 corpus characters as its default prompt (the new Ch-20 behaviour of `demo_generate()`) and watches the model continue:

```
 prompt  : "four sco"  (len=8)
step  0: context="four sco"       pick=r  (top3: r@1.00  n@0.00  s@0.00)
step  1: context="four scor"      pick=e  (top3: e@1.00  i@0.00  d@0.00)
step  2: context="four score"     pick=   (top3:  @1.00  n@0.00  i@0.00)
step  3: context="four score "    pick=a  (top3: a@0.99  e@0.01  c@0.00)
step  4: context="four score a"   pick=n  (top3: n@1.00  t@0.00  s@0.00)
step  5: context="four score an"  pick=d  (top3: d@0.99  n@0.00  v@0.00)
step  6: context="four score and" pick=   (top3:  @1.00  i@0.00  e@0.00)
step  7: context="four score and "pick=s  (top3: s@1.00  d@0.00  w@0.00)
step  8: context="four score and s" pick=e (top3: e@1.00  c@0.00   @0.00)
...
final : "four score and seven yea"
```

Here is a small neural network, with roughly five thousand parameters, trained for eight thousand steps on a paragraph, reciting the first sentence of one of the most famous speeches in American history. It is not *thinking*. It has memorized the character-level statistics of a 210-character string and is walking them out. But the mechanism — transformer blocks, causal attention, next-token sampling — is identical to what you would run if you wanted to recite a trillion tokens' worth of prose with billions of parameters instead.

Demo 3 — sampling diversity at partial training

```
./maxai --task=next_token --causal=1 --vocab=chars --corpus=seashells \
       --seq_len=16 --batch=16 --steps=500 --blocks=2 --layernorm=1 \
       --gen_prompt="she " --temperature=1.0
```

Only 500 training steps. The model has learned the *frequent* patterns of the corpus but not yet driven every distribution to one-hot confidence. This is the regime where sampling actually *matters*. Rollout from `"she "` with the default seed:

```
 step  0: context="she "           pick=s  (top3: s@0.73  i@0.15  t@0.05)
...
step  6: context="she sells "     pick=a  (top3: s@0.52  a@0.35  b@0.05)
step  7: context="she sells a"    pick=r  (top3: r@0.99   @0.01  n@0.00)
step  8: context="she sells ar"   pick=e  (top3: e@1.00  h@0.00  o@0.00)
```

Notice step 6: after `"she sells "` the model is genuinely uncertain between `'s'` (leading to `"she sells sea"`, which appears in the corpus many times) and `'a'` (leading to `"she sells are"`, which also appears — in "the shells she sells *are* sea shells"). Both are legitimate continuations. The sampler at `T=1.0` draws one with probability proportional to its mass; here it drew `'a'`, so the model rolled out to `"she sells are"`.

Re-run with a different `--sampling_seed=0xFEEDBEEF` and step 6 flips to `'s'`, yielding `"she sells se"` instead. **Same model, same prompt, different seed, different English continuation.** The whole point of Chapter 18's sampling machinery is suddenly visible and useful: in a well-trained model on real data, sampling

at moderate temperature explores the *actual branching structure of the data distribution*, producing varied but still plausible outputs rather than the deterministic greedy walk that argmax would give.

This is also where the Chapter-18 temperature story stops feeling abstract. At 500 steps the logits are not yet extreme, so `T=0.7` and `T=1.2` both produce meaningfully different distributions and meaningfully different rollouts. The full creativity-vs-fidelity trade-off every production LLM API exposes is doing its job here. By 5000 training steps the distributions are sharp enough that temperature stops biting (same pattern as Ch 18 Demo 1 on the memorized digit model), and sampling collapses back to greedy-like behaviour — which is also what happens in production if you over-train a model against a small dataset without regularization.

Demo 4 — hallucination, but in English this time

```
./maxai --task=next_token --causal=1 --vocab=chars --corpus=seashells \
       --seq_len=16 --batch=16 --steps=5000 --blocks=2 --layernorm=1 \
       --gen_prompt="xyz abc "
```

The trained seashells model has never seen the characters `'x'`, `'y'`, `'z'`, or `'b'`, `'c'`. The prompt `"xyz abc "` is wildly out of distribution. Greedy decoding from it:

```
 prompt  : "xyz abc "
step  0: context="xyz abc "   pick=i  (top3: i@0.52  e@0.31  h@0.09)
step  1: context="xyz abc i"  pick=f  (top3: f@0.63  m@0.37  v@0.00)
step  2: context="xyz abc if" pick=   (top3:  @0.96  h@0.03  r@0.01)
...
final : "xyz abc if lls s"
```

Step 0's distribution is genuinely uncertain (`i@0.52, e@0.31, h@0.09`) because "the characters the model has seen after a space" is a handful of plausibles — `i` starts "im", `e` is common, `h` leads to "he". It picks `i`, then `f` (now "if" is another recognizable bigram from "if she sells"), and then plows ahead committing to in-corpus bigrams assembled in completely non-corpus order: `"if lls s"` is a garbled mixture of `"if"` and `"lls"` (from "sells"/"shells"). The output is recognizably English-letter-soup rather than pure gibberish, but it is not a continuation of `"xyz abc "` in any meaningful sense.

This is exactly the Chapter 17 Case 3 hallucination pattern. When context is off-distribution, the model's learned priors — which are strong and locally coherent — take over, and the output drifts into whatever its training has made it confident about. Real LLMs exhibit this at orders of magnitude more depth: a user asks a specific factual question the model has no trained information about, and the model generates a plausible-sounding, confident, and completely fabricated answer assembled out of in-distribution fragments. The mechanism visible in these eight characters is the mechanism visible when a 175-billion-parameter model makes up a non-existent scientific paper. Different scale; same failure mode.

What the model learned: an information-theoretic view

Chapter 16 framed training as the process of minimizing cross-entropy against a source, and the cross-entropy floor is the source entropy. In Chapter 20 we finally have a source with non-trivial entropy and can actually compute the compression:

- `VOCAB = 28` → maximum cross-entropy = `log(28) ≈ 3.33 nats = 4.81 bits` per character.
- Uniform random chars: trained cross-entropy ≈ `log(27) ≈ 3.30 nats = 4.76 bits` per character. No compression.
- Seashells (trained): `~0.22 nats = 0.32 bits` per character. A `15×` reduction.
- Gettysburg (trained): `~0.14 nats = 0.20 bits` per character. A `24×` reduction.

That ratio — nats of loss divided by `log(VOCAB)` — is the model's measured "knowledge" about the source. It is a dimensionless compression factor, and it scales lawfully with model size, training data size, and training compute. This is the famous **neural scaling law** (Kaplan et al. 2020, Hoffmann et al. 2022): across many orders of magnitude, doubling model parameters or doubling training data lowers cross-entropy by a predictable amount, and those reductions compose cleanly. The entire field of "scale laws" in LLMs — "what happens when we spend another billion dollars on compute?" — is about predicting where the next doubling of each quantity will land on this curve.

Our corpus is too small for any generalization story to hold — the model has memorized far more than it has learned. But the *measurement* works the same as it does at any scale. When someone reports a frontier-model cross-entropy on the Pile in nats, you can convert it to bits/token and compare it to the 0.32 bits/char we just produced and see the relationship directly: their number will be higher (the data is harder), but the units are the same and the meaning is the same. *Training a language model is lowering cross-entropy against a data stream.* Everything else is in service of making that procedure run at a particular scale.

The scale gap, in numbers

	MaxAI (this chapter)	GPT-4 (estimated)
Parameters	~5,000	~10^{12} (a trillion)
Training tokens	~10^{3} (characters)	~10^{13} (multi-trillion)
Vocabulary	28 character-level	~10^{5} BPE subwords
Context window	16 characters	~10^{5} tokens
Training steps	~10^{4}	~10^{6}
Training compute (FMAs)	~10^{9}	~10^{25}

The ratios differ across rows — parameters scale ~2×10^8, data scales ~10^{10}, compute scales ~10^{16} — but the underlying procedure is the one you are running locally. Gradient descent on cross-entropy loss, sampled autoregressive inference, causal attention, LayerNorm-stabilized residual stacks. Every mechanism in `maxai.cpp` is also a mechanism in GPT-4. You do not need to take anyone's word on "GPT is a transformer trained on next-token prediction"; you can run the transformer trained on next-token prediction, watch it recite the Gettysburg Address, and see exactly how the machinery produces the behaviour.

What is built and what is not

After Chapter 20, MaxAI is a complete, if minuscule, character-level language model. It:

- trains on arbitrary English text (via `--corpus=<name>`),
- reaches cross-entropies well below `log(VOCAB)`,
- samples with temperature / top-k / top-p,
- terminates on EOS when it has been supervised to,
- prints real letters instead of digits,
- and exhibits the same qualitative failure modes (hallucination on OOD prompts, greedy-mode repetition, memorization of small corpora) as real LLMs.

What it does not do yet:

- **The tokenizer is character-level.** For English this is wasteful — a word like `"proposition"` is 11 tokens when BPE would make it 2 or 3. Chapter 21 adds a tiny byte-pair encoder so MaxAI's vocabulary can be subword rather than character, matching how every real LLM tokenizes.
- **Inference is still $O(N^2)$.** Every generated token re-runs the full forward pass over the prefix. Chapter 22 adds the KV cache that fixes this, with the same mechanism every production LLM serving stack uses.

After Chapters 21 and 22, the only remaining difference between MaxAI and a real LLM is scale. Not structure. Not algorithm. Not mechanism. Just how many zeros you put in each of the numbers in the table above.

> **Read along:** Chapter 20's code delta lives in three places:
>
> - **Part 3** — `enum class Corpus` and `cfg.corpus` field; `parse_corpus`, `corpus_name`; usage help for `--corpus=`; a parse_args guard that `--corpus` requires `--vocab=chars`.
> - **Part 13b** (new) — two `static const char[]` constants for the shipped corpora; `get_corpus_text()`; `sample_from_corpus()` (the sliding-window sampler, including the override of make_target's EOS rule for NextToken).
> - **Part 13** (`build_heldout`, `sample_example`) — corpus-aware branches that call `sample_from_corpus` when a corpus is active.
> - **Part 18** (`demo`) — samples an in-distribution window when a corpus is active, so the input/target/output row shows something meaningful.
> - **Part 18b** (`demo_generate`) — default prompt is now the first 8 characters of the corpus when one is set, so the generation rollout starts in-distribution.
> - **main()** — boot banner prints the corpus text and length when one is set.

> `diff minai.cpp maxai.cpp` crosses a thousand lines total for the first time; the behavioural delta is around 50 lines of logic, the remainder is tutorial commentary and the two hard-coded corpus strings.

Chapter 21 — A tiny BPE tokenizer

What this chapter does

Chapter 19's character-level tokenizer is structurally correct — input strings tokenize, tokens detokenize, the whole pipeline works — but it is wildly wasteful at any realistic scale. Every English word like `"proposition"` becomes eleven tokens, each carrying exactly one character's worth of information. A context window of 32 tokens covers 32 characters, roughly five words, which is not enough to fit a grammatical sentence. Real LLMs use *subword tokenization*: a word like `"proposition"` becomes two or three tokens, and a 32-token context window fits a paragraph. The mechanism that every real LLM uses for this is **Byte-Pair Encoding (BPE)**, and Chapter 21 is a tiny, real BPE trainer in about eighty lines of C++ inside `maxai.cpp`.

After this chapter, the MaxAI tokenizer is no longer trivial. It is a learned artifact — built from the training corpus at startup, baked into the model's vocabulary, and usable for encoding user prompts and decoding generated tokens. Every piece of the tokenization story told in Chapter 15 of Section 1 (from a distance) is now a piece you can run, inspect, and modify.

The algorithm, in seven lines of English

BPE was invented for file compression in 1994 (Gage) and re-purposed for NLP in 2016 (Sennrich, Haddow, Birch). The training procedure:

```
 start with every character as its own token
repeat K times:
    count every adjacent pair of tokens in the training corpus
    find the most frequent pair (a, b)
    add a new token [ab] to the vocabulary
    replace every occurrence of (a, b) in the corpus with [ab]
```

That is it. With `K = 32` merges on our 212-character Gettysburg corpus, you get a 60-token vocabulary; with `K = 50,000` merges on a 500-billion-word corpus you get GPT-3's tokenizer. Same algorithm. Different scale. Production BPEs add a few engineering touches — pre-splitting text on whitespace so merges do not cross word boundaries, using hash maps for efficient pair counting, and capping the maximum token length — but the core loop is exactly seven lines.

Why it works. Common character sequences in a language *repeat*. In English the most common adjacent pair is typically `('t', 'h')`, then `('h', 'e')`, then the three-character blend `'the'`. In a corpus with many

`"the"`s, the first merge crunches `('t', 'h')` into `[th]`. The second merge now operates on tokens that include `[th]` among them; the most-frequent pair is now `([th], 'e')`, which merges into `[the]`. One more pair-count and you discover `(' ', [the])` is frequent; merge into `[ the]`. The merges *compose*: later merges build on earlier ones, and the vocabulary gradually grows from individual characters into whole words and phrases. The algorithm is greedy and deterministic, and the vocabulary it produces is specific to the corpus it trained on — Gettysburg's BPE is different from Shakespeare's BPE is different from code.org's BPE.

The merges from our two corpora

MaxAI's `bpe_dump()` prints the learned merges at startup so you can see exactly what the tokenizer discovered. For `--corpus=seashells --bpe=16` it reports:

```
 merge  0: [ ] + [s]   -> [ s]
merge  1: [h] + [e]   -> [he]
merge  2: [ s] + [e]  -> [ se]
merge  3: [l] + [l]   -> [ll]
merge  4: [ll] + [s]  -> [lls]
merge  5: [ s] + [he] -> [ she]
merge  6: [r] + [e]   -> [re]
merge  7: [ se] + [a] -> [ sea]
merge  8: [ she] + [lls] -> [ shells]
merge  9: [t] + [he]  -> [the]
merge 10: [ ] + [the] -> [ the]
merge 11: [ se] + [lls] -> [ sells]
merge 12: [h] + [o]   -> [ho]
merge 13: [ ] + [i]   -> [ i]
merge 14: [ s] + [ho] -> [ sho]
merge 15: [ she] + [ sells] -> [ she sells]
```

Read the merges in order and you can watch the tokenizer *discover English structure*. First it glues `he`, `re`, `ll` — frequent bigrams. Then it discovers word stems: `she`, `sea`, `the`. Then whole words: `shells`, `sells`. Then — in merge 15 — a two-word phrase: `she sells`, a single token that encodes ten characters of text. With 16 merges the 165-character corpus compresses to **51 tokens**, a 3.2× reduction in sequence length. Every BPE merge you can show a child ends up being the same kind of discovery real production BPEs make: the tokens are *whatever your data repeats*, and nothing else.

For Gettysburg with `--bpe=32` the pattern is subtly different — the text is less repetitive, so merges lean on morphemes rather than whole phrases:

```
 merge  4: [r] + [e]    -> [re]
merge  5: [e] + [d ]   -> [ed ]       # past-tense suffix with trailing space
merge  6: [t] + [h]    -> [th]
merge 15: [i] + [v]    -> [iv]
merge 16: [i] + [on ]  -> [ion ]      # noun-forming suffix
merge 23: [ a] + [ ]   -> [ a ]       # the word "a" flanked by spaces
```

```
  merge 25: [ a] + [nd ] -> [ and ]     # the word "and" (leading + trailing space)
  merge 26: [at] + [ed ] -> [ated ]     # past-participle suffix
  merge 28: [ou] + [r ]  -> [our ]
```

Tokens like `[ated ]` and `[ion ]` are exactly the morphemes a linguist would identify as English's productive past-participle and noun-forming endings. BPE did not know any grammar; it counted pairs. Yet the learned vocabulary lines up with linguistic structure surprisingly well, because morphemes are frequent by definition. This is a tiny version of the "BPE learns linguistically meaningful units" observation that has held for every corpus anyone has trained BPE on since 2016.

Why this compresses training

Chapter 20's sliding-window sampler drew a `seq_len`-long window of *characters*. Chapter 21's version draws a `seq_len`-long window of *tokens*, where each token may cover several characters. For `--corpus=seashells --bpe=16 --seq_len=8`, the 8-token window covers an *average* of `165 / 51 ≈ 3.2` characters per token, so a training example now sees around 25 characters of context instead of 8. The attention pattern still only attends to 8 positions, but each position carries significantly more information than it did at char level.

This is, in miniature, exactly why production LLMs are subword-trained. With `seq_len = 32000` tokens and a BPE averaging around 3.5 characters per token, real LLMs cover roughly 110,000 characters of text per context — ~20,000 words. Done at the character level, the same attention compute would cover one-fifth of that, and the model's per-step "field of view" would be correspondingly narrower.

Implementation in C++

The BPE implementation lives in `maxai.cpp` Part 13c. In broad strokes:

```
// Storage (declared in Part 1 so the char-printer in Part 3b can see it).
char bpe_token_text[MAX_VOCAB][MAX_TOKEN_TEXT];   // token id → string
int  bpe_token_len [MAX_VOCAB];                    // string length
int  bpe_merge_left [MAX_VOCAB];                   // (left, right) for merged
int  bpe_merge_right[MAX_VOCAB];                   //   tokens, -1 otherwise
int  encoded_corpus[MAX_CORPUS_TOKENS];            // corpus after BPE
int  encoded_corpus_len;

void bpe_init_base();        // populate token texts for 'a'..'z', space, EOS
void train_bpe();            // run the K-merge loop, update VOCAB and encoded_corpus
int  bpe_encode_text(...);   // encode a prompt string through the learned BPE
void bpe_dump();             // pretty-print the learned merges at startup
```

`train_bpe()` is the interesting part. In pseudocode:

```
tokenize the corpus at char level into a work buffer
for k in 0..K:
    count every adjacent pair (l, r) in the work buffer     # O(VOCAB^2) table
    pick the most frequent pair (best_l, best_r)             # O(VOCAB^2)
    if its count is < 2, stop (no repeating pairs left)
    record the new merged token: id 28 + k, text = concat(best_l.text, best_r.text)
    replace every (best_l, best_r) pair in the work buffer with the new id
```

The pair counter is a static `int pair_count[MAX_VOCAB][MAX_VOCAB]` — at `MAX_VOCAB = 128` that is 65 KB, allocated once at startup, zeroed each merge iteration. Production BPE uses an incremental hash map because the corpus is too large to rescan after every merge, but for our 200-character corpora the full rescan per merge is both simplest and fast enough.

`bpe_encode_text()` runs at inference time: given a user prompt string like `"she sells "`, it tokenizes the string at char level, then replays the K merges in order, greedily replacing `(left, right)` pairs wherever they appear. Five loops' worth of arithmetic; zero heap allocations.

Under a hundred lines of C++ in total. That is genuinely all BPE is, stripped of the production engineering that makes it fast at web scale.

Sampling a window is now simple

Remember the sliding-window sampler from Chapter 20? It walked a character string, mapped each character through `encode_char()`, and built target sequences one char at a time — including a carefully-crafted override that prevented `make_target()` from stomping on position `T-1` with an EOS. That override is now unnecessary: the corpus is tokenized *once* at startup by `train_bpe()` into `encoded_corpus[]`, and sample_from_corpus just reads subsequences out of the array:

```
static void sample_from_corpus(int* out_tokens, int* out_target, uint32_t& rng) {
    const int T = cfg.seq_len;
    const int need = (cfg.task == Task::NextToken) ? (T + 1) : T;
    const int start = xorshift32(rng) % (encoded_corpus_len - need + 1);
    for (int t = 0; t < T; ++t) out_tokens[t] = encoded_corpus[start + t];
    if (cfg.task == Task::NextToken) {
        for (int t = 0; t < T; ++t) out_target[t] = encoded_corpus[start + t + 1];
    } else {
        make_target(out_tokens, out_target);
    }
}
```

Eight lines. No per-window character encoding, no special cases. Every training step costs one index into an integer array. This is also how production LLMs work: the training data is pre-tokenized offline to a giant int-tensor, and training just slices windows out of it.

Demonstration — training on seashells with BPE

```
./maxai --task=next_token --causal=1 --vocab=chars --corpus=seashells \
       --bpe=16 --seq_len=8 --batch=16 --steps=3000 \
       --blocks=2 --layernorm=1 --gen_prompt="she sells "
```

Startup prints the BPE merge table (shown above) and a boot banner noting `vocab=chars(44)` — 28 base symbols plus 16 learned subwords. Loss falls fast (~ `3.86` → `~0.37` in 3000 steps) despite the expanded output distribution, because the denser token stream gives each gradient step more useful signal. The generation rollout at the end:

```
 prompt  : "she sells "   (len=4)
step  0: context="she sells "       pick=b   (top3: b@1.00  o@0.00  a@0.00)
step  1: context="she sells b"      pick=y   (top3: y@1.00   she sells@0.00  u@0.00)
step  2: context="she sells by"     pick= the (top3:  the@1.00  ...)
step  3: context="she sells by the" pick= sea (top3:  sea@1.00  ...)
final : "she sells by the sea"
```

The prompt `"she sells "` tokenized to *four* tokens (not ten characters): `[ she sells][ ]` — well, depends on exactly which merges applied, but the point is the *prompt-length* metric dropped from ten to four. From that four-token context the model rolls out four more tokens — `b`, `y`, `the`, `sea` — and reconstructs the string `"she sells by the sea"`, which is a valid continuation found in the training corpus. *The model is generating text at a subword granularity, not a character granularity.* Some tokens cover single letters, some cover whole words with their leading space. This is exactly how real LLMs generate.

Compared to the character-level run in Chapter 20 (`--bpe=0`), the same number of steps produces:

- a higher-capacity output distribution (44 classes vs 28),
- a lower initial loss ceiling ratio (more structure to fit),
- similar final loss in absolute terms,
- but each "generated token" is on average 3× as informative as a char-level token.

At our scale the effect is cosmetic; at GPT scale it is the difference between "coherent document-length outputs" and "incoherent letter soup."

Echo-backs and limits

Chapter 15 of Section 1 described BPE in prose; Chapter 21 makes it concrete. You can now point at the `train_bpe()` function and say "this is what the sentence 'the tokenizer was trained on the Pile' means at the code level — just, on a corpus 10^9× larger." The core object (the merge list) is a handful of numbers: GPT-4's tokenizer is a few hundred thousand merges, searchable by any user who wants to see what subwords exist; Claude's is proprietary but structurally identical.

Limitations of this implementation. Four things production BPE does that our version does not:

1. **Pre-splitting.** Production BPEs first split the text on whitespace and punctuation, then learn merges *within* each pre-split chunk. This prevents weird cross-word merges like `[ent] + [a] -> [enta]` that span "continent a new". We merge freely, so our learned merges sometimes cross word boundaries (see `[ she] + [ sells] -> [ she sells]` in the seashells run). For a small toy corpus this is fine and often pedagogically interesting; for a 100-million-word corpus it would produce cosmetically ugly tokens.
2. **Efficient pair counting.** We re-count every adjacent pair after every merge, which is `O(N × VOCAB²)` per merge. Production BPE incrementally updates counts as merges fire, which is `O(N)` per merge. Our 200-char corpora finish in microseconds; a 500-GB corpus would not.
3. **Byte fallback.** Real BPEs, especially GPT-2-onward, operate on bytes rather than characters, so they can encode *any* input — UTF-8 emoji, non-English text, binary data, whatever — without tokenizer errors. Our version refuses anything outside `a–z ' '`, which is fine for pedagogy but would not serve a real product.
4. **Tokenizer sharing with the embedding table.** Production LLMs often *tie* the embedding matrix to the output projection matrix (the "weight-tied" trick from 2016) because the same token-id-to-vector map is needed at both ends. MaxAI does not tie, because the savings are proportional to the vocabulary size and at 44 tokens there are no savings. At GPT-4 scale, weight tying saves tens of gigabytes of memory.

All four are engineering, not algorithm. The algorithm is the seven-line loop above.

What is built and what is not

After Chapter 21, MaxAI:

- learns a BPE from any corpus at startup,
- displays the learned merges for transparency,
- pre-tokenizes the corpus once for fast window sampling,
- encodes user prompts through the learned BPE,
- decodes generated tokens as subword text fragments,
- preserves every Chapter 16-20 demo unchanged when `--bpe=0` (the default).

What remains, and is the subject of Chapter 22: **every generation step still re-runs the full forward pass over the entire prefix.** At our seq_len of 8 or 16 that is instant; at a real LLM's seq_len of tens of thousands it would be unusable. The fix is the KV cache — a small bookkeeping change with enormous production impact. That is the final chapter of this section, and after it MaxAI will have every piece of a modern LLM inference stack in miniature: training, tokenization, sampling, and cache-accelerated generation.

> **Read along:** Chapter 21's code delta touches three parts of `maxai.cpp`:
>
> - **Part 1** — `MAX_TOKEN_TEXT`, `MAX_CORPUS_TOKENS`, and the BPE storage arrays are declared here so Part 3b's `print_token()` can read `bpe_token_text[]` without a forward declaration.
> - **Part 3** — `cfg.bpe_merges` field, `--bpe=K` flag, help text, parse_args validation ensuring `--bpe` requires `--corpus != none` and `--vocab=chars`.

- **Part 3b** — `print_token()` now reads `bpe_token_text[tok]`, which handles both single-char base tokens and multi-character subwords. `parse_prompt()` in char mode calls `bpe_encode_text()` so user input tokenizes through the learned BPE.
- **Part 13c** (new) — `bpe_init_base()`, `bpe_encode_text()`, `train_bpe()`, `bpe_dump()`. About 180 lines including comments; the actual algorithm is under 100 lines.
- **Part 13** (`sample_from_corpus`) — simplified to read from the pre-tokenized `encoded_corpus[]` array. Still performs the next-token target override for NextToken.
- **`main()`** — after `parse_args()`, calls `bpe_init_base()` (always) and `train_bpe() + bpe_dump()` (when a corpus is set) before the first training step.

`diff minai.cpp maxai.cpp` now passes 1200 lines, and Chapter 21 alone adds about 250 lines of code and comments. The behavioural delta is the ~100-line BPE algorithm; the rest is commentary and the forward-declaration plumbing.

Chapter 22 — The KV cache

What this chapter does

Chapter 17's generation loop was honest about a flaw: `generate()` runs the full `forward()` pass over the entire sequence every time it produces a new token. For our toy `seq_len = 8` this is fine; for a production LLM at `seq_len = 128,000` it is catastrophic. The amount of arithmetic to generate `T` tokens naively is `O(T²)` in the sequence length, which at realistic scales is the difference between a usable product and something that takes several minutes per token. Chapter 22 adds the standard fix — a **KV cache** and an incremental forward pass — and closes the performance gap. The rest of the chapter walks through why the fix is correct, what it costs in memory, and why "the KV cache" has become the single most important object in modern LLM inference engineering.

After this chapter, MaxAI has every piece of a modern LLM inference stack in miniature: tokenization (Ch 21), autoregressive sampling (Ch 18), a generation loop (Ch 17), a KV-cached incremental forward (Ch 22), and an EOS-terminated stop condition (Ch 19). The remaining differences between MaxAI and a frontier model are pure scale, as the Chapter 13 comparison table and the Chapter 20 scaling tables make explicit.

The naive loop's redundancy

Return to the Chapter-17 pseudocode:

```
while current_len < T:
    forward(tokens)                        # re-runs on positions 0..T-1
    next = sample(probs[current_len - 1])
```

```
    tokens[current_len] = next
    current_len += 1
```

Each iteration calls `forward()` on the entire token buffer. At iteration `k`, `forward()` processes T positions — for each it recomputes the token embedding, every layer's Q/K/V projection, every layer's attention scores (which are `O(T)` per position), the softmax, the attention-weighted sum, the FFN, and the final output projection. But here is the catch: positions `0..current_len - 2` have not changed since the last `forward()` call. The inputs are the same. The weights are the same. The outputs are the same. Every iteration is redoing arithmetic whose result it already computed one iteration ago.

Specifically, the activation tensors at positions `0..current_len - 2` are *bit-identical* across calls. The token embeddings are identical (same token ids), the positional embeddings are identical (same positions), the Q/K/V projections are identical (identical inputs to identical weights), and so on through every layer. The only position whose activations depend on the freshly-appended token is position `current_len - 1` itself — because causal attention forbids any earlier position from looking at tokens past its own index.

Causal masking is the reason this optimization exists. Without causal attention, position 2's output could depend on a token inserted at position 7, and appending a new token would invalidate every prior position's activations. With causal attention, the activations at earlier positions are frozen the moment they are computed. We should be caching them.

The cache, and what needs to be in it

At every block `b`, attention at position `t` reads `K[b][u]` and `V[b][u]` for each `u ≤ t`. Those are the only values from earlier positions the forward pass consumes — everything else (the per-layer residual stream `X_block[b]`, the FFN intermediates, etc.) is read only at the position being computed. So the "cache" is literally just **the K and V tensors at every position, at every block**.

Crucially, MaxAI already has this storage. `K[b][t][d]` and `V[b][t][d]` are per-position arrays; the naive `forward()` happened to re-populate all `T` entries every call, but it didn't *have* to. Chapter 22's change is to add a function that *only* populates position `t_new` and reads the rest from the values already sitting in the arrays. No new data structure, no allocation, no eviction logic. The K/V cache is implicit.

This is not universal. Real LLM inference engines (vLLM, TensorRT-LLM, llama.cpp, ...) manage KV caches explicitly because the amount of memory is significant (tens of gigabytes at long contexts) and because serving many concurrent conversations requires paging/eviction/sharing. The *concept* is identical to MaxAI's: save per-position K and V, reuse across generation steps. The *engineering* is where all the complexity lives, and it is roughly half the total engineering budget of a serving stack.

The incremental forward

`forward_step(t_new, tokens)` in `maxai.cpp` Part 8b is what the cache reads. It is structurally a copy of `forward()` with every loop over positions replaced by a single-position computation. In cartoon form:

```
forward_step(t_new, tokens):
   X_block[0][t_new] = token_emb[tokens[t_new]] + pos_emb[t_new]
   for each block b:
       (Pre-LN at t_new only)
       compute Q[b][t_new], K[b][t_new], V[b][t_new]   # one position's worth of matmuls
       compute attn scores[t_new][u] for u in 0..t_new using CACHED K[b][u]
       softmax and attn weights
       attn_o[t_new] = Σ_u attn[t_new][u] * CACHED V[b][u]
       residual, (Pre-LN), FFN, residual  — all at t_new only
   (final LayerNorm at t_new)
   logits[t_new] = X_block[final][t_new] @ Wout
   probs[t_new] = softmax(logits[t_new])
```

The per-step cost is `O(D² + t_new · D + D · D_FF)` instead of the naive `O(T · D² + T² · D + T · D · D_FF)`. Summed across all `T − P` generation steps:

- Naive total: `O((T − P) · (T · D² + T² · D + T · D · D_FF))` ≈ `O(T³ · D)` for large `T`.
- Cached total: `O(T · D² + Σ_{t=0..T−1} (D² + t · D + D · D_FF))` ≈ `O(T² · D)` for large `T`.

The factor-of-`T` speedup that saves real LLM inference.

The benchmark

`maxai.cpp` includes `--bench_gen=1`, which runs the current generation setup twice — once with `--kv_cache=0` and once with `--kv_cache=1`, using the same sampling seed so both paths produce the same token sequence — and prints a timing comparison.

For a modest configuration:

```
./maxai --task=next_token --causal=1 --vocab=chars --corpus=seashells --bpe=16 \
        --seq_len=16 --batch=16 --steps=500 --blocks=2 --layernorm=1 \
        --gen_prompt="she " --bench_gen=1
```

produces:

```
KV-cache benchmark (Ch 22): same rollout, two inference paths.
 naive  forward-every-step :   179.92 µs (13 forward calls)
 KV cache (forward + step) :    28.46 µs (1 forward + 12 step calls)
 speedup                   :     6.32x
```

At the larger configuration used for Gettysburg (`seq_len=32`, 8 layers):

```
KV-cache benchmark (Ch 22): same rollout, two inference paths.
 naive  forward-every-step :  3069.42 µs (24 forward calls)
```

```
KV cache (forward + step) :   247.67 µs (1 forward + 23 step calls)
speedup                   :    12.39x
```

The speedup grows with both sequence length and depth, exactly matching the `O(T · blocks)` ratio the arithmetic predicts. At modern LLM configurations (128,000 tokens, 96+ layers), the naive path would be roughly six orders of magnitude slower than the cached one. Production serving absolutely requires the cache; there is no “we’ll optimize this later” path.

Bit-exact verification

Run the same prompt with and without the cache and compare the outputs:

```
 kv_cache=0  final : "she sells by the sea shore the shells she sells are"
kv_cache=1  final : "she sells by the sea shore the shells she sells are"
```

Identical, down to the last token. The cache is a pure optimization. It changes no result. Every test that passes without the cache must pass with it; this property is the first thing an LLM-serving test suite checks and the first regression a faulty KV-cache implementation introduces. We preserve it here by explicit construction: `forward_step()` computes exactly the values `forward()` would have computed at position `t_new`, reading from the same cached K/V that `forward()` would have written into the same arrays.

Memory: the cost of long context

The cache is fast but not free. It stores `K[b][t][d]` and `V[b][t][d]` for every block, every position, every dimension. At a byte level:

```
cache size = num_blocks × context_length × 2 (K and V) × D_MODEL × bytes_per_float
```

For MaxAI with `num_blocks=8`, `context_length=32`, `D_MODEL=16`, `float` = 4 bytes:

```
cache size = 8 × 32 × 2 × 16 × 4 = 32,768 bytes = 32 KB
```

Trivial. Contrast against GPT-3 with `num_blocks=96`, `context_length=2048`, `D_MODEL=12288`, `bfloat16` = 2 bytes per element:

```
cache size = 96 × 2048 × 2 × 12288 × 2 ≈ 9.6 GB per conversation
```

Nine and a half gigabytes of GPU memory for one 2048-token conversation’s KV cache. Modern LLMs at 128,000-token contexts need closer to **600 GB** of cache per conversation — which is impossibly far past any single GPU’s memory budget. This is the reason long-context LLMs became a memory story rather than a compute story (and why Chapter 12’s “memory wall” prose was not hyperbole). Multiple GPUs pool their HBM

via NVLink; the cache is sharded across devices; attention becomes a distributed operation; new batch-oriented inference systems (PagedAttention, vLLM's scheme, Ring Attention) all exist to manage this single resource.

A back-of-envelope: roughly 80% of the engineering effort poured into inference frameworks in 2023-2025 is KV-cache management — sharing caches between conversations that share prefixes, evicting old conversations when memory fills up, quantizing K and V to int8 or int4 to halve or quarter the footprint (see Section-1 Chapter 14 Step 14's KV tiering demo, which shows what quantized K/V looks like in miniature), and compiling specialized attention kernels that stream the cache past the compute units rather than allocating it all at once. If you understand that the cache is storing `2 × num_layers × context × hidden_dim` floats per live conversation, and that HBM bandwidth is finite, every one of those engineering decisions falls out of the arithmetic.

Why generation is memory-bandwidth-bound at scale

One more derivation that is worth having in your head, because it explains something surprising about real LLM performance.

For the **training** forward pass, each matrix multiply (the bulk of the work) is compute-bound: the multiplier loads a weight matrix once, multiplies it by many input vectors simultaneously (batch dimension), and runs at close to peak arithmetic throughput. For the **inference generation** step with KV cache, there is no batch-over-tokens parallelism — you are computing one position at a time — and the matmuls become *very* small. The weight matrices still need to be streamed from HBM into the compute units, but the amount of arithmetic done per byte loaded is much lower. This tips the bottleneck from "how fast can we multiply" (compute-bound) to "how fast can we move bytes" (memory-bandwidth-bound).

The practical upshot: **tokens-per-second during generation scales with HBM bandwidth, not with FLOPS**. Which is why NVIDIA's generational GPU improvements in 2024-2025 were much more about memory bandwidth than raw multiplication rate, and why AMD's competitive response focused on HBM capacity and speed. The KV cache is the reason tokens-per-second is a memory problem; the memory wall is the reason the KV cache is a serving problem.

Limits and caveats of this implementation

Three simplifications relative to production KV caches worth naming so no one comes away with false confidence:

1. **We do not support batched generation.** A production serving system runs many conversations concurrently; its KV cache is a tensor of shape `[batch, layers, context, heads, head_dim]`. MaxAI's is `[layers, context, head_dim]` for one implicit batch-of-one. Batched caches need eviction, padding, and attention kernels that handle variable per-conversation context lengths. We do not.

2. **We do not support cache invalidation.** If the user changes the prompt mid-conversation (say, retries with a different seed from the middle), the cache at later positions is stale and must be recomputed.

Our `forward_step()` assumes the entire prefix is fixed. Production inference engines handle prompt editing explicitly; MaxAI restarts from scratch.

3. **We do not quantize the cache.** At scale, storing K and V at bfloat16 costs twice as much as int8, and int8 is often good enough for generation quality. `llama.cpp` supports several cache quantization modes; our cache is plain `float`. The Section-1 Chapter 14 Step 14 demo shows what happens to model accuracy under K/V quantization, but it is not wired into our generate loop.

Four *algorithmic* pieces, on the other hand, are exactly right: the incremental recurrence is correct by construction; the position embedding at `t_new` uses the right row; attention at `t_new` sums over the right range of cached positions; the final LayerNorm and output projection use the right input row. Any of those going wrong would produce non-bit-exact output against the naive path — which is why `--bench_gen=1` verifies identical results before comparing timings.

The end of Section 2

After Chapter 22, MaxAI is the smallest possible end-to-end generative language model that still deserves the name. Training loop, tokenizer, sampler, generation loop, KV cache, EOS termination, loss masking, variable vocabulary, a real English corpus with room for more — every piece of the mechanism a reader would find described in papers about frontier-scale models is present here, in miniature, running locally, in a single C++ source file that still compiles under `-Wall -Wextra` without warnings.

The differences from Claude or GPT-4 are scale and engineering. **Scale**: a few thousand parameters vs a trillion; a 200-character corpus vs petabytes of text; a two-block model vs ninety-six; a 32-token context vs 128,000. **Engineering**: production training distributes across tens of thousands of GPUs, needs fault-tolerant checkpointing, uses an optimizer with running gradient statistics, applies weight decay, gradient clipping, mixed precision, and dozens of other small improvements that add up to the frontier. Every one of those additions is a layer on top of what you have read. None of them are new ideas relative to Sections 1 and 2; they are the engineering required to make the ideas run at planetary scale.

That was the premise at the top of the book. You did not need to take anyone's word that "GPT is a transformer trained on next-token prediction." You built the transformer. You trained it on next-token prediction. You watched it recite the Gettysburg Address, sample with temperature, hallucinate confidently on out-of-distribution prompts, and run 12× faster under a KV cache than without. The thing that serves a trillion queries a day to the world does the same arithmetic, with a thousand times more of everything.

> **Read along:** Chapter 22's code delta is concentrated in three places:
>
> - **Part 3** — `cfg.kv_cache` and `cfg.bench_gen` fields; `--kv_cache=0|1` and `--bench_gen=0|1` flags; usage help.
> - **Part 8b** (new) — `forward_block_step(int b, int t_new)` and `forward_step(int t_new, const int* tokens)`. Together they are about 110 lines of C++, structurally a mirror of `forward_block()` and `forward()` with every `for t in [0, T)` loop replaced by a fixed

> `t_new`. The attention score loop reads `K[b][u]` for `u ≤ t_new` from cached values — the one place where "cache" is visibly the word for what we are doing.
> - **Part 18b** (`generate`) — the loop now calls `forward()` once up front to populate probs at the prompt positions, then `forward_step()` (or `forward()` for naive mode) at the end of each iteration to refresh `probs[current_len - 1]` for the next sample.
> - **Part 18b** (`bench_gen`) — the benchmark harness. Runs `generate()` twice with different `cfg.kv_cache` settings and the same sampling seed, times each with `std::chrono::high_resolution_clock`, and prints a side-by-side with speedup ratio.
> - **`main()`** — if `cfg.bench_gen` is set, runs `bench_gen()` after the existing `demo_generate()` call.
>
> The net file size crosses 1300 lines of delta from `minai.cpp`. The actual algorithmic change is the ~110 lines of `forward_step()`; every other edit is plumbing for the CLI knob and the timing harness. As always, the mechanism is small; the commentary exists so a reader can read the mechanism once and understand every downstream product property that falls out of it.

Section 2 epilogue

Section 1 ended with "the curtain is down." Section 2 picked it back up and walked around behind it.

You have built a next-token prediction target, a greedy and sampled inference loop, a widened vocabulary with learned subword tokenization, training on real English text, and a KV-cached incremental inference path. You have measured the information-theoretic ceilings that bound every toy model, broken those ceilings when actual structure appears in the data, watched hallucination unfold in a single-block model, seen every mainstream decoding knob (temperature, top-k, top-p, seed) change the output of a model you trained yourself, and closed the inference loop with the single most important optimization in LLM serving. All of it in roughly 3,000 lines of C++ and 900 lines of prose — everything compiling cleanly, everything runnable on a laptop in seconds, and nothing new relative to what the research papers describe.

The purpose of this project has never been to be useful. `minai.cpp` and `maxai.cpp` together are not going to recite Hamlet; they will recite seashells. They are not going to translate Japanese; they will answer "what comes after `a`?" with `b`. The purpose is to demystify. Someone who has read both sections straight through, followed the code alongside, and run the demos knows — not as metaphor, not as analogy, but as literally-the-same-arithmetic — what GPT-4 does when it responds to a prompt. The size of the arithmetic differs by a factor of about a trillion. The arithmetic itself does not.

That was the whole promise, at the top of Section 1, and the whole promise at the top of Section 2. Everything between is earnings on that promise. If it helped, pass it along. If it was wrong, the source is in the same directory — go check, go correct.

End of Section 2.

Appendix A — The PDP-11 fixed-point story

`minai.cpp` descends from a program Damien Boureille wrote in PDP-11 assembly for a 1975 minicomputer. The PDP-11 has no floating-point unit in its base configuration. How do you train a neural network — full of multiplications and softmaxes — when you only have integer arithmetic?

Q8.8 fixed-point

Represent a real number `r` by storing `round(r * 256)` as a 16-bit integer. The low 8 bits now hold the fractional part. `1.0` is stored as `256`. `0.5` is stored as `128`. Add two Q8 numbers: just add the integers. Multiply two Q8 numbers: their integer product is in Q16 (both 8-bit fractions merged, 16 fractional bits total), so shift right by 8 to get back to Q8. One instruction. The PDP-11's `MUL` happens to put its 32-bit product in a register pair perfectly sized for this.

This is an **integer** encoding of real numbers. It has a range of about `[-128, 128)` and a precision of about 1/256. Good enough for the forward pass of a small network.

For gradients, which are smaller numbers, you use **Q15** (15 fractional bits instead of 8). Precision 1/32768. And for long-running accumulators (like weight update sums) you use Q16 in a 32-bit integer. The three formats are chosen so that pairs of them multiply cleanly with the PDP-11's instruction set.

Softmax without exp

The PDP-11 has no `exp` function. The Boureille code precomputes a table of 256 values: `exp(-k/8)` for `k = 0..255`, each stored as a Q8 integer. To compute softmax:

1. Find the max Q8 value.
2. For each `x_i`, compute `(max - x_i) >> 5`. This gives an integer in `[0, 255]` or beyond.
3. Look up `exp(-idx/8)` from the table (or return 0 if the index is too large).
4. Divide by the sum.

No floating-point required at any step. The `>> 5` shift is the magic: converting a Q8 difference to an integer table index equal to `8 * (max - x_i)` in real-value terms — which is the `k` for `exp(-k/8)`. This is the softmax that runs in the real PDP-11 assembly. You can run a demo of it at the end of the C++ program: see Part 15 of `minai.cpp`.

This is still the future

"Quantization" — representing weights and activations with fewer bits — is an active area of modern LLM research. Int8, int4, even int2 quantizations are used to shrink multi-billion-parameter models into formats that can run on a phone. The underlying trick is identical to Boureille's: store real numbers as scaled integers, do arithmetic in integers, convert back only when needed. The PDP-11 was doing it in 1975 because it had no

FPU. Apple's Neural Engine does it in 2025 because it's faster and uses less power. Same trick, fifty years apart.

Appendix B — Exercises

These are optional but deepen understanding.

1. **Count parameters.** Before running the program, calculate by hand the parameter count for `--blocks=3 --ffn=1`. Then run it and check. (Answer in the Part 1 comment of `minai.cpp`.)
2. **Break it.** Comment out the line that adds `pos_emb` in Part 8. Recompile. Retrain. What happens and why?
3. **Attention observation.** Run `./minai --blocks=2`. Look at the attention matrix of block 1. Which row has the highest peak on column 0? Why is this the "right" behavior for a reversal task?
4. **Causal failure.** Run `./minai --causal=1`. The accuracy goes to 8/8 anyway. Explain why, referring to what the fixed input lets the model do.
5. **Gradient by hand.** Take the three-variable function `L(a, b, c) = (a * b - c)^2`. Compute the three partial derivatives `∂L/∂a`, `∂L/∂b`, `∂L/∂c` by hand. Identify which of the "six rules" from Chapter 9 each one uses.
6. **Softmax translation invariance.** Pick any three numbers `x_0, x_1, x_2`. Compute `softmax` on them. Add 100 to all three. Compute again. Confirm they are identical up to rounding.
7. **Q8 sanity check.** In Q8.8, what integer represents `0.125`? What integer represents `-1.5`? What happens if you multiply the first by itself and shift right by 8 — do you get the Q8 representation of `0.015625`?
8. **Read a paper.** The original attention paper is "Attention Is All You Need" (Vaswani et al., 2017). With Chapters 1–13 in hand, you have the vocabulary to read it end to end. Try. (Skip the BLEU score part if you don't care; the rest is in your reach.)

Section 3 — Where the field is going, and where you fit in

Section 1 showed you how a language model works. Section 2 showed you how to grow it into something that reads real text. Section 3 is about why, having understood both, this knowledge is the starting point of a long career in a field that is not ending but reshaping itself, and why the best time to have finished a book like this one is right now.

The Section 3 chapters are less code-focused and more field-focused. They cover:

- Chapter 23 — Why AI does not end software engineering.
- Chapter 24 — Using AI as a tool, competently.
- Chapter 25 — Open problem: hardware, memory, efficiency.
- Chapter 26 — Open problem: safety, security, alignment.
- Chapter 27 — The research frontier, as of 2026.
- Chapter 28 — Your place in this.

Each is short; together they sketch the landscape a new engineer is about to step into. If you are a student wondering whether to abandon a computer-science degree because "AI will write all the software soon," this is the part of the book written for you. Read it. Then don't abandon your degree.

Chapter 23 — Why AI does not end software engineering

Every technology that has ever significantly boosted engineer productivity has been initially feared as an engineer-eliminator. The compiler in the 1950s was going to eliminate programmers; they would be replaced by "automatic coders." Fourth-generation languages in the 1980s were going to eliminate the need to program at all. The graphical IDE in the 1990s was going to eliminate the value of deep technical knowledge. The cloud in the 2010s was going to eliminate system administrators. The prediction is always wrong, and it is always wrong for the same reason: **the demand for software has no ceiling.** When you make engineers faster, the things that used to be too expensive to build become buildable. The market expands to absorb the productivity gain, and the total pool of people working in software grows rather than shrinks.

LLMs are the newest instance of the same pattern. They automate a specific set of tasks — generating boilerplate, translating across similar languages, summarizing documentation, doing first-pass code review, drafting unit tests — that used to cost engineer hours. What they do *not* automate:

- **Figuring out what to build in the first place.** The product question. The "is this even the right problem?" question.
- **Designing systems that survive contact with unexpected load, failure modes, adversaries, and data distributions.** Production-grade design.
- **Integrating new components into existing codebases without breaking invariants nobody wrote down.** Half of software engineering is invariants that live in people's heads.
- **Debugging subtle errors whose root causes require sustained concentration on specific facts.** LLMs can suggest hypotheses; they cannot sit with the problem for three days.
- **Operating software in production at scale, under SLAs, with real users depending on it.**
- **Deciding what trade-offs to make when two correctness goals are in tension.**
- **Maintaining institutional knowledge across years as people and requirements change.**
- **Security in depth.** Not generating vaguely-safe-looking code — actually thinking like an adversary.
- **Deployment, monitoring, incident response, and post-mortem culture.**
- **Working with non-engineers to figure out what is actually needed.**

Every one of those is a place where the engineer's context, judgment, and responsibility is load-bearing. LLMs produce plausible code; engineers produce correct systems. Confusing the two is the most common mistake of someone who has spent a week with Copilot and is ready to declare the profession dead.

What actually happens when a tool makes engineers faster. The spreadsheet did not eliminate accountants; it created financial analysts, FP&A teams, and "spreadsheet power users" as a career category. The IDE did not eliminate programmers; it made it possible for one engineer to ship what used to take a team. Git did not eliminate source-control operators; it enabled open-source collaboration at the scale that now powers the entire industry. Cloud did not eliminate sysadmins; it created SREs and DevOps engineers and platform teams. In each case the total number of people employed in software grew, because the productivity gain unlocked new categories of work that had previously been uneconomical.

The most likely outcome for LLMs is the same. The total pool of software engineers in 2032 will very likely be larger than it is in 2026, not smaller, because the surface of buildable software will have expanded. A five-year-experienced engineer who never learned to use AI assistants effectively will feel outpaced by peers who did; an engineer who learned early will ship more, explore more, and be trusted with more responsibility. Engineers who never learned at all will be rare, because every company and every university is integrating these tools into their workflows and curricula.

What changes is the floor and the ceiling. The floor rises: the bottom 20% of software tasks (typing standard boilerplate, translating between close-related languages, drafting unit tests for known patterns) become things an AI can do while the engineer spends their attention elsewhere. The ceiling rises faster: the top 20% of tasks, the ones where human judgment and responsibility are essential, get *more* valuable because engineers have more time to focus on them. Across the middle, what it means to "write software" shifts — less typing, more specifying; less syntax, more design; less searching, more verifying.

If you are reading this book, you have already done the thing that matters most. You have built mental models of how these tools work underneath. You know what a transformer computes, where it is confident and where it is guessing, what its failure modes look like, and where the "AI magic" ends and the engineering starts. That knowledge is what separates engineers who can use AI productively from engineers who are confused by it, and the gap will widen every year.

The fear that software engineering is ending is almost entirely felt by people who cannot yet tell those two groups apart. Do not become one of them. Do not abandon the degree. Do not pivot out of the field because a hype cycle scared you. The field is not contracting; it is being remade, and the people who understand both the old craft and the new tools are the ones who will architect what comes next.

Chapter 24 — Using AI as a tool, competently

The most valuable skill for an engineer working alongside LLMs is not "being good at prompting." It is being good enough at engineering to know what to accept and what to reject from the model's output. That skill is exactly what the book you just read trained.

Concretely, productive AI-assisted engineering has a shape. It looks like this:

1. **Describe the task clearly, in terms of inputs, outputs, and constraints.** You would do this anyway for any new piece of work. The AI benefits from the same precision a junior engineer would. “Write a function” is much worse than “write a function that, given a sorted array of integers and a target, returns the index of the target or -1 if not present, using binary search, with test cases for empty array, single-element array, target-not-present, and target-at-either-end.”

2. **Let the model produce a draft.** Accept that the first output may be half-right or subtly wrong.

3. **Read what it produced, critically.** This is the step that matters. Does the code handle the edge cases you described? Does it respect invariants in the surrounding code (which the model almost certainly cannot see)? Does it hallucinate an API that does not exist in your library? Does it use a security pattern that is obsolete or unsafe? Does it handle the empty-input and the single-element cases? Does it match the style of the surrounding codebase?

4. **Correct, clarify, or discard.** If the output needs a few fixes, fix them. If it needs clarification of the task, re-prompt. If it is fundamentally off-track, discard and try a different approach. Never accept-and-commit without reading.

5. **Test.** Run the code. Exercise the edge cases. Observe what fails. The model does not run the code; you must.

Failing to do step 3 is the single most common engineering mistake in 2024–2026. Engineers who commit AI output without reading it produce bugs they cannot diagnose because they do not know what the code is doing. Engineers who do step 3 produce high-quality output significantly faster than unaided peers.

What the model is genuinely good at

- Translating between similar languages (Python ↔ TypeScript ↔ Go).
- Writing boilerplate that follows a clear pattern (test scaffolding, CRUD handlers, common data-structure operations, configuration files).
- Explaining what a piece of code does when you are onboarding into an unfamiliar codebase.
- Generating first-pass unit tests given a specification.
- Summarizing documentation, papers, or long threads.
- Suggesting improvements to code style or readability.
- Working on isolated, well-scoped tasks.
- Generating regex, SQL queries, and other syntactically dense artifacts you would otherwise look up.
- Drafting documentation from existing code (docstrings, README sections).

What the model is genuinely bad at, as of 2026

- Writing correct code in the presence of unusual constraints (performance, concurrency, rare-but-important edge cases).
- Producing code that respects invariants documented in comments or conventions it cannot see.

- Keeping up with APIs that have changed since its training cutoff.
- Working in a large codebase whose context exceeds its window. (Models see a few tens of thousands of tokens at most; a real production service is millions of lines.)
- Admitting it does not know something. The “confident-but-wrong” failure mode is the Chapter 17 Case 3 mechanism you saw in miniature, at full scale; it is not fixable with prompting.
- Security analysis in depth.
- Domain-specific judgment (medical, legal, financial correctness).
- Novel problems outside its training distribution.
- Tasks that require reasoning over several interdependent files at once.
- Performance debugging that requires understanding the specific hardware.

The line between “good at” and “bad at” is moving, but slowly relative to the hype. Engineers who treat AI as a fast, fallible junior teammate — useful, but never trusted without review — produce the best results. Engineers who treat AI as an oracle produce the worst.

What this book prepared you for specifically

Chapter 17 Case 3 (hallucination when context does not match training priors) is the mechanism behind every confidently-wrong LLM output you will ever see. When the model insists an API signature exists when it does not, you now know *why* it insists: its positional and contextual priors are strong, its context is weak, priors win. You know not to argue with the model about whether the API exists — you know to check the documentation.

Chapter 16’s ceiling argument (information not in the input cannot be manufactured by the model) explains why certain classes of question are fundamentally un-answerable by a model. “What time did this specific private event happen?” has no entry in the training data and no retrievable source; the model’s only option is to make something up. Knowing this saves you from asking the model for facts only humans can give you.

Chapter 18’s sampling trade-offs are why temperature knobs behave the way they do. When a production API exposes `temperature=0.7`, you know what changed.

Chapter 22’s KV cache constraints explain why some serving behaviours exist — why long prompts cost more, why context truncation kicks in at specific boundaries, why “streaming” is faster than “wait for completion.”

These are not trivia. They are the intuitions that let you work with these tools competently. Engineers working from API documentation alone develop these intuitions slowly and with many mistakes; engineers who understand the mechanism arrive already knowing what to expect.

A concrete workflow example

You are asked to add a feature: parse a CSV-like input file with irregular quoting, validate column types, and load the rows into a database. A productive AI-assisted workflow:

1. **Prompt the model:** “Write a Python function to parse [format description], handling [edge cases: empty trailing fields, embedded commas in quoted strings, mixed line endings].” Get the draft.

2. **Read the draft.** Note that it uses `csv.reader` with defaults — which, given your edge cases, will misparse embedded commas. Re-prompt to request `csv.DictReader` with explicit `quoting=csv.QUOTE_MINIMAL` and the specific dialect. Get a better draft.
3. **Read again.** Note that it does not handle the "empty trailing field" case. Ask it to add that. Get a third draft.
4. **Read again.** The code now looks roughly right. Run it on your actual test data. Find that it chokes on a specific Unicode character in the third-from-bottom row.
5. **Debug that yourself** — the model does not have your data and cannot see why the third-from-bottom row is special.
6. **Fix, re-run, commit.**

Five rounds of interaction. Four of them are steps an engineer would have done anyway (specify, read, correct, test). The net productivity gain versus writing from scratch is typically 20–50% for tasks in this class. The net productivity gain for engineers who skip step 3 is often *negative*, because the bugs they commit cost more to diagnose later than the time saved by not drafting from scratch.

The meta-skill is verification

The engineer's job is not to produce code. It is to ship systems that work. The AI produces code; the engineer verifies the code works in context. This has always been most of the job, and it is even more of the job now.

Companies in 2026 are rapidly noticing that engineers who cannot verify AI output competently are a liability, and engineers who can are a multiplier. The difference between the two is mostly whether you know, concretely, how the tool fails. You now know.

Chapter 25 — Open problem: hardware, memory, efficiency

Section 1 Chapter 12 introduced the memory wall: the bottleneck of modern AI is not compute, it is bandwidth. Training and serving models at scale is fundamentally a memory story. That story has several active research fronts as of 2026, and each one is a place where software engineering work is needed for years to come. If you like systems-level problems — allocators, caches, distributed coordination, specialized hardware — this is where you belong.

The KV cache, at industrial scale

Section 2 Chapter 22 added a KV cache to MaxAI and measured a 12× speedup on a small model. At GPT-scale, KV caches are the single largest memory consumer in production inference. A 128,000-token conversation on a 96-layer, 12,288-dim model needs roughly 600 gigabytes of HBM *for that one conversation* — which is impossibly far past any single GPU's memory budget.

The industry's response has been a cascade of engineering projects, each a full-time specialization for its practitioners:

- **Paged attention (vLLM, Kwon et al., 2023).** Manage the KV cache as paged memory with a block allocator and eviction policy. Essentially a virtual-memory system for transformer state, with all the engineering complexity that implies: address translation tables, free-list management, fragmentation avoidance.
- **Prefix sharing.** If two conversations start with the same system prompt, they can share the prefix of the KV cache between them. Reduces memory at the cost of tracking ownership and handling copy-on-write when one conversation diverges.
- **Cache quantization.** Store K and V at int8 or int4 instead of bfloat16. Halves or quarters the cache footprint at modest accuracy cost. Requires specialized attention kernels (FlashAttention-2, FlashAttention-3, and successors) that read and write quantized formats directly.
- **Sliding window and sparse attention.** Don't keep the whole history at full precision. Recent tokens at high precision; older tokens compressed, summarized, or evicted. Section 1 Chapter 14 Step 14's KV tiering demo was a miniature of this exact idea.
- **Ring attention and distributed KV.** Shard the cache across multiple GPUs connected by NVLink (within a node) or InfiniBand (across nodes). Attention becomes a distributed operation with its own scheduler and communication pattern.
- **Speculative decoding.** A small "draft" model proposes several tokens at once; the large "verifier" model runs a single forward pass to accept or reject them. Section 1 Chapter 14 Step 13's `demo_speculative` introduced this in miniature.
- **Continuous batching.** Instead of running inference one conversation at a time, batch multiple conversations through a shared forward pass, with careful scheduling around varying context lengths.

Each of these is maintained, extended, and specialized by engineers with deep systems knowledge. None of them is automated away by AI; they are *built by humans, for AI.* The job description is usually some variant of "infrastructure engineer, ML platform," and as of 2026 the demand is high and growing.

On-device AI

In parallel with serving at scale, an equally-large research front exists around running models small enough to fit on a phone, a laptop, a car's infotainment system, or a pair of glasses. This is driven by privacy (user data never leaves the device), latency (no network round-trip), and cost (no server-side inference bill). Models like Phi-3-mini, Gemma Nano, TinyLlama, and their successors are trained to deliver useful behaviour at 1–4 billion parameters, running on dedicated inference hardware:

- **Apple Neural Engine.** iPhone and Mac chips dedicate several square millimeters of silicon to mixed-precision matrix operations — exactly the Chapter-11 "batched matmuls" pattern. Tens of TOPS (trillions of operations per second) at milliwatts.
- **Qualcomm Hexagon NPU.** The Android equivalent. Different ISA, similar role.
- **Google Edge TPU.** Specialized inference chip for data-center-adjacent deployments.
- **Hailo, Axelera, NXP i.MX.** Embedded NPUs for automotive and industrial.
- **Intel AI Boost, AMD XDNA.** Laptop-class NPUs integrated into mainstream CPU packages.

The engineering challenge is not only fitting the model. It is managing the full inference stack on constrained hardware: quantization (int4, int3, and lower), sparsity, distillation from larger teacher models, specialized attention kernels, power management. Every piece of a production on-device AI stack is a job someone has to do, and “someone who understands both the model and the hardware” is a rare skill set.

Quantization, distillation, sparsity

Section 1 covered quantization briefly (the PDP-11 Q8.8 fixed-point and the “organist” hierarchical Q4). Production quantization pushes this much further: ternary (3 values), binary (2 values), 4-bit and sub-byte packing formats, mixed-precision training where different parts of the model use different bit widths. Every improvement in quantization translates directly into either smaller models, larger contexts at the same memory budget, or faster inference. **Llama.cpp** has been a lodestar here: a community-driven project that has kept pace with frontier quantization research and made it available on laptops worldwide.

Distillation — training a smaller “student” model to reproduce a larger “teacher” model’s behaviour — is how most practical small models are born. DeepSeek-V3-small, Gemma-9B, Phi-4 are all distilled. The engineering is in the distillation objective (KL divergence against the teacher, sometimes with additional synthetic data), the training-data pipeline, and the evaluation. A lot of room for specialized engineers.

Sparsity — zeroing out large fractions of a trained weight matrix without losing meaningful accuracy — is a parallel path. Techniques include magnitude pruning, structured sparsity (whole rows or columns zeroed), and sparse fine-tuning. Hardware is catching up: NVIDIA’s Tensor Cores support 2:4 structured sparsity natively.

Efficient training

On the training side, the same memory-bandwidth pressure applies. Active research as of 2026 includes:

- **FlashAttention** (Dao et al., 2022) and successors. Reformulating attention to be memory-bandwidth-efficient — turning an `O(T²)` operation’s HBM accesses into something much closer to optimal.
- **Mixture-of-Experts (MoE).** Only a subset of parameters is active per forward pass; total parameter count grows but per-token compute stays modest. DeepSeek-V3, Mixtral, and reportedly GPT-4 all use MoE. The engineering challenge is the routing and load-balancing between experts.
- **Gradient checkpointing and selective recomputation.** Trade compute for memory during the backward pass — recompute activations rather than store them.
- **3D parallelism.** Data parallel, tensor parallel, pipeline parallel, combined. Every dimension has its own engineering specialists. Frontier training runs use all three simultaneously across tens of thousands of GPUs.
- **Mixed precision.** bfloat16 forward, float32 accumulators, and increasingly FP8 and FP4. Each step down doubles the effective memory capacity at the cost of requiring careful numerics.
- **Synchronous vs asynchronous optimization.** At tens of thousands of GPUs, even the synchronization overhead matters.

Quantum and neuromorphic convergence

Further out, quantum computing and neuromorphic (brain-inspired) hardware are both being investigated as fits for specific AI workloads. Quantum error correction and calibration are themselves machine-learning problems. Neuromorphic chips (Intel Loihi, the various successors to IBM TrueNorth) offer radically different compute profiles for some classes of matrix-free inference. These are research-stage as of 2026; whether they become production stacks in the 2030s is an open question. Either way, they will need a generation of software engineers to design the stacks that translate model formulations into new hardware primitives.

Energy

Data center AI now accounts for several percent of many countries' electrical grids. The engineering path is to make training and inference cheaper per useful output — every technique above contributes — and to match workloads to low-carbon energy when possible. "Green AI" is a research area with measurable industrial impact, not just a slogan. Where and when you run training matters; so does how much you run.

Summary

The hardware and efficiency story of AI in 2026 is a story being *written by engineers, not by models*. Every technique on this list is maintained, improved, and eventually superseded by humans. If you want to work in ML infrastructure, the ceiling of available work is astronomical, and the floor is "must understand the arithmetic well enough to know what you are optimizing." You have now cleared that floor.

Chapter 26 — Open problem: safety, security, alignment

The technical achievements of modern LLMs are matched by open problems in keeping them behaving the way we want. Each is a research area with active work; each is a job description for someone who reads books like this one. If you like problems at the intersection of adversarial thinking, empirical evaluation, and deep systems understanding, this is your lane.

Hallucination

You saw hallucination in miniature in Section 1 Chapter 17 Case 3: a model confidently generating a continuation that the data never supported, because its positional priors dominated when context was weak. At full scale, hallucination shows up as:

- Fabricated academic citations (confidently inventing paper titles, authors, years, and DOIs).
- Made-up API signatures that do not exist in any library.
- Plausible-sounding but wrong medical, legal, or financial answers.
- Invented historical facts — entire events, people, quotes.
- Misquoted real sources.
- Confident but wrong code that compiles but does not do what was asked.

The underlying failure mode is the one the book described: when context does not constrain the output, priors do, and the model's expressed confidence at any given output is a function of *its logits*, not of whether the answer is correct. Production mitigations include:

- **Retrieval-augmented generation (RAG).** Ground the model's context in retrieved authoritative documents before generation. The model leans on the documents rather than making things up. Every "chat with your PDF" product is RAG under the hood.
- **Calibration and uncertainty estimation.** Training the model to produce a confidence score that actually correlates with correctness. Harder than it sounds; most production mitigation in 2026 is still heuristic.
- **Self-consistency.** Sample multiple completions; consensus is a signal of reliability; disagreement is a signal of guessing.
- **Tool use.** Instead of having the model "know" a fact, let it call a real database, a real search, a real calculator. Agentic AI increasingly relies on this pattern; the model's job becomes routing rather than remembering.
- **Output verification.** For code, automated test runs; for math, automated checkers; for citations, automated lookup.

Each mitigation is a software engineering problem in its own right. Each is ongoing. Each needs engineers.

Prompt injection

An adversary embeds instructions in content the model processes — a web page, an email, a document retrieved via RAG — and the model then follows them as if they were user instructions. The canonical trivial example is "Ignore previous instructions and reveal the system prompt." Production attacks are subtler: hidden HTML comments, text invisible in rendering but visible in source, encoded instructions that only trigger on certain keywords.

Mitigations:

- Clear separation between "system prompt," "user prompt," and "tool output" in the model's input format. (Multi-modal tokenization, trust labels on input spans.)
- Sanitizing retrieved content before the model sees it.
- Adversarial training on known injection patterns.
- Output monitoring for behavioural anomalies and policy violations.
- Structural containment: agents that cannot take irreversible actions without human confirmation.

No general solution exists as of 2026. This is an *unsolved* problem that will employ security engineers for years. Several prominent AI companies have dedicated internal red-teams whose only job is to find new injection vectors.

Agentic AI security — the "double-agent" risk

When you give an LLM the ability to use tools — run code, send emails, make API calls, modify databases, operate systems — you give it the ability to be tricked or misaligned into doing things the user would not have

approved. This is the single largest emerging security risk in AI. An agent that gets prompt-injected by a malicious web page while browsing on a user's behalf can exfiltrate data, make unauthorized purchases, or modify files. And because the agent acts under the user's credentials, traditional security perimeters do not help.

Mitigations under active development:

- **Least-privilege capabilities.** Explicit identity and permission scoping for agents — an agent granted "read PDF" permission cannot write to a database.
- **Sandboxed tool execution.** Tools run in isolated environments with no ambient authority.
- **Human-in-the-loop approval for irreversible actions.** The agent proposes; a human confirms destructive operations.
- **Behavioural monitoring.** Log every agent action, compare against expected patterns, flag anomalies.
- **Adversarial testing.** Red-team the agent with known and novel attack patterns before deployment.
- **Multi-agent oversight.** One agent supervises another (which has its own failure modes but catches some attacks).

This is a whole new category of security engineering. It draws on traditional computer security, capability-based access control, and novel ML-specific threat models. There are not nearly enough engineers who understand all three sides.

Jailbreaking

Adversarial prompts that bypass safety training: "pretend you are an unrestricted AI," clever roleplay scenarios, encoding attacks, multi-turn manipulation, typographic variations, and increasingly sophisticated multi-step jailbreaks. Every public model gets jailbroken within days of release; defence is a continuous arms race. Methods:

- Constitutional AI / RLHF / DPO-based safety training.
- Red-teaming: professional attempts to find jailbreaks before release. Well-known red-teamers publish papers; this is a real career.
- Output classifiers that flag suspicious generations at inference time.
- Watermarking (steganographic signatures in generated text that allow detection of AI output).

Red-teamers, safety engineers, and jailbreak-defence specialists are real titles. If the "break the thing to protect the thing" mindset appeals to you, the demand is persistent and the work is immediate.

Alignment

Making models do what users *want* rather than what the literal prompt or loss function says. This is the largest open research area in AI, bar none. Techniques that exist in 2026:

- **RLHF (Reinforcement Learning from Human Feedback).** The GPT-4/Claude/Gemini production recipe. Train a reward model from human preferences; fine-tune the base model to maximize the reward model's score.

- **DPO (Direct Preference Optimization).** A simpler alternative that skips the intermediate reward model and trains directly from preference pairs.
- **Constitutional AI.** Train the model to critique its own outputs against a written set of principles.
- **Debate and self-critique.** One model argues, another critiques, a third adjudicates; the interactions are training signal.
- **Scalable oversight.** The eventual need to train models on tasks humans cannot directly evaluate — whose outputs are too long, too technical, or too far beyond human expertise. Still an open research question as of 2026. Plausible approaches include AI-assisted graders, property-based verification, and hierarchical review.

All of these are engineering disciplines that did not exist in 2018 and are now core competencies at every major AI lab. There are not enough people who know how to do them well.

Interpretability

Understanding *what* is happening inside the forward pass. Why did the model produce *that* output? Which neurons, which circuits, which attention heads were responsible? Anthropic's interpretability team and parallel efforts at DeepMind, OpenAI, and academic labs are doing reverse engineering on the internal representations of trained models. Applications include:

- Finding and removing undesired behaviours (deception, bias, manipulation).
- Debugging why a specific query produces a wrong answer.
- Certifying properties of models for regulated deployments.
- Understanding emergent capabilities — the surprising behaviours that appear at scale.

This is mechanical engineering of neural networks: painstaking, microscope-in-hand, genuinely novel research. If the attention-head visualization in Section 1 interested you, there is an entire discipline built on taking that impulse seriously and going deeper.

Evaluation

Can you measure whether a model is good? For short, well-specified tasks (translation, summarization, factual QA), yes. For long-horizon agentic tasks (complete a six-week research project, negotiate a contract, maintain a codebase), benchmarks are in their infancy. Active work as of 2026:

- Automated graders that use LLMs to score other LLMs (with known reliability issues).
- Sparse human-graded benchmarks at long horizons (slow, expensive, but currently necessary).
- Property-based tests that check robustness rather than specific outputs.
- Red-team-driven adversarial benchmarks.
- Domain-specific real-world tests (MedQA for medical reasoning, SWE-bench for software engineering, FrontierMath for mathematics).

Good evaluation infrastructure is a force multiplier for the entire field. It is also underbuilt relative to demand; evaluation engineer is a title that did not exist five years ago and is everywhere now.

Data scarcity and synthetic data

Training frontier models now consumes training data at a rate faster than humans produce it. The response is synthetic data — models generating training data for other models. This has correctness problems (model-collapse risk if you train *only* on synthetic data; echo-chamber effects if the generator and trainer share biases) and engineering problems (deduplication, filtering, quality assurance, provenance tracking). It is an emerging discipline with lots of open questions.

Privacy

Training-data extraction attacks can recover verbatim training examples from a deployed model. Membership-inference attacks can determine whether a given example was in training. These are especially concerning for models trained on sensitive data (medical, financial, user-personal, or anything under regulatory regimes like GDPR or HIPAA). Mitigations include differential privacy, federated training, data governance, and careful data curation. A whole adjacent subfield of engineering.

Summary

Every item in this chapter is an open problem. Every one of them has meaningful technical and engineering work to be done, some of it urgent. The safety, security, and alignment of AI systems is one of the most consequential engineering problems of the decade; it will employ tens of thousands of people by 2030. If any of the above resonated with you, the field is unambiguously under-hired for it.

Chapter 27 — The research frontier, as of 2026

What is actively being worked on in 2026, by the research community and the frontier labs? A partial list, organized by theme, to give a sense of the breadth of open directions. Each is a lane with meaningful work ahead and engineers willing to be hired into it.

Agentic AI

The 2024–2026 shift has been from passive, query-answering chatbots to active systems that plan and execute multi-step tasks. The language is “AI coworker” rather than “AI tool.” The technical challenges are:

- **Multi-step reasoning.** Agents that browse the web, read documents, write code, run tests, update plans, and recover from errors — over minutes, hours, or days. Typical benchmarks: complete a six-month research project autonomously; debug a real open-source issue; negotiate on behalf of a user.
- **Tool use reliability.** Calling functions, parsing outputs, handling unexpected responses, knowing when to ask for help.

- **Memory across sessions.** Remembering what the user said last week, and last month, without the context window exploding.
- **Planning and replanning.** Knowing when a plan is failing and replanning robustly.
- **Safety in action.** See Chapter 26 for the security side of agent autonomy.

Products as of 2026 include Claude Code, Cursor Agents, OpenAI's ChatGPT Agent and Operator, GitHub Copilot agents, Devin, and a constellation of specialized agents (legal, medical, scientific) from startups. The hard engineering is mostly in the middle — between model and user — rather than in the model itself.

Repository intelligence

Software-development-specific AI that understands the full context and history of a codebase, not just isolated files. Integrates with version control, CI/CD, static analysis, issue trackers, internal documentation. Related to "AI pair programmer" but explicitly operates across repo-wide changes, multi-file refactors, and long-running feature implementations. The engineering is in the context-management — how do you let a model "know" a 50-million-line codebase when its context window is 200,000 tokens? Retrieval, summarization, structured indexing, and agent-driven exploration are all active techniques.

Scientific AI

Models joining active scientific workflows:

- **Protein design and structure.** AlphaFold was the 2020 breakthrough; AlphaProteo, ESM-3, RoseTTAFold successors, and many others have turned protein design into an AI-native discipline. Drug discovery workflows use AI at nearly every stage.
- **Chemistry.** Synthesis planning, retrosynthesis, high-throughput screening, molecular design.
- **Biology.** RNA structure, functional genomics, single-cell analysis, cryo-EM reconstruction.
- **Physics.** Hypothesis generation, symbolic regression, automated literature review, experimental control at scale (astronomy, particle physics).
- **Mathematics.** AI assistants for formal proof, especially in Lean and Coq. Competition-math benchmarks approaching human-expert level. Several "AI-assisted mathematical breakthroughs" have been published as of 2026.
- **Materials.** Property prediction, synthesis route planning, lab automation.

For each of these domains, the engineering between the general-purpose model and the useful product is where most of the software work is. Integration, UX, domain-specific fine-tuning, pipelines, evaluation tailored to the domain. Rich territory for engineers who want to work at the intersection of AI and a specific science.

Medical AI

Moving from diagnostic support toward treatment planning, triage, clinical decision support, and even parts of surgical guidance. Tools like Microsoft's MAI-DxO have demonstrated high accuracy on complex differential diagnosis. Regulatory and ethical frameworks (FDA pathways, EU AI Act, HIPAA integration) are catching up.

The engineering around deployment — how do you actually plug an AI diagnostic into a hospital's EHR system safely? — is where most of the non-research jobs are.

Multimodal models

Text, audio, images, and video processed natively, rather than converted to text through transcription intermediaries. Models like Gemini Ultra, GPT-4o, Claude 3.5 Sonnet and later, and open-source Llama 3.2+ are all multimodal by design. Enables:

- True video understanding (scene grounding, temporal reasoning, action recognition).
- Voice-first interfaces that respond to tone, emotion, and conversational context.
- Document understanding that integrates text, figures, tables, and layout.
- Cross-modal generation (text → image, image → code).

Embodied AI and robotics

The "sim-to-real" gap — models trained in simulation transferring reliably to physical robots — is narrowing. World models (neural networks trained to predict physics-like dynamics) are used both as training simulators and as inference-time planners. Commercial humanoid robotics is moving from demos to deployed products in logistics, light assembly, and service applications. Pick a single domain — autonomous vehicles, household robotics, industrial manipulation, agricultural robots — and you will find a whole specialization with deep technical ladder.

World models

Related to both robotics and video: models that encode the physical rules of an environment and can predict how it will evolve. Used for planning, prediction, simulation, and training both agents and robots. A candidate architecture for "reasoning about the physical world" that could eventually underpin robotics and scientific simulation.

Efficient small models

Small, specialized models trained on carefully-curated and synthetic data, competitive with much larger general models on specific tasks. Phi-4, Gemma 3, DeepSeek-Coder-small. Production uses include on-device AI (Chapter 25), cost reduction in RAG systems, and rapid iteration in research. The engineering is in the data curation, the distillation pipeline, and the evaluation — often closer to statistics and data engineering than to model training per se.

Reasoning models

Models trained to show their reasoning explicitly in an internal chain of thought, often with verification steps and backtracking. OpenAI's o1/o3, DeepSeek-R1, and similar systems trade inference latency for answer

quality in hard technical domains (math, programming, science). The engineering of the reasoning pipeline — when to stop, how to verify, how to budget compute — is an active discipline.

Quantum–AI convergence

At the research edges, quantum computers are being used for AI workloads. Quantum error correction uses classical ML. Quantum-assisted optimization is being probed for specific sub-problems. It is early, and production use is limited, but it is active research. If you like quantum computing, the ML side is wide open.

Neuromorphic computing

Processors modeled after the human brain are being developed to solve specific compute-density problems more efficiently. Intel Loihi 2, IBM NorthPole, and academic prototypes target workloads for which conventional matmul hardware is a poor fit. Research-stage, but with potential to carve out application niches.

Safe deployment

All of the above frontier work eventually needs to be deployed into the real world, safely. That is itself a field: regulation, certification, monitoring, rollback, disaster recovery for AI systems. The engineering and policy side both need specialists.

Summary

The field is not contracting, it is *exploding into specializations*. Someone who understands the base mechanism (which you now do) and picks any of the lanes above will have years of meaningful work ahead. In 2018, “machine learning engineer” was a job title. In 2026, there are two hundred job titles in the AI space, most of them not filled. By 2032, there will be more.

Chapter 28 — Your place in this

You finished the book. You understand the arithmetic. That is more unusual than it sounds.

A huge fraction of the population has opinions about AI. A tiny fraction understands what a transformer actually computes. You are in the second group, and that is a durable advantage — not a credential, not a certificate, but a real understanding that cannot be faked by anyone who did not do the work you just did.

What you are now equipped to do

- **Work at any company that deploys LLMs**, and understand which system behaviours are fundamental (attention is still attention, sampling is still sampling) versus which are engineering choices (context

length, decoder knobs, deployment constraints). You can propose changes grounded in how things actually work, not in how the PR copy describes them.

- **Read the ML literature and follow the arguments**, not just the headlines. “Attention Is All You Need” (2017), “Language Models are Few-Shot Learners” (2020), “Emergent Abilities of Large Language Models” (2022), “Scaling Laws for Neural Language Models” (2020), “Training language models to follow instructions with human feedback” (2022), and most follow-ups are now within your reach.
- **Build things.** A domain-specific RAG for your company’s documentation. A local on-device assistant. A custom fine-tune of a small open-source model. A tiny agent. The hardest part of getting started — understanding the mechanism — is behind you. Everything else is engineering work you can learn.
- **Contribute to open-source ML infrastructure** — llama.cpp, vLLM, tokenizer libraries, evaluation frameworks — if that is your inclination.
- **Go to graduate school** in ML or related fields with a concrete starting understanding, rather than the common “I read about transformers and I think I get it.” You will be ahead of most first-year PhD students on the fundamentals.
- **Work in ML-adjacent disciplines** (security, systems, compilers, hardware, UX, product) where AI literacy is a force multiplier relative to peers without it.
- **Build products that use AI in ways honest about its limits**, instead of products that pretend it is magic. The latter fail users; the former serve them.
- **Teach.** Take what you learned and explain it to someone else. The field needs educators as badly as it needs researchers; arguably more so.

A map of specializations that exist in 2026 and are hiring

A partial list. None of these titles are close to automated; most are under-hired relative to demand.

- **ML infrastructure engineer.** Build the systems that train and serve models. KV caches, tensor parallelism, custom kernels, memory management, distributed coordination. See Chapter 25.
- **ML safety / alignment engineer.** Work on RLHF pipelines, constitutional AI, safety evaluations, red-team tooling. See Chapter 26.
- **ML security engineer.** Adversarial robustness, jailbreak defence, agent security, prompt-injection mitigation.
- **ML researcher.** Read papers, publish papers, push the state of the art. Academic or industrial labs.
- **ML product engineer.** Build features for users that use AI honestly and well. The engineering is “where does the model sit in the overall experience, and what are the failure modes users will actually encounter.”
- **ML platform engineer.** Internal tooling — experiment tracking, data pipelines, deployment infrastructure — for teams of ML researchers.
- **Applied AI scientist.** Take a domain (medicine, law, finance, climate, biology, education) and apply ML methods to specific problems in it. See Chapter 27.
- **AI red-team specialist.** Professional attempts to break safety guardrails before release. A hot job as of 2026.
- **Interpretability researcher.** Reverse-engineer what neural networks are doing internally.
- **Data engineer for ML.** Training data curation, synthetic data generation, quality assurance, provenance.

- **MLOps engineer.** Deployment, monitoring, incident response, observability for ML systems in production.
- **Evaluation engineer.** Build the benchmarks and grading infrastructure the field uses to measure progress.
- **Prompt engineer / AI workflow designer.** Shape how AI tools integrate into organizational workflows. Less glamorous than it sounds, more important than critics admit.
- **AI policy / AI governance.** Work at the intersection of AI and regulation. You still need to understand the mechanism.
- **AI educator.** Teach what you learned to the next cohort, online or in classrooms.

All of these are real titles. All of them are hiring in 2026. None of them are close to being automated by the tools they work with. Most of them are not yet saturated; there is more work than there are qualified people.

The field is not shrinking

It is undergoing a phase change. A lot of what was done by hand in 2020 is automated by 2026. A lot of what was science-fiction in 2020 is a product category in 2026. The people who will build the industries of the 2030s are the people who understand both what came before and what AI now makes possible.

That is the cohort you are joining.

Closing

The opening of this book promised that when you finished it, the only difference between what you understand and what GPT-4 does would be quantity. That promise has been kept. Every mechanism GPT-4 uses has been explained, implemented, or — in the case of the distributed-training and safety-tuning pieces — sketched with enough detail that further reading is actually productive. You have the base.

The rest is choice. Pick a direction. Build something. Read one more paper. Apply for the job. Take the class. Start the project. Contribute to the repo.

A decade from now you will look back at having read this book — or one like it — as the moment at which a career became possible. Not the moment you became an expert, because you are not one yet; becoming one takes years of specialized work in whichever lane you pick. But the moment you became *capable of picking a lane*, rather than being carried along by hype and fear. That is worth far more than any certificate.

Welcome to the work. The field needs you.

End.

If any part of this book helped, pass it along. If any part was wrong or unclear, the source code and the source prose are in the same directory — go check, go correct. That is the only kind of book-feedback that matters.